INSACC ENTERPRISE ERP DOCUMENTATION

InsAcc Enterprise Documentation Suite

Complete Master Enterprise Reference Manual

InsAcc Enterprise Asset & Investment Platform v2.0.0 Target Server

Single Source of Truth Specification — Official Installation Manual

Release Date: July 22, 2026 | Status: Official Publication

Classification: Commercial Enterprise Documentation

InsAcc Enterprise ERP Documentation Suite

Complete Master Edition

title: "InsAcc Enterprise ERP - PostgreSQL 17 Database Server Step-by-Step Installation Guide" document_id: "POSTGRESQL_DATABASE_SERVER_STEP_BY_STEP_GUIDE.md" version: "1.0.0" release_date: "2026-07-22" app_version: "v2.0.0 Target Server" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Official Step-by-Step Server Setup Guide" classification: "Commercial Enterprise Documentation"

InsAcc Enterprise ERP Platform

Complete Step-by-Step PostgreSQL 17 Database Server Setup Manual `[To Be Implemented]`

> **Single Source of Truth Reference**: All server provisioning rules, Linux commands, PostgreSQL 17 configurations, memory tuning parameters, DDL schemas, and balance triggers defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade step-by-step database setup manual	Chief Architecture Review Board

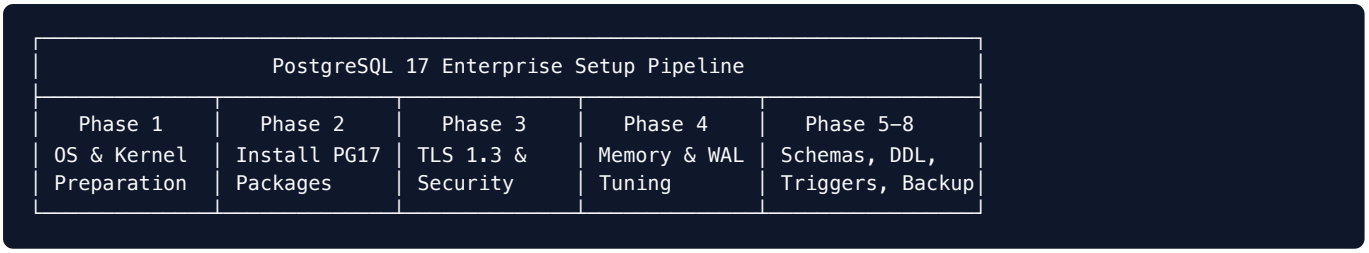
Table of Contents

---

1. Executive Summary & Architecture Overview

This document provides an absolute, step-by-step operational installation manual for deploying an enterprise-grade **PostgreSQL 17 Relational Database Server** for the InsAcc ERP platform.

Every command, configuration file, SQL script, and verification test is presented in chronological order with exact shell commands to copy and paste.



---

2. Prerequisites & Hardware Specifications

Recommended Target Hardware (Standard Enterprise Deployment):

- **Operating System**: Ubuntu 24.04 LTS (64-bit Server Edition) or RHEL 9
- **Processor (CPU)**: 4 vCPU / Cores (2.4 GHz+)
- **System Memory (RAM)**: 8 GB System Memory
- **Disk Storage**: 100 GB Enterprise NVMe SSD (RAID-1 Recommended)
- **Network Interface**: 1 Gbps / 10 Gbps Ethernet (Static IP Address: `10.0.4.50`)

---

3. Phase 1: Linux OS Environment Preparation

3.1 Update System Packages & Install Essential Utilities

Log in to the server via SSH as a sudo-privileged user and update all OS packages:

```
# Step 1: Update apt index and upgrade system packages
sudo apt-get update && sudo apt-get upgrade -y

# Step 2: Install essential system tools and utilities
sudo apt-get install -y \
  curl \
  gnupg \
  ca-certificates \
  lsb-release \
  ufw \
  htop \
  rsync \
  vim \
  tar \
  ufw \
  ufw-extras
```

3.2 Kernel Parameter Optimization (`sysctl.conf`)

Tune the Linux kernel for PostgreSQL high-throughput memory operations:

```
# Step 1: Create PostgreSQL kernel parameter configuration file
sudo bash -c 'cat << "EOF" > /etc/sysctl.d/99-postgresql.conf
# Shared Memory Segment Limits
kernel.shmmax = 18446744073709551615
kernel.shmall = 18446744073709551615

# Memory Overcommit Configuration for PostgreSQL
vm.overcommit_memory = 2
vm.overcommit_ratio = 80
vm.swappiness = 10

# Disk Write Buffer Flushing
vm.dirty_background_ratio = 3
vm.dirty_ratio = 10
EOF'

# Step 2: Apply the kernel parameters immediately
sudo sysctl -p /etc/sysctl.d/99-postgresql.conf
```

3.3 File Descriptor & User Limit Configuration (`limits.conf`)

Increase open file limits for the `postgres` user to support concurrent client connections:

```
# Step 1: Create PostgreSQL user limits configuration file
sudo bash -c 'cat << "EOF" > /etc/security/limits.d/99-postgres.conf
postgres soft nofile 65536
postgres hard nofile 65536
postgres soft nproc 4096
postgres hard nproc 4096
EOF'
```

4. Phase 2: Installing PostgreSQL 17 & Packages

4.1 Add Official PostgreSQL PGDG Repository

Add the official PostgreSQL Global Development Group (PGDG) APT repository:

```
# Step 1: Create directory for keyring
sudo install -d /etc/apt/keyrings

# Step 2: Download and install the official PostgreSQL signing key
curl -fsSL https://www.postgresql.org/media/keys/ACCC4CF8.asc | sudo gpg --dearmor -o /etc/apt/keyrings/postgr

# Step 3: Add PGDG repository to APT sources
echo "deb [signed-by=/etc/apt/keyrings/postgresql.gpg] http://apt.postgresql.org/pub/repos/apt $(lsb_release -
```

4.2 Install PostgreSQL 17 Engine & Extensions

Install the PostgreSQL 17 database engine, client binaries, and contrib modules:

```
# Step 1: Update package list with PGDG packages
sudo apt-get update

# Step 2: Install PostgreSQL 17 packages
sudo apt-get install -y postgresql-17 postgresql-contrib-17 postgresql-client-17
```

4.3 Verify Database Service Status

Verify that PostgreSQL 17 is running and enabled on boot:

```
# Step 1: Check service status
sudo systemctl status postgresql@17-main --no-pager

# Step 2: Enable automatic startup on system boot
sudo systemctl enable postgresql@17-main
```

5. Phase 3: Network Security & TLS 1.3 Encryption

5.1 Configure Network Address Listening (`postgresql.conf`)

Allow PostgreSQL to accept connections from the internal application network:

```
# Step 1: Update listen_addresses parameter in postgresql.conf
sudo sed -i "s/#listen_addresses = 'localhost'/listen_addresses = '*'/g" /etc/postgresql/17/main/postgresql.co
```

5.2 Host-Based Authentication Rules (`pg_hba.conf`)

Restrict network access in `/etc/postgresql/17/main/pg_hba.conf` to require **SCRAM-SHA-256 password authentication over SSL**:

```
# Step 1: Append secure hostssl rule to pg_hba.conf
sudo bash -c 'cat << "EOF" >> /etc/postgresql/17/main/pg_hba.conf

# InsAcc Enterprise Client Network Access Rule (SSL Required)
hostssl insacc_db       insacc_user     10.0.4.0/24             scram-sha-256
hostssl insacc_db       insacc_user     127.0.0.1/32            scram-sha-256
EOF'
```

5.3 Generate & Configure SSL/TLS 1.3 Certificates

Generate a self-signed 4096-bit RSA TLS certificate for testing, or deploy your enterprise CA certificate:

```
# Step 1: Generate self-signed TLS certificate and private key
sudo openssl req -new -x509 -days 3650 -nodes \
  -text -out /etc/ssl/certs/insacc_server.crt \
  -keyout /etc/ssl/private/insacc_server.key \
  -subj "/CN=db.insacc.internal/O=InsAcc Enterprise/OU=IT Operations"

# Step 2: Set strict ownership and permissions for the private key
sudo chown postgres:postgres /etc/ssl/certs/insacc_server.crt /etc/ssl/private/insacc_server.key
sudo chmod 600 /etc/ssl/private/insacc_server.key

# Step 3: Enable SSL in postgresql.conf
sudo sed -i "s/ssl = off/ssl = on/g" /etc/postgresql/17/main/postgresql.conf
sudo bash -c 'cat << "EOF" >> /etc/postgresql/17/main/postgresql.conf

# SSL Certificate Parameters
ssl_cert_file = '\''/etc/ssl/certs/insacc_server.crt'\''
ssl_key_file = '\''/etc/ssl/private/insacc_server.key'\''
ssl_min_protocol_version = '\''TLSv1.3'\''
EOF'
```

5.4 Configure Linux Firewall (UFW / Firewalld)

Open port `5432` only to the application subnet (`10.0.4.0/24`):

```
# Step 1: Allow SSH access (Port 22)
sudo ufw allow 22/tcp

# Step 2: Allow PostgreSQL traffic from enterprise app subnet only
sudo ufw allow from 10.0.4.0/24 to any port 5432 proto tcp

# Step 3: Enable firewall
sudo ufw --force enable
sudo ufw status verbose
```

6. Phase 4: Enterprise Performance & Memory Tuning

6.1 Configure Memory Allocation Parameters

Apply hardware-tuned memory settings in `/etc/postgresql/17/main/postgresql.conf` (Calculated for 8 GB RAM):

```
# Step 1: Append performance tuning parameters to postgresql.conf
sudo bash -c 'cat << "EOF" >> /etc/postgresql/17/main/postgresql.conf

# ========================================
# InsAcc Performance Tuning Parameters (8GB RAM)
# ========================================
shared_buffers = 2GB                  # 25% of System Memory
effective_cache_size = 6GB            # 75% of System Memory
maintenance_work_mem = 512MB          # Index build memory
work_mem = 32MB                       # Memory per query operation
password_encryption = scram-sha-256
EOF'
```

6.2 Configure Write-Ahead Log (WAL) & Checkpoints

```
sudo bash -c 'cat << "EOF" >> /etc/postgresql/17/main/postgresql.conf

# Write-Ahead Log (WAL) & Checkpoint Parameters
wal_level = replica
max_wal_size = 4GB
min_wal_size = 1GB
checkpoint_completion_target = 0.9
checkpoint_timeout = 15min
EOF'
```

6.3 Configure Parallel Worker Threads & SSD Planner Costs

```
sudo bash -c 'cat << "EOF" >> /etc/postgresql/17/main/postgresql.conf

# Parallel Query Processors & SSD Cost Tuning
max_worker_processes = 4
max_parallel_workers_per_gather = 2
max_parallel_workers = 4
random_page_cost = 1.1                # Fast SSD page access cost
EOF'
```

6.4 Restart PostgreSQL to Apply Configurations

Restart the PostgreSQL service to load all new memory, security, and network settings:

```
# Step 1: Restart PostgreSQL service
sudo systemctl restart postgresql@17-main

# Step 2: Confirm service is running cleanly
sudo systemctl status postgresql@17-main --no-pager
```

---

7. Phase 5: Database, User Roles & Schema Provisioning

7.1 Create Restricted Non-Root Role `insacc_user`

Log into PostgreSQL as the `postgres` superuser and create the application role:

```
sudo -u postgres psql -c "CREATE ROLE insacc_user WITH LOGIN PASSWORD 'SecureEnterprisePassword123!';"
```

7.2 Create Database `insacc_db`

Create the production database owned by `insacc_user`:

```
sudo -u postgres psql -c "CREATE DATABASE insacc_db OWNER insacc_user;"
sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE insacc_db TO insacc_user;"
```

7.3 Provision the 5 Enterprise Schemas

Create the 5 isolated domain schemas inside `insacc_db`:

```
sudo -u postgres psql -d insacc_db -c "
CREATE SCHEMA IF NOT EXISTS accounting AUTHORIZATION insacc_user;
CREATE SCHEMA IF NOT EXISTS investment AUTHORIZATION insacc_user;
CREATE SCHEMA IF NOT EXISTS property AUTHORIZATION insacc_user;
CREATE SCHEMA IF NOT EXISTS auth AUTHORIZATION insacc_user;
CREATE SCHEMA IF NOT EXISTS audit AUTHORIZATION insacc_user;
"
```

7.4 Execute Complete DDL Table Specifications

Deploy the production table DDL schemas using `psql`:

```
sudo -u postgres psql -d insacc_db << "EOF"
-- 1. ACCOUNTING SCHEMA TABLES
CREATE TABLE accounting.accounts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    code VARCHAR(20) NOT NULL UNIQUE,
    name VARCHAR(255) NOT NULL,
    type VARCHAR(50) NOT NULL CHECK (type IN ('Asset', 'Liability', 'Equity', 'Revenue', 'Expense')),
    parent_id UUID REFERENCES accounting.accounts(id),
    opening_balance NUMERIC(15, 2) NOT NULL DEFAULT 0.00,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE accounting.vouchers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    voucher_number VARCHAR(50) NOT NULL UNIQUE,
    voucher_type VARCHAR(20) NOT NULL CHECK (voucher_type IN ('Receipt', 'Payment', 'Journal')),
    voucher_date DATE NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'Draft' CHECK (status IN ('Draft', 'Approved', 'Posted', 'Cancelled',
    narration TEXT,
    created_by VARCHAR(100) NOT NULL,
    posted_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE accounting.voucher_lines (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    voucher_id UUID NOT NULL REFERENCES accounting.vouchers(id) ON DELETE CASCADE,
    account_id UUID NOT NULL REFERENCES accounting.accounts(id),
    line_type VARCHAR(10) NOT NULL CHECK (line_type IN ('Debit', 'Credit')),
    amount NUMERIC(15, 2) NOT NULL CHECK (amount > 0),
    memo TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- 2. INVESTMENT SCHEMA TABLES
CREATE TABLE investment.investments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    asset_name VARCHAR(255) NOT NULL,
    asset_type VARCHAR(50) NOT NULL CHECK (asset_type IN ('Gold', 'Silver', 'Stocks', 'Bonds', 'Mutual Funds',
    quantity NUMERIC(18, 6) NOT NULL CHECK (quantity > 0),
    purchase_value NUMERIC(15, 2) NOT NULL CHECK (purchase_value >= 0),
    current_price NUMERIC(15, 2) NOT NULL CHECK (current_price >= 0),
    buyer VARCHAR(255),
    notes TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- 3. PROPERTY SCHEMA TABLES
CREATE TABLE property.buildings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    address TEXT NOT NULL,
    total_units INT NOT NULL DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE property.units (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    building_id UUID NOT NULL REFERENCES property.buildings(id) ON DELETE CASCADE,
    unit_number VARCHAR(50) NOT NULL,
    unit_type VARCHAR(50) NOT NULL,
    annual_rent NUMERIC(15, 2) NOT NULL DEFAULT 0.00,
    status VARCHAR(30) NOT NULL DEFAULT 'Vacant' CHECK (status IN ('Vacant', 'Occupied', 'Under Maintenance'))
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE property.tenants (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    national_id VARCHAR(100) NOT NULL,
    phone VARCHAR(50) NOT NULL,
    email VARCHAR(255) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE property.leases (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    unit_id UUID NOT NULL REFERENCES property.units(id),
    tenant_id UUID NOT NULL REFERENCES property.tenants(id),
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    annual_rent NUMERIC(15, 2) NOT NULL,
    payment_frequency VARCHAR(30) NOT NULL CHECK (payment_frequency IN ('Annual', 'Semi-Annual', 'Quarterly',
    security_deposit NUMERIC(15, 2) NOT NULL DEFAULT 0.00,
    status VARCHAR(20) NOT NULL DEFAULT 'Active',
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE property.pdc_cheques (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    lease_id UUID NOT NULL REFERENCES property.leases(id) ON DELETE CASCADE,
    cheque_number VARCHAR(50) NOT NULL,
    bank_name VARCHAR(255) NOT NULL,
    amount NUMERIC(15, 2) NOT NULL,
    due_date DATE NOT NULL,
    status VARCHAR(30) NOT NULL DEFAULT 'Received' CHECK (status IN ('Received', 'Deposited', 'Cleared', 'Boun
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Grant privileges to insacc_user
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA accounting TO insacc_user;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA investment TO insacc_user;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA property TO insacc_user;
EOF
```

8. Phase 6: Deploy Double-Entry Balance Triggers

8.1 Create PL/pgSQL Balance Validation Function

Deploy the PL/pgSQL trigger function that calculates total debits and credits and throws an exception if $\sum D \neq \sum C$:

```
sudo -u postgres psql -d insacc_db << "EOF"
CREATE OR REPLACE FUNCTION accounting.fn_validate_voucher_balance()
RETURNS TRIGGER AS $$
DECLARE
    v_total_debit NUMERIC(15,2);
    v_total_credit NUMERIC(15,2);
BEGIN
    -- Sum Debits and Credits for lines attached to the voucher
    SELECT
        COALESCE(SUM(CASE WHEN line_type = 'Debit' THEN amount ELSE 0 END), 0),
        COALESCE(SUM(CASE WHEN line_type = 'Credit' THEN amount ELSE 0 END), 0)
    INTO v_total_debit, v_total_credit
    FROM accounting.voucher_lines
    WHERE voucher_id = NEW.id;

    -- Enforce balance equality with floating point tolerance (< 0.001)
    IF ABS(v_total_debit - v_total_credit) >= 0.001 THEN
        RAISE EXCEPTION 'Voucher Posting Rejected: Debit sum (%) does not equal Credit sum (%)',
            v_total_debit, v_total_credit;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
EOF
```

8.2 Attach Trigger to Voucher Status Updates

Attach the trigger to fire whenever a voucher's status transitions to `Posted`:

```
sudo -u postgres psql -d insacc_db << "EOF"
CREATE TRIGGER trg_validate_voucher_posting
BEFORE UPDATE OF status ON accounting.vouchers
FOR EACH ROW
WHEN (NEW.status = 'Posted')
EXECUTE FUNCTION accounting.fn_validate_voucher_balance();
EOF
```

9. Phase 7: Automated Daily Backups & WAL Archiving

9.1 Create Automated `pg_dump` Script

Create the backup shell script at `/usr/local/bin/insacc_db_backup.sh`:

```
sudo bash -c 'cat << "EOF" > /usr/local/bin/insacc_db_backup.sh
#!/usr/bin/env bash
# InsAcc Production Database Daily Backup Script

set -euo pipefail

BACKUP_DIR="/var/backups/postgresql"
TIMESTAMP=$(date +'%Y-%m-%d_%H%M%S')
BACKUP_FILE="${BACKUP_DIR}/insacc_db_${TIMESTAMP}.dump"

mkdir -p ${BACKUP_DIR}

echo "[INFO] Starting compressed database backup to ${BACKUP_FILE}..."
PGPASSWORD="SecureEnterprisePassword123!" pg_dump -h localhost -U insacc_user -F c -b -v -f "${BACKUP_FILE}" i

# Retention: Remove dumps older than 30 days
find ${BACKUP_DIR} -type f -name "insacc_db_*.dump" -mtime +30 -delete

echo "[SUCCESS] Backup completed: ${BACKUP_FILE}"
EOF'

# Make script executable
sudo chmod +x /usr/local/bin/insacc_db_backup.sh
```

9.2 Schedule Daily Cron Job Execution

Schedule the backup script to run automatically every night at 02:00 AM:

```
# Step 1: Append cron job entry to root crontab
sudo bash -c '(crontab -l 2>/dev/null; echo "0 2 * * * /usr/local/bin/insacc_db_backup.sh >> /var/log/insacc_b
```

9.3 Configure WAL Archiving for Point-in-Time Recovery (PITR)

```
# Step 1: Create WAL archive directory
sudo mkdir -p /var/lib/postgresql/wal_archive
sudo chown postgres:postgres /var/lib/postgresql/wal_archive

# Step 2: Configure archiving parameters in postgresql.conf
sudo bash -c 'cat << "EOF" >> /etc/postgresql/17/main/postgresql.conf

# WAL Archiving for Point-In-Time Recovery
archive_mode = on
archive_command = '\''test ! -f /var/lib/postgresql/wal_archive/%f && cp %p /var/lib/postgresql/wal_archive/%f
EOF'

# Step 3: Reload PostgreSQL configuration
sudo systemctl reload postgresql@17-main
```

10. Phase 8: End-to-End Verification & Validation

10.1 Verify Remote Database Connectivity

Test SSL database connection using `psql`:

```
PGPASSWORD="SecureEnterprisePassword123!" psql "sslmode=require host=127.0.0.1 dbname=insacc_db user=insacc_us
```

Expected Verification Output:

```
 current_database |  current_user  |                          version
------------------+----------------+------------------------------------------------------------
 insacc_db        | insacc_user    | PostgreSQL 17.0 on x86_64-pc-linux-gnu ...
(1 row)
```

10.2 Verify Double-Entry Balance Trigger Enforcement

Test Case A: Balanced Voucher (Should Succeed)

```
-- Connect to insacc_db
PGPASSWORD="SecureEnterprisePassword123!" psql -h 127.0.0.1 -U insacc_user -d insacc_db << "EOF"
-- Insert test accounts
INSERT INTO accounting.accounts (id, code, name, type) VALUES
  ('11111111-1111-1111-1111-111111111111', '1120.001', 'Test Bank', 'Asset'),
  ('22222222-2222-2222-2222-222222222222', '4120', 'Test Revenue', 'Revenue');

-- Insert voucher header
INSERT INTO accounting.vouchers (id, voucher_number, voucher_type, voucher_date, status, narration, created_by
  ('33333333-3333-3333-3333-333333333333', 'RV-TEST-001', 'Receipt', CURRENT_DATE, 'Draft', 'Test Balanced Vou

-- Insert balanced lines: Debit 1000 == Credit 1000
INSERT INTO accounting.voucher_lines (voucher_id, account_id, line_type, amount) VALUES
  ('33333333-3333-3333-3333-333333333333', '11111111-1111-1111-1111-111111111111', 'Debit', 1000.00),
  ('33333333-3333-3333-3333-333333333333', '22222222-2222-2222-2222-222222222222', 'Credit', 1000.00);

-- Update status to Posted -> Trigger executes & allows update
UPDATE accounting.vouchers SET status = 'Posted' WHERE id = '33333333-3333-3333-3333-333333333333';
SELECT voucher_number, status FROM accounting.vouchers WHERE id = '33333333-3333-3333-3333-333333333333';
EOF
```

Expected Result: Status updates cleanly to `Posted`.

Test Case B: Unbalanced Voucher (Must Fail & Reject Update)

```
PGPASSWORD="SecureEnterprisePassword123!" psql -h 127.0.0.1 -U insacc_user -d insacc_db << "EOF"
-- Insert voucher header
INSERT INTO accounting.vouchers (id, voucher_number, voucher_type, voucher_date, status, narration, created_by
  ('44444444-4444-4444-4444-444444444444', 'RV-TEST-002', 'Receipt', CURRENT_DATE, 'Draft', 'Test Unbalanced V

-- Insert UNBALANCED lines: Debit 1000 != Credit 500
INSERT INTO accounting.voucher_lines (voucher_id, account_id, line_type, amount) VALUES
  ('44444444-4444-4444-4444-444444444444', '11111111-1111-1111-1111-111111111111', 'Debit', 1000.00),
  ('44444444-4444-4444-4444-444444444444', '22222222-2222-2222-2222-222222222222', 'Credit', 500.00);

-- Attempt status update to Posted -> MUST THROW EXCEPTION
UPDATE accounting.vouchers SET status = 'Posted' WHERE id = '44444444-4444-4444-4444-444444444444';
EOF
```

Expected Result: Throws error: `ERROR: Voucher Posting Rejected: Debit sum (1000.00) does not equal Credit sum (500.00)`.

10.3 Verify Automated Backup File Creation

Test the backup script manually:

```
sudo /usr/local/bin/insacc_db_backup.sh
ls -lh /var/backups/postgresql/
```

11. Troubleshooting & Maintenance Runbook

Symptom / Issue	Root Cause	Resolution Command
`Connection refused (port 5432)`	PostgreSQL not listening on network IP	Verify `listen_addresses = '*'` in `postgresql.conf` and restart service.
`FATAL: no pg_hba.conf entry`	Client IP not allowed in `pg_hba.conf`	Append CIDR block (e.g. `hostssl insacc_db insacc_user <IP>/32 scram-sha-256`) to `pg_hba.conf`.
`FATAL: password authentication failed`	Incorrect password or SCRAM mismatch	Reset role password: `ALTER ROLE insacc_user WITH PASSWORD 'NewPass';`
`Out of memory / OOM Killer`	`shared_buffers` or `work_mem` set too high	Reduce `work_mem` in `postgresql.conf` and restart PostgreSQL.

12. Summary & Verification Checklist

Phase #	Installation Phase Title	Primary Goal	Status
Phase 1	OS & Kernel Preparation	Sysctl kernel tuning & user limits	[ ] Completed
Phase 2	Install PostgreSQL 17	PGDG repo & PostgreSQL 17 packages	[ ] Completed
Phase 3	Network & Security	TLS 1.3 certificates, UFW firewall & `pg_hba.conf`	[ ] Completed
Phase 4	Performance Tuning	Memory parameters (`2GB` shared buffers) in `postgresql.conf`	[ ] Completed
Phase 5	Database & Schemas	`insacc_user`, `insacc_db` & 5 domain schemas created	[ ] Completed
Phase 6	Balance Triggers	PL/pgSQL function `fn_validate_voucher_balance` active	[ ] Completed
Phase 7	Automated Backups	Daily cron job & WAL archiving active	[ ] Completed
Phase 8	E2E Verification	Remote SSL login & trigger test cases passed	[ ] Completed

End of Step-by-Step PostgreSQL 17 Database Server Setup Manual.

InsAcc Enterprise Master Architecture Specification

Document ID: MASTER_ARCHITECTURE.md
Version: 1.0.0
Status: Single Source of Truth for Platform Architecture
Release Date: 2026-07-22
Target Software: InsAcc Enterprise Asset & Investment Accounting Platform v1.0.0

> **IMPORTANT: GOVERNANCE DIRECTIVE**: This document is the **single source of truth** for all architectural, operational, data modeling, and engineering standards for the InsAcc platform. All technical documentation, developer guides, user manuals, and specifications MUST reference this file. Do not duplicate information across documents — link back to `MASTER_ARCHITECTURE.md`.

Table of Contents

---

1. Business Overview

1.1 Purpose

InsAcc (Intelligent Asset & Investment Accounting) is an enterprise financial management application designed for tracking physical asset portfolios, financial investments, and property real estate operations with integrated double-entry accounting.

1.2 Target Audience & Use Cases

- **Asset Managers & Family Offices**: Track precious metals (gold, silver), equities, bonds, mutual funds, and custom asset classes.

- **Property Operations Teams**: Manage property hierarchies (Categories → Buildings → Units), tenant leases, rent collections, and Post-Dated Cheque (PDC) lifecycles.
- **Financial Operations & Accounting Departments**: Record financial events through double-entry vouchers (Receipt, Payment, Journal), generate trial balances, profit & loss statements, balance sheets, and cash flow reports.

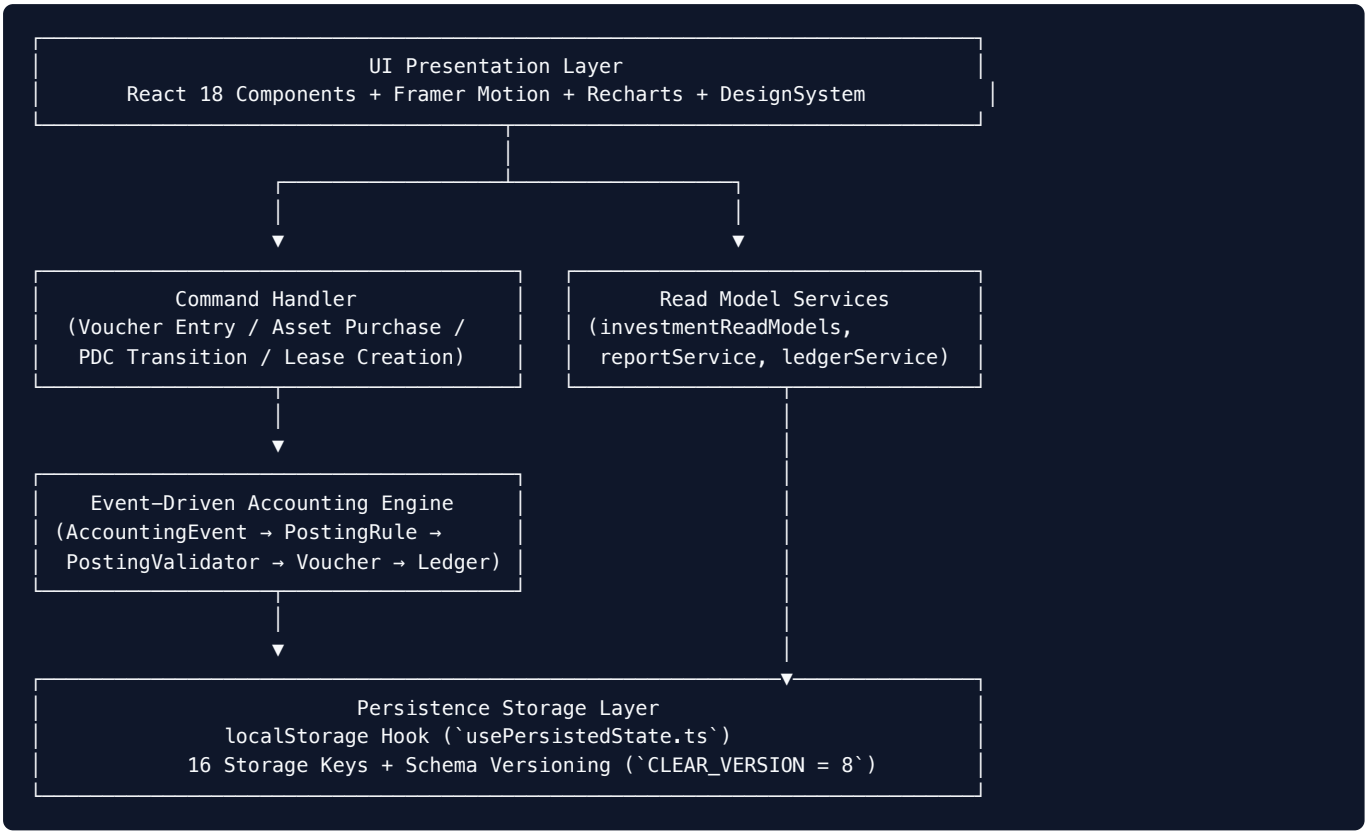
1.3 Core Modules

1. **Investment Portfolio Module**: Holdings, purchase ledger, income/expense/journal transactions, bank accounts, and asset performance analytics.
2. **Property Management Module**: Property units, tenant leases, rent collection, security deposit tracking, and PDC cheque lifecycle management.
3. **General Ledger & Accounting Core**: Shared event-driven accounting engine enforcing double-entry integrity, chart of accounts management, and financial statement generation.

---

2. Software Architecture

InsAcc v1.0.0 uses a **CQRS-inspired (Command Query Responsibility Segregation), local-first desktop application architecture**.



2.1 Write Path (Command Pipeline)

1. User actions dispatch a business command or `AccountingEvent`.
2. The `AccountingEngine` resolves the corresponding `PostingRule` from `postingRules.ts`.
3. A draft `Voucher` is generated and validated by `PostingValidator`.
4. Upon approval and posting, `VoucherService` updates state and invalidates `LedgerService` balance caches.
5. `usePersistedState` serializes updated state to `localStorage`.

2.2 Read Path (Query Pipeline)

1. Screen components invoke pure read-model functions (e.g., `calculateAssetAllocation`, `calculateNetWorth`).
2. Read-models query state and calculate projections dynamically using `useMemo`.
3. Formatted display values are produced at render time via `reportFormatters.ts`.

---

3. Technology Stack

Layer	Technology	Version	Purpose
Core Framework	React	18.3.1	Component-based user interface
Language	TypeScript	5.3.3	Strict static typing across all modules
Bundler	Vite	5.x	Dev server (port 5174) & HMR production bundler
Desktop Wrapper	Electron	28.x	Native cross-platform desktop shell
Packaging	Electron Builder	23.6.0	Distributable packaging (NSIS, DMG, AppImage)
Charts	Recharts	2.x	Responsive data visualization
Animations	Framer Motion	11.x	Screen transitions & micro-interactions
Styling	Vanilla CSS / CSS Custom Properties	Custom	Design tokens & themes (`src/renderer/styles/theme.css`)
Typography	Inter	woff2	Locally bundled typography (8 font weights)
Persistence	Browser `localStorage`	HTML5	Local state persistence via `usePersistedState` hook
Testing	Playwright	1.x	End-to-end integration test suite

---

4. Deployment Models

4.1 Current Production Model (v1.0.0)

- **Local Desktop Standalone Application**: Executable packaged via Electron Builder.
- **Client OS**: Windows 10/11 (x64), macOS 12+ (Apple Silicon / Intel), Linux (x64).
- **Infrastructure**: Zero server requirements. Runs 100% locally on user machine.

4.2 Target Enterprise Model `[To Be Implemented]`

- **Architecture**: Client-Server / Multi-tenant Deployment.
- **Backend API**: Node.js / Express REST API microservice cluster.
- **Database**: Dedicated PostgreSQL 17 relational database server.
- **Reverse Proxy**: Nginx with SSL/TLS termination.

---

5. Electron Architecture

The application runs in Electron's isolated two-process model:

```
src/main/main.js (Main Process)
  ├── Creates BrowserWindow (1400x900, min 1200x800)
  ├── Security Configuration:
  │     ├── nodeIntegration: false
  │     ├── contextIsolation: true
  │     └── sandbox: false
  ├── Registers IPC Handlers: 'save-file'
  └── Preload Bridge Execution: src/main/preload.js
        │
        ▼
window.api Context Bridge (`src/main/preload.js`)
  └── Exposes: window.api.saveFile(filename, content) → Promise<string>
        │
        ▼
src/renderer/ (Renderer Process — React App)
  └── Single Page Application built by Vite
```

---

6. Backend Architecture

6.1 Current Implementation (v1.0.0)

InsAcc v1.0.0 has **no server-side backend process**. All business services, data transformations, accounting rules, and reporting engines execute entirely inside the renderer process.

6.2 Target Server Backend `[To Be Implemented]`

- **Framework**: Node.js / Express REST API.
- **ORM / Query Builder**: Prisma or Kysely.
- **API Endpoints**: `/api/v1/investments`, `/api/v1/properties`, `/api/v1/accounting/vouchers`, `/api/v1/reports`.

---

7. PostgreSQL Architecture

7.1 Current Implementation (v1.0.0)

No SQL database is included in v1.0.0. Persistence is handled via browser `localStorage` using 16 key-value stores.

7.2 Target Relational Database Schema `[To Be Implemented]`

The target PostgreSQL database will comprise three main schemas:

1. `accounting_*`: Accounts, Vouchers, Voucher Lines, Posting Rules, Fiscal Periods.
2. `investment_*`: Investments, Purchase Records, Holdings, Transactions.
3. `property_*`: Categories, Buildings, Units, Tenants, Leases, Rent Payments, PDC Cheques, Security Deposits.

---

8. Security Architecture

8.1 Client-Side Security Controls

- **Context Isolation**: Enabled (`contextIsolation: true`). Node integration disabled (`nodeIntegration: false`).
- **Font & Asset Security**: Zero external web requests. Fonts (Inter woff2) and resources are 100% bundled locally.
- **IPC Surface**: Restricted to single `save-file` channel in `preload.js`.

8.2 Current Limitations & Hardening Plan

- **Password Storage**: Currently stored in plaintext in `localStorage` (`storedPassword`).
  Roadmap: Replace with Argon2 / bcrypt hashing via Web Crypto API `[To Be Implemented]`.
- **Data at Rest**: Unencrypted `localStorage`.
  Roadmap: Migrate persistence to encrypted SQLCipher / SQLite database `[To Be Implemented]`.

---

9. Authentication Architecture

9.1 Current Profile Selection & Authorization (v1.0.0)

- **Profiles**: Pre-configured profiles (`Investor`, `Property Manager`).
- **Role-Based Access Control (RBAC)**:
  - `Admin`: Full permissions (create, edit, delete, reset data, manage users).
  - `Accounts`: Operational permissions (record transactions, vouchers, view reports).
- **Auth Flow**: Screen state transition (`login` → `profiles` → `module` → `dashboard`).

9.2 Target JWT Authentication `[To Be Implemented]`

- Bearer token authentication, refresh tokens, role claims, and session expiry timers.

---

10. Accounting Architecture

InsAcc contains a fully featured, rule-governed **Double-Entry Accounting Engine**.

10.1 Key Accounting Principles

1. **Double-Entry Mandate**: Every voucher must balance exactly: $\sum \text{Debits} = \sum \text{Credits}$.
2. **Voucher State Machine**:

```
Draft ──► Approved ──► Posted
  │         │
  ▼         ▼
Cancelled  Reversed
```

3. **Immutability of Posted Vouchers**: Once a voucher transitions to `Posted`, its accounting lines cannot be edited. Corrections require an explicit Reversal Voucher (`reverseVoucher()`).

10.2 Core Accounting Domain Files (`src/renderer/accounting/`)

- `accountingEngine.ts`: Core event processor dispatching commands.
- `postingRules.ts`: Event-to-posting-rule resolution engine.
- `postingValidator.ts`: Balance and line validation rules.
- `voucherService.ts`: Voucher creation, state transition, and numbering.
- `ledgerService.ts`: Account balance calculation and memoized balance caching.
- `chartOfAccountsService.ts`: System Chart of Accounts management and hierarchy.
- `systemAccountRegistry.ts`: Pre-defined standard account code registry.

10.3 Primary System Account Codes

Account Code	Account Name	Type	Normal Balance
`1110`	Cash on Hand	Asset	Debit
`1120`	Bank Accounts	Asset	Debit
`1130`	Rent Receivable	Asset	Debit
`1410`	PDC Cheques Held	Asset	Debit
`2110`	Unearned / Deferred Rent	Liability	Credit
`2120`	Tenant Security Deposits	Liability	Credit
`2200`	Owner's Capital / Equity	Equity	Credit
`4110`	Dividend & Interest Income	Revenue	Credit
`4120`	Rental Revenue	Revenue	Credit
`5110`	Property Maintenance Expense	Expense	Debit

11. Folder Structure

```
InsAcc/
├── index.html                          # Vite entry HTML
├── package.json                        # Dependencies, scripts, electron-builder config
├── vite.config.ts                      # Vite bundler configuration
├── tsconfig.json                       # TypeScript compiler configuration
├── docs/                               # Platform documentation
│   ├── MASTER_ARCHITECTURE.md          # SINGLE SOURCE OF TRUTH (this file)
│   ├── PRD.md                          # Product Requirements Document
│   ├── ACCOUNTING_ARCHITECTURE.md      # Accounting Engine Deep Dive
│   ├── BANKING_ARCHITECTURE.md         # Banking & Reconciliation Architecture
│   ├── PURCHASE_LEDGER_ARCHITECTURE.md # Purchase Ledger Specification
│   └── REPORTS_ARCHITECTURE.md          # Financial Reports Specification
├── src/
│   ├── main/
│   │   ├── main.js                     # Electron main process
│   │   └── preload.js                  # IPC context bridge
│   └── renderer/
│       ├── index.tsx                   # React application entry point
│       ├── App.tsx                     # Root component, routing, top-level state
│       ├── usePersistedState.ts        # localStorage persistence hook
│       ├── utils.ts                    # Utility functions and translation helper t()
│       ├── accounting/                 # Double-entry accounting engine core
│       ├── components/                 # React UI screens & components
│       │   ├── charts/                 # Recharts data visualization wrappers
│       │   └── design/                 # Reusable Design System components
│       ├── data/                       # Domain data types and default datasets
│       ├── readModels/                 # CQRS read-side projection services
│       ├── services/                   # Business domain services
│       └── styles/                     # CSS custom properties & local fonts
└── release/                            # Packaged desktop executables (gitignored)
```

12. Coding Standards

1. **TypeScript Strictness**: `tsconfig.json` has `strict: true`. No implicit `any`. All props and service responses must have explicit interfaces.
2. **React Patterns**:
   - Functional components only.
   - Use `useMemo` for derived dataset queries and aggregations.
   - Component state for ephemeral UI toggles; `usePersistedState` for persistent application data.
3. **No Inline Styling**: All visual styling MUST use CSS classes defined in `theme.css` or CSS Custom Properties. Never pass inline `style={{ ... }}` objects except for dynamic layout measurements.
4. **Pure Service Layer**: Service files in `services/` and `accounting/` must remain pure functions with zero React imports or DOM references.

---

13. Database & Persistence Standards

13.1 Entity Identification

- Every entity ID is an **immutable string**. Format: UUID v4 or timestamp-prefixed non-colliding string (e.g., `ba-1719500000000`, `P-1719500000000-1`).
- IDs are primary keys and foreign keys (`accountId`, `tenantId`, `propertyId`). Re-generating an ID is strictly forbidden.

13.2 Derived Balance Rule

> **GOLDEN RULE**: Account balances and bank balances are ALWAYS DERIVED values.
> $\text{Current Balance} = \text{Opening Balance} + \sum \text{Debits} - \sum \text{Credits}$
> Balances MUST NEVER be persisted as independent, manually editable scalar values. To correct a balance mismatch, a user MUST enter a corrective accounting voucher or bank transaction.

13.3 Storage Key Schema (16 Keys)

All application data is persisted under the `insacc_*` namespace in `localStorage`. Schema versioning is governed by `insacc_clear_version` (`CLEAR_VERSION = 8`).

---

14. UI & Design System Standards

- **Theme Palette**: Premium Navy + Gold branding palette in Dark Mode (`#0C0C0D` background, `#1C1C1F` surface, `#6366F1` primary, `#F59E0B` gold accents). Light mode supported via `.dark-mode` class toggle.
- **Typography**: Inter woff2 font. 8-weight hierarchy (400 regular, 500 medium, 600 semi-bold, 700 bold).
- **Spacing Grid**: Strict 4-point spacing scale (`--space-1`: 4px to `--space-16`: 64px).
- **Border Radius**: Buttons/Inputs `10px` (`--radius`), Cards `16px` (`--radius-lg`), Dialogs `20px` (`--radius-xl`).
- **Icons**: Inline SVG components in `src/renderer/components/design/DesignSystem.tsx`. Zero third-party icon packages.

---

15. Documentation Standards

1. **Reference Mandate**: Every technical document, architectural RFC, or manual created in the InsAcc codebase MUST link back to `docs/MASTER_ARCHITECTURE.md` as its primary reference.

2. **Zero Fabrication**: Features not yet present in the codebase MUST be marked `[To Be Implemented]`.
3. **Diagram Standard**: Visual workflows must use valid Mermaid.js code blocks embedded directly in Markdown files.
4. **Maintenance**: Any PR or commit altering system architecture, account rules, or persistence keys MUST update `MASTER_ARCHITECTURE.md` concurrently.

End of Master Architecture Specification.

InsAcc Database Design Specification

Document ID: DATABASE_DESIGN_SPECIFICATION.md
Version: 1.0.0
Status: Official Database Design Specification
Release Date: 2026-07-22
Target Software: InsAcc Enterprise Asset & Investment Accounting Platform v1.0.0
Single Source of Truth Reference: MASTER_ARCHITECTURE.md

> **IMPORTANT: GOVERNANCE DIRECTIVE**: InsAcc v1.0.0 persists operational data locally using HTML5 `localStorage` across 16 storage keys. All relational SQL schemas, PostgreSQL tables, DDL triggers, views, and functions documented herein represent the **Planned Target Enterprise Architecture [Planned]** for server release v2.0.0.

Table of Contents

1. Database Architecture & Overview

InsAcc manages financial assets, double-entry general ledger vouchers, property real estate units, tenant contracts, and bank transactions.

Dual Architecture Paradigm

- **Current v1.0.0 State**: Local-first `localStorage` JSON document storage with schema versioning (`CLEAR_VERSION = 8`).
- **Target Enterprise State [Planned]**: Multi-tenant PostgreSQL 17 relational database split into 5 logical schemas (`accounting` , `investment` , `property` , `auth` , `audit`).

2. Current LocalStorage Data Key Schema (v1.0.0)

The v1.0.0 desktop distribution uses **16 primary key-value storage collections**:

Key Name	TypeScript Model Type	Primary Key Identifier	Description
`insacc_investments`	`Investment[]`	`id` (UUID / String)	Active portfolio holdings and asset positions.
`insacc_transactions`	`Transaction[]`	`id` (UUID / String)	Income, expense, and general journal records.
`insacc_statement`	`StatementEntry[]`	Array Index / `id`	Legacy bank statement entries.
`insacc_balance`	`number`	N/A	Legacy bank balance scalar (Deprecated).
`insacc_documents`	`DocItem[]`	`id` (UUID)	Document metadata and Base64 file attachments.
`insacc_logs`	`LogEntry[]`	`id` (UUID)	System operational audit logs.
`insacc_purchase_categories`	`PurchaseCategory[]`	`id` (UUID)	Purchase ledger category taxonomy.
`insacc_purchases`	`Purchase[]`	`id` (`P-timestamp-N`)	Individual purchase ledger asset transactions.
`insacc_inv_users`	`UserEntry[]`	`id` (UUID)	Investment module user profile credentials.
`insacc_prop_users`	`UserEntry[]`	`id` (UUID)	Property module user profile credentials.
`insacc_prop_categories`	`PropertyCategory[]`	`id` (UUID)	Property category classifications.
`insacc_prop_buildings`	`PropertyBuilding[]`	`id` (UUID)	Building master records per category.
`insacc_prop_units`	`PropertyUnit[]`	`id` (UUID)	Rentable units within buildings.
`insacc_prop_tenants`	`PropertyTenant[]`	`id` (UUID)	Tenant master records and active lease terms.
`insacc_prop_rent`	`RentPayment[]`	`id` (UUID)	Per-unit rent collection payment records.
`insacc_clear_version`	`string`	N/A	Schema version tracking (`CLEAR_VERSION = '8'`).

3. Naming & Data Type Standards

3.1 SQL Naming Rules [Planned]

- **Schemas**: `snake_case` (e.g., `accounting`, `property`).
- **Tables**: `snake_case` plural nouns (e.g., `accounts`, `voucher_lines`, `leases`).
- **Columns**: `snake_case` (e.g., `voucher_number`, `opening_balance`, `created_at`).
- **Primary Keys**: `id` UUID type generated via `gen_random_uuid()`.
- **Foreign Keys**: `<singular_table_name>_id` (e.g., `account_id`, `tenant_id`).
- **Indexes**: `idx_<table_name>_<column_name>` (e.g., `idx_vouchers_status`).
- **Triggers**: `trg_<table_name>_<action>` (e.g., `trg_vouchers_updated_at`).

3.2 Monetary & Precision Rules

- **Monetary Amounts**: `NUMERIC(15, 2)` (Supports up to 999 trillion with 2 decimal places).
- **Asset Quantities**: `NUMERIC(18, 6)` (Supports fractional holdings such as 0.000001 gold grams or crypto lots).
- **Timestamps**: `TIMESTAMPTZ` (ISO 8601 with UTC offset).

---

4. Planned PostgreSQL Relational ER Diagrams

4.1 General Ledger & Accounting ERD [Planned]

```
erDiagram
    ACCOUNTS ||--o{ ACCOUNTS : "parent_id"
    ACCOUNTS ||--o{ VOUCHER_LINES : "account_id"
    VOUCHERS ||--|{ VOUCHER_LINES : "voucher_id"
    POSTING_RULES ||--o{ ACCOUNTS : "debit/credit_account"

    ACCOUNTS {
        uuid id PK
        string code UK
        string name
        string type
        uuid parent_id FK
        numeric opening_balance
        boolean is_active
    }

    VOUCHERS {
        uuid id PK
        string voucher_number UK
        string voucher_type
        date voucher_date
        string status
        text narration
        string created_by
    }

    VOUCHER_LINES {
        uuid id PK
        uuid voucher_id FK
        uuid account_id FK
        string line_type
        numeric amount
        text memo
    }
```

4.2 Property Management ERD [Planned]

```
erDiagram
    CATEGORIES ||--o{ BUILDINGS : "category_id"
    BUILDINGS ||--o{ UNITS : "building_id"
    UNITS ||--o{ LEASES : "unit_id"
    TENANTS ||--o{ LEASES : "tenant_id"
    LEASES ||--o{ RENT_PAYMENTS : "lease_id"
    LEASES ||--o{ PDC_CHEQUES : "lease_id"

    CATEGORIES {
        uuid id PK
        string name
    }

    BUILDINGS {
        uuid id PK
        uuid category_id FK
        string name
        string address
    }

    UNITS {
        uuid id PK
        uuid building_id FK
        string unit_number
        string status
    }

    TENANTS {
        uuid id PK
        string name
        string national_id
        string phone
    }

    LEASES {
        uuid id PK
        uuid unit_id FK
        uuid tenant_id FK
        date start_date
        date end_date
        numeric rent_amount
    }

    PDC_CHEQUES {
        uuid id PK
        uuid lease_id FK
        string cheque_number
        date maturity_date
        numeric amount
        string status
    }
```

---

5. Accounting Schema Specification [Planned]

Schema: `accounting`

Table: `accounting.accounts`

- **Purpose**: Stores system and user-defined Chart of Accounts nodes.

- **Primary Key**: `id` UUID.

Column Name	Data Type	Nullable	Constraints	Description
`id`	UUID	No	PRIMARY KEY	Immutable unique identifier.
`code`	VARCHAR(20)	No	UNIQUE	Account code (e.g., `1110`, `1120.001`).
`name`	VARCHAR(255)	No	—	Account description.
`type`	VARCHAR(50)	No	CHECK	`Asset`, `Liability`, `Equity`, `Revenue`, `Expense`.
`parent_id`	UUID	Yes	FK -> `accounts(id)`	Parent account node ID.
`opening_balance`	NUMERIC(15,2)	No	DEFAULT 0.00	Initial opening balance baseline.
`is_active`	BOOLEAN	No	DEFAULT TRUE	Active status flag.
`created_at`	TIMESTAMPTZ	No	DEFAULT NOW()	System creation timestamp.

Table: `accounting.vouchers`

- **Purpose**: Master record for double-entry vouchers (`RV`, `PV`, `JV`).

Column Name	Data Type	Nullable	Constraints	Description
`id`	UUID	No	PRIMARY KEY	Unique voucher identifier.
`voucher_number`	VARCHAR(50)	No	UNIQUE	Numbering sequence (e.g., `RV-2026-0001`).
`voucher_type`	VARCHAR(20)	No	CHECK	`Receipt`, `Payment`, `Journal`.
`voucher_date`	DATE	No	—	Economic transaction date.
`status`	VARCHAR(20)	No	CHECK	`Draft`, `Approved`, `Posted`, `Cancelled`, `Reversed`.
`narration`	TEXT	Yes	—	Master voucher description.
`created_by`	VARCHAR(100)	No	—	User ID of creator.
`posted_at`	TIMESTAMPTZ	Yes	—	Timestamp of general ledger posting.

Table: `accounting.voucher_lines`

- **Purpose**: Detail debit and credit lines for each voucher.

Column Name	Data Type	Nullable	Constraints	Description
`id`	UUID	No	PRIMARY KEY	Line item identifier.
`voucher_id`	UUID	No	FK -> `vouchers(id)`	Foreign key to parent voucher.
`account_id`	UUID	No	FK -> `accounts(id)`	Foreign key to general ledger account.
`line_type`	VARCHAR(10)	No	CHECK	`Debit` or `Credit`.
`amount`	NUMERIC(15,2)	No	CHECK > 0	Line transaction amount.
`memo`	TEXT	Yes	—	Line item descriptive memo.

6. Investment Schema Specification [Planned]

Schema: `investment`

Table: `investment.investments`

- **Purpose**: Master holdings position table.

Column Name	Data Type	Nullable	Constraints	Description
`id`	UUID	No	PRIMARY KEY	Investment position ID.
`asset_name`	VARCHAR(255)	No	—	Asset title (e.g. `24K Gold Bar 1kg`).
`asset_type`	VARCHAR(50)	No	—	`Gold`, `Silver`, `Stocks`, `Bonds`, `ETFs`.
`quantity`	NUMERIC(18,6)	No	CHECK >= 0	Total units owned.
`purchase_value`	NUMERIC(15,2)	No	CHECK >= 0	Historical cost basis.
`current_price`	NUMERIC(15,2)	No	CHECK >= 0	Current market price per unit.

7. Property Schema Specification [Planned]

Schema: `property`

Table: `property.pdc_cheques`

- **Purpose**: Post-Dated Cheque state tracking.

Column Name	Data Type	Nullable	Constraints	Description
`id`	UUID	No	PRIMARY KEY	Cheque identifier.
`lease_id`	UUID	No	FK -> `leases(id)`	Foreign key to tenant lease contract.
`cheque_number`	VARCHAR(50)	No	—	Physical cheque leaf number.
`bank_name`	VARCHAR(100)	No	—	Drawee bank name.
`maturity_date`	DATE	No	—	Cheque maturity date.
`amount`	NUMERIC(15,2)	No	CHECK > 0	Cheque face value.
`status`	VARCHAR(20)	No	CHECK	`Received`, `Deposited`, `Cleared`, `Bounced`, `Replaced`, `Cancelled`.

8. User & Auth Schema Specification [Planned]

Schema: `auth`

Table: `auth.users`

- **Purpose**: System user profiles and authentication.

Column Name	Data Type	Nullable	Constraints	Description
`id`	UUID	No	PRIMARY KEY	User identifier.
`email`	VARCHAR(255)	No	UNIQUE	User login email address.
`password_hash`	VARCHAR(255)	No	—	Salted password hash (Argon2 / bcrypt).
`role`	VARCHAR(20)	No	CHECK	`Admin` or `Accounts`.
`is_active`	BOOLEAN	No	DEFAULT TRUE	Active user status.

9. Audit & System Settings Schema Specification [Planned]

Schema: `audit`

Table: `audit.events`

- **Purpose**: Complete system operational audit trail.

Column Name	Data Type	Nullable	Constraints	Description
`id`	UUID	No	PRIMARY KEY	Audit log entry ID.
`user_id`	UUID	Yes	FK -> `users(id)`	User executing action.
`event_type`	VARCHAR(100)	No	—	Action descriptor (e.g. `VOUCHER_POSTED`).
`payload`	JSONB	No	—	Complete state snapshot before/after action.
`created_at`	TIMESTAMPTZ	No	DEFAULT NOW()	System log timestamp.

10. Database Triggers, Functions & Views [Planned]

10.1 Double-Entry Balance Validator Trigger Function [Planned]

```
CREATE OR REPLACE FUNCTION accounting.fn_validate_voucher_balance()
RETURNS TRIGGER AS $$
DECLARE
    v_debit_sum NUMERIC(15,2);
    v_credit_sum NUMERIC(15,2);
BEGIN
    SELECT COALESCE(SUM(amount), 0) INTO v_debit_sum
    FROM accounting.voucher_lines
    WHERE voucher_id = NEW.id AND line_type = 'Debit';

    SELECT COALESCE(SUM(amount), 0) INTO v_credit_sum
    FROM accounting.voucher_lines
    WHERE voucher_id = NEW.id AND line_type = 'Credit';

    IF ABS(v_debit_sum - v_credit_sum) > 0.001 THEN
        RAISE EXCEPTION 'Voucher % is unbalanced! Total Debits (%) != Total Credits (%)',
            NEW.voucher_number, v_debit_sum, v_credit_sum;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validate_voucher_posting
BEFORE UPDATE OF status ON accounting.vouchers
FOR EACH ROW
WHEN (NEW.status = 'Posted')
EXECUTE FUNCTION accounting.fn_validate_voucher_balance();
```

11. Migration Strategy (LocalStorage to PostgreSQL)

To migrate an active workstation from `localStorage` to PostgreSQL 17:

1. **State Export**: Administrator exports state snapshot via `Settings` -> `Export JSON State`.
2. **Database Initialization**: Run `scripts/postgresql/init-database.sh`.
3. **ETL Ingestion**: Node.js ingestion script converts JSON objects to SQL `INSERT` statements using parameterized transactions.
4. **Validation Check**: Asserts that: $$\sum \text{PostgreSQL General Ledger Debits} = \sum \text{LocalStorage Ledger Debits}$$

End of Database Design Specification.

title: "InsAcc Enterprise Asset & Investment ERP - Database Server Setup Guide [To Be Implemented]" document_id: "DATABASE_SERVER_SETUP_GUIDE.md" version: "1.0.0" release_date: "2026-07-22" app_version: "v2.0.0 Target Server" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Official Database Server Setup Specification" classification: "Commercial Enterprise Documentation"

InsAcc Enterprise ERP Platform

Database Server Setup & Administration Guide `[To Be Implemented]`

> **Single Source of Truth Reference**: All server provisioning rules, PostgreSQL 17 configurations, memory tuning parameters, and DDL schema deployments defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise database setup specification	Chief Architecture Review Board

Table of Contents

1. Purpose

This document provides Database Administrators (DBAs), Infrastructure Engineers, and DevOps personnel with a complete technical guide for installing, configuring, hardening, tuning, and deploying the PostgreSQL 17 relational database server cluster required for InsAcc Target Enterprise Server v2.0.0 `[To Be Implemented]`.

2. Scope

This specification covers:

- Linux operating system kernel tuning (`sysctl.conf` , `limits.conf`) and disk file system options.
- Official PGDG repository setup and PostgreSQL 17 package installation.
- Host-based authentication (`pg_hba.conf`) and TLS 1.3 certificate deployment.
- High-performance memory allocation (`shared_buffers` , `work_mem` , `effective_cache_size`) and WAL parameters.
- Multi-schema database deployment (`accounting` , `investment` , `property` , `auth` , `audit`).
- Automated PL/pgSQL double-entry trigger functions (`fn_validate_voucher_balance()`).
- Automated backup automation (`pg_dump`), WAL streaming replication, and Point-In-Time Recovery (PITR).

Out of Scope:

- Current desktop local-first `localStorage` architecture (covered in Volume 02 Chapter 01).
- Express REST API service cluster setup (covered in Volume 05 Chapter 03).

3. Audience

This document is authored for:

- Database Administrators (DBAs) & Data Infrastructure Engineers
- Systems Administrators & DevOps Pipeline Engineers
- Information Security & Regulatory Compliance Auditors

4. Prerequisites

Before initiating database server provisioning:

1. Review the PostgreSQL architecture defined in MASTER_ARCHITECTURE.md#7-postgresql-architecture.
2. Provision a dedicated physical server or virtual machine running Ubuntu 24.04 LTS or RHEL 9 (Minimum: 4 vCPU, 8 GB RAM, 100 GB Enterprise SSD).

5. Warnings & Operational Hazards

> **WARNING: To Be Implemented**: The PostgreSQL 17 database server architecture, DDL deployment pipeline, and server administration procedures documented in this manual are target specifications for enterprise release v2.0.0. InsAcc v1.0.0 operates as an offline desktop application using `localStorage`.

> **WARNING: SUPERUSER PRODUCTION ACCESS HAZARD**: The PostgreSQL default superuser role (`postgres`) MUST NOT be used by application services. Application connections MUST authenticate using the restricted, non-superuser role `insacc_user`.

6. Notes & Architecture Context

> **NOTE: Zero Balance Drift via Triggers**: Database-level integrity is guaranteed by PL/pgSQL trigger functions (`trg_validate_voucher_posting`). The database engine automatically rejects any SQL `INSERT` or `UPDATE` operation on `accounting.voucher_lines` that violates debit-credit balance equality ($\sum D = \sum C$).

7. Main Content

7.1 OS Provisioning & Kernel Parameter Tuning

Prior to installing PostgreSQL 17, tune Linux kernel parameters to optimize memory page allocation and file descriptor limits.

1. Kernel Parameter Optimization (`/etc/sysctl.d/99-postgresql.conf`):

```
# Maximum shared memory segment size (bytes)
kernel.shmmax = 18446744073709551615
kernel.shmall = 18446744073709551615

# Memory overcommit tuning for PostgreSQL
vm.overcommit_memory = 2
vm.overcommit_ratio = 80
vm.swappiness = 10
vm.dirty_background_ratio = 3
vm.dirty_ratio = 10
```

Apply sysctl settings:

```
sudo sysctl -p /etc/sysctl.d/99-postgresql.conf
```

2. File Descriptor & Security Limits (`/etc/security/limits.d/99-postgres.conf`):

```
postgres soft nofile 65536
postgres hard nofile 65536
postgres soft nproc 4096
postgres hard nproc 4096
```

7.2 PostgreSQL 17 Package Installation (Ubuntu 24.04 LTS / RHEL 9)

Execute the automated server setup script (`docs/enterprise/scripts/postgresql/server-setup.sh`) or follow manual package setup:

Step-by-Step Installation on Ubuntu 24.04 LTS:

```
# 1. Install prerequisites and PGDG repository signing key
sudo apt-get update
sudo apt-get install -y curl ca-certificates gnupg lsb-release
sudo install -d /etc/apt/keyrings
curl -fsSL https://www.postgresql.org/media/keys/ACCC4CF8.asc | sudo gpg --dearmor -o /etc/apt/keyrings/postgr

# 2. Add PostgreSQL official PGDG repository
echo "deb [signed-by=/etc/apt/keyrings/postgresql.gpg] http://apt.postgresql.org/pub/repos/apt $(lsb_release -

# 3. Update apt index and install PostgreSQL 17
sudo apt-get update
sudo apt-get install -y postgresql-17 postgresql-contrib-17

# 4. Verify service status
sudo systemctl status postgresql@17-main
```

7.3 Network Access Control & TLS 1.3 Encryption (`pg_hba.conf`)

Configure host-based authentication in `/etc/postgresql/17/main/pg_hba.conf`:

```
# TYPE  DATABASE        USER            ADDRESS                 METHOD

# Local unix socket connections for admin
local   all             postgres                                peer

# Local loopback connections
host    all             all             127.0.0.1/32            scram-sha-256
host    all             all             ::1/128                 scram-sha-256

# Restricted Enterprise Application Subnet Access (TLS 1.3 Required)
hostssl insacc_db       insacc_user     10.0.4.0/24             scram-sha-256
```

Enforce SCRAM-SHA-256 Password Encryption:

In `/etc/postgresql/17/main/postgresql.conf`:

```
password_encryption = scram-sha-256
ssl = on
ssl_cert_file = '/etc/ssl/certs/insacc_server.crt'
ssl_key_file = '/etc/ssl/private/insacc_server.key'
ssl_min_protocol_version = 'TLSv1.3'
```

7.4 Performance Tuning & Memory Allocation (`postgresql.conf`)

Tune PostgreSQL parameters based on server hardware capacity (Target Server Spec: 4 vCPU, 8 GB RAM):

Location: `docs/enterprise/configs/postgresql/postgresql.conf`

```
# Memory Configuration (8 GB Total RAM Baseline)
shared_buffers = 2GB                  # 25% of Total System Memory
huge_pages = try
work_mem = 32MB                       # Per-operation sort memory
maintenance_work_mem = 512MB          # Index build memory
effective_cache_size = 6GB            # 75% of Total System Memory

# Write-Ahead Log (WAL) & Checkpoints
wal_level = replica
max_wal_size = 4GB
min_wal_size = 1GB
checkpoint_completion_target = 0.9
checkpoint_timeout = 15min

# Query Planner & Worker Threads
max_worker_processes = 4
max_parallel_workers_per_gather = 2
max_parallel_workers = 4
random_page_cost = 1.1                # Optimized for SSD storage

# Logging & Auditing
log_destination = 'stderr'
logging_collector = on
log_directory = 'log'
log_filename = 'postgresql-%Y-%m-%d.log'
log_min_duration_statement = 250      # Log queries exceeding 250ms
```

7.5 Multi-Schema DDL Deployment Pipeline

Initialize the database structure using the automated initialization script (`docs/enterprise/scripts/postgresql/init-database.sh`):

```
#!/usr/bin/env bash
# InsAcc PostgreSQL Database Initialization Script [To Be Implemented]

set -euo pipefail

DB_NAME="insacc_db"
DB_USER="insacc_user"
DB_PASS="SecurePassword123"

echo "[INFO] Creating database '${DB_NAME}' and role '${DB_USER}'..."

sudo -u postgres psql <<EOF
DO \$\$
BEGIN
    IF NOT EXISTS (SELECT FROM pg_catalog.pg_roles WHERE rolname = '${DB_USER}') THEN
        CREATE ROLE ${DB_USER} WITH LOGIN PASSWORD '${DB_PASS}';
    END IF;
END
\$\$;

SELECT 'CREATE DATABASE ${DB_NAME} OWNER ${DB_USER}'
WHERE NOT EXISTS (SELECT FROM pg_database WHERE datname = '${DB_NAME}')\gexec

GRANT ALL PRIVILEGES ON DATABASE ${DB_NAME} TO ${DB_USER};
EOF

echo "[INFO] Deploying 5-Schema DDL Architecture..."
sudo -u postgres psql -d ${DB_NAME} <<EOF
CREATE SCHEMA IF NOT EXISTS accounting AUTHORIZATION ${DB_USER};
CREATE SCHEMA IF NOT EXISTS investment AUTHORIZATION ${DB_USER};
CREATE SCHEMA IF NOT EXISTS property AUTHORIZATION ${DB_USER};
CREATE SCHEMA IF NOT EXISTS auth AUTHORIZATION ${DB_USER};
CREATE SCHEMA IF NOT EXISTS audit AUTHORIZATION ${DB_USER};
EOF
```

7.6 Automated Balance Validation Trigger Functions

Deploy the PL/pgSQL trigger function to guarantee double-entry balance equality ($\sum D = \sum C$):

```
-- Connect to insacc_db
\c insacc_db

CREATE OR REPLACE FUNCTION accounting.fn_validate_voucher_balance()
RETURNS TRIGGER AS $$
DECLARE
    v_total_debit NUMERIC(15,2);
    v_total_credit NUMERIC(15,2);
BEGIN
    -- Calculate total debits and credits for the target voucher
    SELECT
        COALESCE(SUM(CASE WHEN line_type = 'Debit' THEN amount ELSE 0 END), 0),
        COALESCE(SUM(CASE WHEN line_type = 'Credit' THEN amount ELSE 0 END), 0)
    INTO v_total_debit, v_total_credit
    FROM accounting.voucher_lines
    WHERE voucher_id = NEW.id;

    -- Assert balance equality
    IF ABS(v_total_debit - v_total_credit) >= 0.001 THEN
        RAISE EXCEPTION 'Voucher posting rejected: Unbalanced entries. Debits (%) != Credits (%)',
            v_total_debit, v_total_credit;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Attach trigger to voucher status updates
CREATE TRIGGER trg_validate_voucher_posting
BEFORE UPDATE OF status ON accounting.vouchers
FOR EACH ROW
WHEN (NEW.status = 'Posted')
EXECUTE FUNCTION accounting.fn_validate_voucher_balance();
```

7.7 Database Backup, WAL Archiving & Point-In-Time Recovery (PITR)

Automated Daily `pg_dump` Backup Script:

```
#!/usr/bin/env bash
# Daily PostgreSQL Backup Script

BACKUP_DIR="/var/backups/postgresql"
TIMESTAMP=$(date +'%Y-%m-%d_%H%M%S')
mkdir -p ${BACKUP_DIR}

# Execute compressed pg_dump export
pg_dump -h localhost -U insacc_user -F c -b -v -f "${BACKUP_DIR}/insacc_db_${TIMESTAMP}.dump" insacc_db

# Retention: Purge dumps older than 30 days
find ${BACKUP_DIR} -type f -name "insacc_db_*.dump" -mtime +30 -delete
```

Point-In-Time Recovery (PITR) Configuration:

In `postgresql.conf`:

```
archive_mode = on
archive_command = 'test ! -f /var/lib/postgresql/wal_archive/%f && cp %p /var/lib/postgresql/wal_archive/%f'
```

---

8. Summary

This guide defines the target database setup manual for InsAcc Target Server v2.0.0. By tuning Linux kernel parameters, securing network access via TLS 1.3 and `pg_hba.conf`, configuring memory allocations in `postgresql.conf`, deploying PL/pgSQL balance validation triggers, and setting up daily `pg_dump` backups, enterprise DBAs maintain a secure database infrastructure.

9. Chapter Appendix

Reference Configuration File Directory

Artifact Description	File Repository Location	Functional Purpose
PostgreSQL Tuning Config	`docs/enterprise/configs/postgresql/postgresql.conf`	Memory, WAL, and planner parameters
Authentication Config	`docs/enterprise/configs/postgresql/pg_hba.conf`	Network access & SCRAM-SHA-256 rules
Server Provisioning Script	`docs/enterprise/scripts/postgresql/server-setup.sh`	OS packages, firewall & sysctl tuning
Database Init Script	`docs/enterprise/scripts/postgresql/init-database.sh`	Roles, database & schema provisioning
Database DDL Specification	`docs/DATABASE_DESIGN_SPECIFICATION.md`	Relational table & column schemas

10. Glossary

- **DBA (Database Administrator)**: A software specialist who configures, maintains, and secures database management systems.
- **PL/pgSQL**: Procedural Language/PostgreSQL Structured Query Language, used to write stored procedures and triggers.
- **SCRAM-SHA-256**: Salted Challenge Response Authentication Mechanism using SHA-256, the recommended password authentication method in PostgreSQL.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Database Design Specification: docs/DATABASE_DESIGN_SPECIFICATION.md
- Target Database Migration Plan: Volume 02 Chapter 03
- API Contract Specification: docs/API_CONTRACT.md

InsAcc Double-Entry Accounting Engine Specification

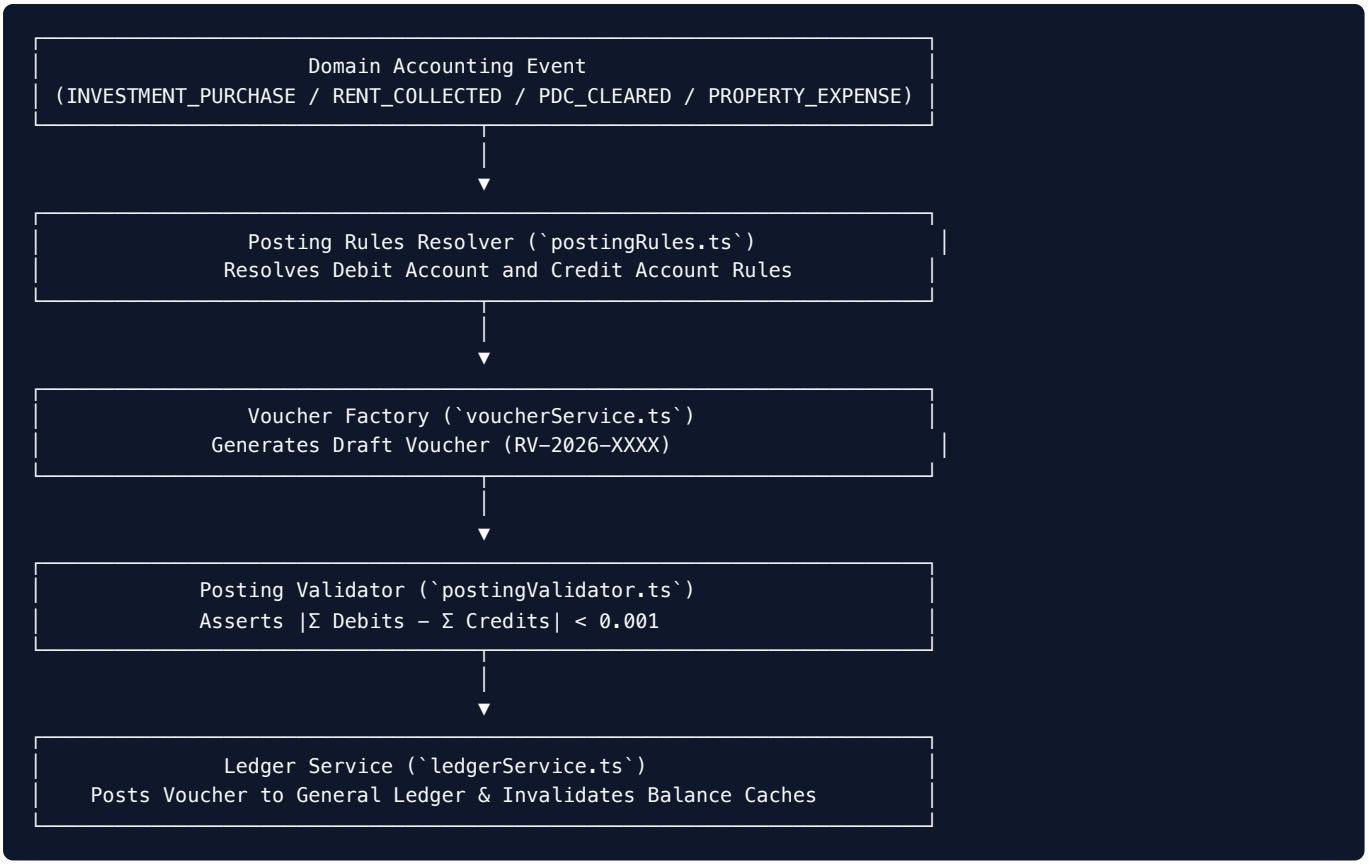
Document ID: ACCOUNTING_ENGINE_SPECIFICATION.md
Version: 1.0.0
Status: Official Accounting Engine Specification
Release Date: 2026-07-22
Target Software: InsAcc Enterprise Asset & Investment Accounting Platform v1.0.0
Single Source of Truth Reference: MASTER_ARCHITECTURE.md

> **IMPORTANT: GOVERNANCE DIRECTIVE**: This document defines the exact accounting rules, journal entries, posting algorithms, voucher lifecycle states, and financial statement calculation rules implemented in the InsAcc codebase (`src/renderer/accounting/`). All developer code, tests, and user documentation MUST adhere strictly to this specification.

Table of Contents

1. Engine Architecture & Core Principles

InsAcc operates an **event-driven, double-entry general ledger accounting engine**.



The Inviolable Core Principles

1. **Mathematical Balance Equality**: Every voucher posted to the General Ledger MUST satisfy: $$\sum \text{Debits} = \sum \text{Credits}$$
2. **Immutability of Posted Vouchers**: Once a voucher transitions to `Posted` status, its lines, amounts, dates, and account codes are permanently locked. Corrections MUST be executed via an explicit Reversal Voucher (`reverseVoucher()`).
3. **Derived Balance Rule**: Account balances are NEVER stored as independent, manually editable scalars. Balances are derived dynamically on read: $$\text{Current Balance} = \text{Opening Balance} + \sum \text{Debits} - \sum \text{Credits}$$

---

2. Chart of Accounts Architecture

The system Chart of Accounts (`src/renderer/accounting/chartOfAccountsService.ts`) organizes accounts into five standard categories:

Category Range	Category Name	Account Type	Normal Balance	Increases With	Decreases With
1000 – 1999	**Assets**	Asset	Debit (D)	Debit	Credit
2000 – 2999	**Liabilities**	Liability	Credit (C)	Credit	Debit
3000 – 3999	**Equity**	Equity	Credit (C)	Credit	Debit
4000 – 4999	**Revenue**	Revenue	Credit (C)	Credit	Debit
5000 – 5999	**Expenses**	Expense	Debit (D)	Debit	Credit

System Reserved Account Registry (`systemAccountRegistry.ts`)

```
export const SystemAccountRegistry = {
  CASH: '1110',               // Cash on Hand
  BANK: '1120',               // Bank Accounts (Parent Group)
  RENT_RECEIVABLE: '1130',    // Tenant Rent Receivable
  PDC_HELD: '1410',           // Post-Dated Cheques Held
  UNEARNED_RENT: '2110',      // Unearned / Deferred Rental Income
  SECURITY_DEPOSITS: '2120',  // Tenant Security Deposit Liability
  OWNER_CAPITAL: '2200',     // Owner Equity / Retained Earnings
  DIVIDEND_INCOME: '4110',    // Investment Dividends & Interest
  RENTAL_REVENUE: '4120',     // Earned Property Rental Revenue
  PROPERTY_EXPENSE: '5110',   // Property Maintenance Expense
  DEPRECIATION_EXP: '5190',   // Depreciation Expense
  ACCUMULATED_DEP: '1290',    // Accumulated Depreciation (Contra-Asset)
} as const
```

---

3. Voucher State Machine & Rules

Every voucher progresses through a formal 5-state lifecycle (`src/renderer/accounting/types.ts`):

State Definitions & Rules

1. **`Draft`** : Newly created voucher. Line amounts and accounts can be edited freely. Does NOT affect account balances.
2. **`Approved`** : Verified by supervisor. Pending ledger posting. Line items locked. Does NOT affect account balances.
3. **`Posted`** : Committed to General Ledger. Modifies derived account balances. Immutable.
4. **`Cancelled`** : Voided prior to posting. Preserved for audit sequence continuity.
5. **`Reversed`** : A posted voucher that has been offset by a corresponding Reversal Voucher.

---

4. Voucher Types Specification

4.1 Receipt Voucher (RV)

- **Purpose**: Record incoming money (cash deposits, bank receipts, tenant payments, dividend payouts).
- **Business Rule**: Must contain at least one Debit entry to Cash (`1110`) or Bank (`1120`), and one or more Credit entries to Revenue, Asset, or Liability accounts.
- **Journal Entry Pattern**:
  - $\text{Debit: Bank / Cash Account (1120 / 1110)}$ $\rightarrow$ $$X$
  - $\text{Credit: Revenue / Asset / Liability Account}$ $\rightarrow$ $$X$
- **Example**: Received AED 15,000 rent payment deposited into Emirates Islamic Bank.
  - $\text{Debit: 1120.001 Emirates Islamic Bank} = \text{AED 15,000}$
  - $\text{Credit: 2110 Unearned Rent Liability} = \text{AED 15,000}$
- **Validation**: `postingValidator.ts` verifies $\sum \text{Debits} = \sum \text{Credits}$ and asserts Bank/Cash debit presence.
- **Error Cases**: Unbalanced lines, selecting a closed account, negative line amounts.
- **Developer Notes**: Handled in UI via `InvestmentReceiptVoucher.tsx` and `PropertyReceiptVoucher.tsx`. Calls `voucherService.createVoucher()`.

4.2 Payment Voucher (PV)

- **Purpose**: Record outgoing money disbursements (vendor invoices, property repairs, asset purchases).
- **Business Rule**: Must contain at least one Credit entry to Cash (`1110`) or Bank (`1120`), and one or more Debit entries to Expense, Asset, or Liability accounts.
- **Journal Entry Pattern**:
  - $\text{Debit: Expense / Asset / Liability Account}$ $\rightarrow$ $$X$
  - $\text{Credit: Bank / Cash Account (1120 / 1110)}$ $\rightarrow$ $$X$
- **Example**: Paid AED 3,500 maintenance bill to Al Mas Plumbing from ADCB Bank.
  - $\text{Debit: 5110 Property Maintenance Expense} = \text{AED 3,500}$
  - $\text{Credit: 1120.002 ADCB Bank} = \text{AED 3,500}$
- **Validation**: Asserts $\sum D = \sum C > 0$ and presence of bank/cash credit line.
- **Error Cases**: Overdrawing non-overdraft accounts, missing expense sub-account.
- **Developer Notes**: Implemented in `InvestmentPaymentVoucher.tsx` and `PropertyPaymentVoucher.tsx`.

4.3 Journal Voucher (JV)

- **Purpose**: Record non-cash accounting adjustments, accruals, depreciation, PDC clearance, and period closing entries.
- **Business Rule**: Multi-line general journal entries allowing arbitrary accounts. Total debits must equal total credits.
- **Journal Entry Pattern**:
  - Debit: Account A $\rightarrow$ $$X$
  - Credit: Account B $\rightarrow$ $$X$
- **Example**: Recognize monthly earned rental revenue of AED 10,000 from deferred unearned rent.
  - $\text{Debit: 2110 Unearned Rent Liability} = \text{AED 10,000}$
  - $\text{Credit: 4120 Rental Revenue} = \text{AED 10,000}$
- **Validation**: Strict balance check ($\left| \sum D - \sum C \right| < 0.001$).

- **Error Cases**: Single-line entry, unbalanced debits/credits.
- **Developer Notes**: Managed via `InvestmentJournalVoucher.tsx` and `PropertyJournalVoucher.tsx`.

4.4 Contra Voucher (CV)

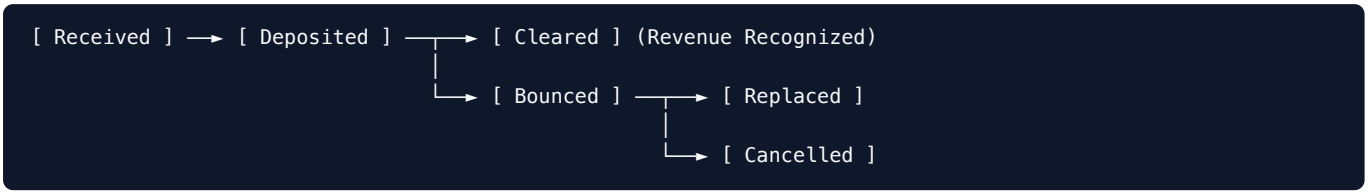
- **Purpose**: Record internal fund transfers between cash and bank accounts without affecting revenue or expense accounts.
- **Business Rule**: All debit and credit lines must be restricted to Asset accounts of type `Cash` (`1110`) or `Bank` (`1120`).
- **Journal Entry Pattern**:
  - $\text{Debit: Destination Bank / Cash Account}$ $\rightarrow$ $$X$
  - $\text{Credit: Source Bank / Cash Account}$ $\rightarrow$ $$X$
- **Example**: Transfer AED 50,000 cash from Petty Cash Vault to Emirates Islamic Bank.
  - $\text{Debit: 1120.001 Emirates Islamic Bank} = \text{AED 50,000}$
  - $\text{Credit: 1110 Petty Cash Vault} = \text{AED 50,000}$
- **Validation**: Asserts all line account codes belong to `1110` or `1120` series.
- **Error Cases**: Including revenue or expense accounts in a contra transfer.
- **Developer Notes**: Handled in `bankTransactionService.ts` inter-account transfer handlers.

---

5. Domain Event Accounting Rules

5.1 PDC Cheque Lifecycle Workflow

Post-Dated Cheques (PDC) progress through a strict 5-stage accounting lifecycle:



Event 1: `PDC_RECEIVED` (Lease Signing)

- **Purpose**: Record receipt of post-dated cheque from tenant upon lease contract signing.
- **Business Rule**: Holds cheque in safe vault. Recognizes liability for unearned future rent.
- **Journal Entry**:
  - $\text{Debit: 1410 Post-Dated Cheques Held (Asset)} = \text{Face Value}$
  - $\text{Credit: 2110 Unearned Rent (Liability)} = \text{Face Value}$
- **Example**: Received 4 quarterly cheques of AED 30,000 each (Total AED 120,000).
  - $\text{Debit: 1410 PDC Held} = \text{AED 120,000}$
  - $\text{Credit: 2110 Unearned Rent} = \text{AED 120,000}$
- **Validation**: Maturity date must be in the future.
- **Developer Notes**: Executed in `propertyPdcService.ts` during lease creation.

Event 2: `PDC_DEPOSITED` (Maturity Date Reached)

- **Purpose**: Send cheque to bank for collection.
- **Business Rule**: Updates internal tracking status from `Received` to `Deposited`.
- **Journal Entry**: Memo status entry; no net balance change until cleared.
- **Developer Notes**: Managed via `PropertyPdcManager.tsx`.

Event 3: `PDC_CLEARED` (Funds Confirmed by Bank)

- **Purpose**: Recognize cleared cash and convert unearned rent to earned rental revenue.
- **Business Rule**: Bank balance increases; PDC held asset decreases. Unearned rent liability decreases; rental revenue increases.
- **Journal Entry**:
  - $\text{Debit: 1120 Bank Account (Asset)} = \text{Cheque Amount}$
  - $\text{Credit: 1410 PDC Held (Asset)} = \text{Cheque Amount}$
  - $\text{Debit: 2110 Unearned Rent (Liability)} = \text{Cheque Amount}$
  - $\text{Credit: 4120 Rental Revenue (Revenue)} = \text{Cheque Amount}$
- **Example**: Cheque 1 (AED 30,000) clears successfully.
  - $\text{Debit: 1120.001 Emirates Islamic Bank} = \text{AED 30,000}$
  - $\text{Credit: 1410 PDC Held} = \text{AED 30,000}$
  - $\text{Debit: 2110 Unearned Rent} = \text{AED 30,000}$
  - $\text{Credit: 4120 Rental Revenue} = \text{AED 30,000}$
- **Validation**: Cheque status must be `Deposited`.
- **Developer Notes**: Triggered via `transitionPdcCheque(chequeId, 'Cleared')`.

Event 4: `PDC_BOUNCED` (Cheque Rejected)

- **Purpose**: Record bank rejection of a deposited cheque (insufficient funds).
- **Business Rule**: Reverses deposit and transfers balance to tenant Rent Receivable (`1130`).
- **Journal Entry**:
  - $\text{Debit: 1130 Rent Receivable (Asset)} = \text{Cheque Amount}$
  - $\text{Credit: 1410 PDC Held (Asset)} = \text{Cheque Amount}$
- **Developer Notes**: Executed via `propertyPdcService.ts` bounced cheque handler.

5.2 Property Lease & Security Deposit Accounting

Event 1: `SECURITY_DEPOSIT_RECEIVED`

- **Purpose**: Record tenant security deposit payment.
- **Business Rule**: Security deposits are fiduciary liabilities owed back to the tenant upon lease termination.
- **Journal Entry**:
  - $\text{Debit: 1120 Bank Account (Asset)} = \text{Deposit Amount}$
  - $\text{Credit: 2120 Tenant Security Deposits (Liability)} = \text{Deposit Amount}$
- **Developer Notes**: Managed in `propertyDepositService.ts`.

Event 2: `SECURITY_DEPOSIT_REFUNDED`

- **Purpose**: Refund security deposit to tenant upon move-out inspection.
- **Business Rule**: Reduces deposit liability and bank balance. Deducts damages if applicable.
- **Journal Entry**:
  - $\text{Debit: 2120 Tenant Security Deposits (Liability)} = \text{Full Deposit}$
  - $\text{Credit: 1120 Bank Account (Asset)} = \text{Refund Amount}$
  - $\text{Credit: 5110 Maintenance Expense (Repair Offset)} = \text{Damage Deductions}$

5.3 Investment Portfolio Accounting

Event 1: `INVESTMENT_PURCHASE`

- **Purpose**: Record purchase of stocks, bonds, or mutual funds.
- **Business Rule**: Capitalizes purchase cost (quantity $\times$ price + fees) into Asset account.
- **Journal Entry**:
  - $\text{Debit: 1200 Investment Assets (Asset)} = \text{Total Cost Basis}$
  - $\text{Credit: 1120 Bank Account (Asset)} = \text{Total Cost Basis}$
- **Developer Notes**: Executed via `investmentAccountingService.ts`.

Event 2: `INVESTMENT_DIVIDEND`

- **Purpose**: Record dividend or coupon interest received.
- **Business Rule**: Recognizes investment revenue without reducing asset principal cost basis.
- **Journal Entry**:
  - $\text{Debit: 1120 Bank Account (Asset)} = \text{Dividend Amount}$
  - $\text{Credit: 4110 Dividend & Interest Income (Revenue)} = \text{Dividend Amount}$

5.4 Physical Asset & Purchase Ledger Accounting

- **Purpose**: Track physical bullion bars, gold coins, and physical commodities.
- **Business Rule**: Purchases are recorded in `insacc_purchases`. Weighted average cost per unit is calculated dynamically: $$\text{Weighted Avg Cost} = \frac{\sum (\text{Lot Qty}_i \times \text{Lot Price}_i)}{\sum \text{Lot Qty}_i}$$
- **Developer Notes**: Service functions in `purchaseAccountingService.ts` and `purchaseLedgerService.ts`.

5.5 Asset Depreciation Accounting

- **Purpose**: Record periodic depreciation of fixed physical assets or real estate improvements.
- **Business Rule**: Depreciates asset over useful life using straight-line method: $$\text{Monthly Depreciation} = \frac{\text{Historical Cost} - \text{Salvage Value}}{\text{Useful Life Months}}$$
- **Journal Entry**:
  - $\text{Debit: 5190 Depreciation Expense (Expense)} = \text{Monthly Depreciation}$
  - $\text{Credit: 1290 Accumulated Depreciation (Contra-Asset)} = \text{Monthly Depreciation}$

---

6. Financial Statement Calculation Engines

6.1 General Ledger & Sub-Ledger Projection Engine

- **Source**: `ledgerService.ts`
- **Function**: Queries all `Posted` vouchers and calculates chronological running balance for every account: $$\text{Running Balance}*t* = |*text{Running Balance}*{t-1} + \text{Debit}_t - \text{Credit}_t$$

6.2 Trial Balance Calculation Engine

- **Source**: `src/renderer/components/TrialBalanceTree.tsx` / `reportService.ts`
- **Function**: Sums closing debit and credit balances across all active accounts: $$\sum \text{Closing Debits} = \sum \text{Closing Credits}$$

6.3 Profit & Loss Statement Calculation Engine

- **Source**: `reportService.ts` (`calculateFinancialSummary()`)
- **Formulas**:
  - $\text{Total Revenue} = \sum \text{Account Balances (4000-4999)}$
  - $\text{Total Expense} = \sum \text{Account Balances (5000-5999)}$
  - $\text{Net Profit / (Loss)} = \text{Total Revenue} - \text{Total Expense}$

6.4 Balance Sheet Calculation Engine

- **Source**: `reportService.ts`
- **Fundamental Accounting Equation**: $$\text{Total Assets} = \text{Total Liabilities} + \text{Total Equity} + \text{Net Profit}$$ Where:
  - $\text{Total Assets} = \sum \text{Account Balances (1000-1999)}$
  - $\text{Total Liabilities} = \sum \text{Account Balances (2000-2999)}$
  - $\text{Total Equity} = \sum \text{Account Balances (3000-3999)}$

---

7. Opening Balances, Closing Entries & Year-End Closing

Year-End Fiscal Closing Pipeline (`periodCloser.ts` / `PeriodClosingWizard.tsx`)

At the close of a fiscal year, the system executes the following steps:

1. **Lock Fiscal Period**: Set period status to `Closed`. Prevent new voucher postings in period range.
2. **Generate Year-End Closing Journal Voucher**:
   - Zero out all Revenue accounts (`4000–4999`):
     - $\text{Debit: Revenue Accounts}$ $\rightarrow$ Full Balance
   - Zero out all Expense accounts (`5000–5999`):
     - $\text{Credit: Expense Accounts}$ $\rightarrow$ Full Balance
   - Net difference transferred to Owner Equity / Retained Earnings (`2200`):
     - $\text{Credit (if Profit) / Debit (if Loss): 2200 Retained Earnings}$
3. **Carry Forward Opening Balances**:
   - Asset, Liability, and Equity balances carry forward as Opening Balances for the new fiscal year.
   - Revenue and Expense opening balances reset to `0.00` .

---

8. Audit Trail & Operational Log Requirements

InsAcc mandates complete audit accountability for all financial events (`auditService.ts`):

```
export interface LogEntry {
  id: string              // Unique log identifier
  timestamp: string       // ISO 8601 UTC timestamp
  user: string            // User ID / Name executing action
  action: string          // Operational action (e.g. "VOUCHER_POSTED")
  details: string         // Detailed description of change
}
```

Immutable Audit Rules:

- Every voucher state change (`Draft` $\rightarrow$ `Approved` $\rightarrow$ `Posted` $\rightarrow$ `Reversed`) writes a non-deletable `LogEntry` to `insacc_logs`.
- Reversal vouchers preserve the original voucher ID in `reference` for audit tracing.
- Audit logs cannot be cleared or modified by non-admin users.

End of Double-Entry Accounting Engine Specification.

InsAcc Interface & API Contract Specification

Document ID: API_CONTRACT.md
Version: 1.0.0
Status: Official Interface & API Specification
Release Date: 2026-07-22
Target Software: InsAcc Enterprise Asset & Investment Accounting Platform v1.0.0
Single Source of Truth Reference: MASTER_ARCHITECTURE.md

> **IMPORTANT: GOVERNANCE DIRECTIVE**: InsAcc v1.0.0 runs as a local desktop application using Electron 28. The desktop process communicates via the Electron IPC bridge (`window.api.saveFile()`). All REST API HTTP endpoints documented in Section 3 represent the **Target Enterprise Server API Contract [To Be Implemented]** for server release v2.0.0.

Table of Contents

1. API Architecture & Protocol Overview

InsAcc interfaces operate across two architectural layers:

1. **Desktop Client IPC Layer (Active v1.0.0)**: Secure asynchronous IPC bridge (`ipcRenderer.invoke` / `ipcMain.handle`) connecting the React renderer process to the Electron main process via `src/main/preload.js` .
2. **Server REST API Layer [To Be Implemented]**: Target OpenAPI 3.0 compliant HTTPS REST API communicating over TLS 1.3 with JSON payloads and Bearer JWT authentication.

---

2. Desktop Electron IPC Bridge API (v1.0.0 Production)

2.1 IPC Channel: `save-file`

- **Purpose**: Writes formatted financial reports (CSV, PDF, Excel) or system backup snapshots to the client filesystem.
- **HTTP Method / Protocol**: Electron IPC Protocol (`ipcRenderer.invoke('save-file', payload)`).
- **Authentication**: Native OS local desktop context (No HTTP auth header required).
- **Headers**: Internal Electron Context Bridge.
- **Request (JavaScript Object Payload)**:

```
interface SaveFilePayload {
  filename: string  // Target filename (e.g. "Balance_Sheet_2026-06-30.csv")
  content: string   // Raw text, CSV string, or Base64 file payload
}
```

- **Validation**:
  - `filename` must be a non-empty string ending with valid extension (`.csv` , `.pdf` , `.xlsx` , `.json` , `.txt`).
  - `content` must be a non-empty string payload.
- **Response**: Returns a `Promise<string>` resolving to the absolute output filepath on disk.

```
// Resolves to absolute path:
"C:\\Users\\Username\\Downloads\\Balance_Sheet_2026-06-30.csv"
```

- **Status Codes**:
  - `SUCCESS` : File written successfully to disk.
  - `EACCES` : Permission denied writing to Downloads directory.
  - `ENOSPC` : No disk space available.
- **Error Codes**: `ERR_IPC_BRIDGE_FAILED` , `ERR_FILE_WRITE_DENIED` .
- **Permissions**: Desktop user account file writing privileges.
- **Examples**:

```
// Renderer Call (Reports.tsx)
const outputPath = await window.api.saveFile('Trial_Balance_2026.csv', csvContent)
console.log('Saved to:', outputPath)
```

- **Rate Limits**: Unrestricted local IPC invocations.
- **Future Extensions**: Support custom user-selected target save directories via native OS Save Dialog.

---

3. Target Enterprise REST API Contract `[To Be Implemented]`

3.1 Authentication Endpoints

`POST /api/v1/auth/login` `[To Be Implemented]`

- **Purpose**: Authenticates user credentials and issues a JWT Bearer Access Token.
- **HTTP Method**: `POST`
- **Authentication**: None (Public Endpoint).
- **Headers**:

```
Content-Type: application/json
```

- **Request Body**:

```
{
  "email": "admin@insacc.com",
  "password": "SecurePassword123"
}
```

- **Validation**:
  - `email`: Must be a valid email string format.
  - `password`: Must be non-empty string, min 8 characters.
- **Response (`200 OK`)**:

```
{
  "status": "success",
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "tokenType": "Bearer",
    "expiresIn": 86400,
    "user": {
      "id": "u-1719500000000",
      "email": "admin@insacc.com",
      "role": "Admin"
    }
  }
}
```

- **Status Codes**: `200 OK`, `400 Bad Request`, `401 Unauthorized`, `429 Too Many Requests`.
- **Error Codes**: `AUTH_INVALID_CREDENTIALS`, `AUTH_ACCOUNT_LOCKED`.
- **Permissions**: Public.
- **Examples**:

```
curl -X POST https://erp.insacc.com/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@insacc.com","password":"SecurePassword123"}'
```

- **Rate Limits**: 10 requests / minute per IP address.
- **Future Extensions**: Multi-factor authentication (MFA / TOTP) support.

`POST /api/v1/auth/refresh` `[To Be Implemented]`

- **Purpose**: Refreshes an expired JWT Access Token using a valid Refresh Token.

- **HTTP Method**: `POST`
- **Authentication**: Refresh Token Cookie / Bearer.
- **Headers**: `Content-Type: application/json`
- **Request Body**: `{"refreshToken": "d9a8c7b6..."}`
- **Validation**: `refreshToken` must be active and non-revoked.
- **Response (`200 OK`)**: Returns new `accessToken` and `expiresIn`.
- **Status Codes**: `200 OK`, `401 Unauthorized`.
- **Error Codes**: `AUTH_TOKEN_EXPIRED`, `AUTH_TOKEN_INVALID`.
- **Permissions**: Public.
- **Examples**: `curl -X POST https://erp.insacc.com/api/v1/auth/refresh -d '{"refreshToken":"..."}'`
- **Rate Limits**: 20 requests / minute per IP.
- **Future Extensions**: Automated token rotation with reuse detection.

3.2 Investment Module Endpoints

`GET /api/v1/investments` **`[To Be Implemented]`**

- **Purpose**: Retrieves all active portfolio investment holdings and valuation metrics.
- **HTTP Method**: `GET`
- **Authentication**: Bearer JWT Access Token.
- **Headers**: `Authorization: Bearer <JWT_TOKEN>`
- **Request Query Parameters**: `?type=Gold&status=active`
- **Validation**: `type` must match known asset classes (`Gold`, `Silver`, `Stocks`, `Bonds`, `ETFs`).
- **Response (`200 OK`)**:

```
{
  "status": "success",
  "data": [
    {
      "id": "inv-1719500000000",
      "assetName": "24K Gold Bar 1kg",
      "assetType": "Gold",
      "quantity": 2.0,
      "purchaseValue": 500000.00,
      "currentPrice": 285000.00,
      "marketValue": 570000.00,
      "unrealizedGain": 70000.00,
      "returnPercentage": 14.0
    }
  ]
}
```

- **Status Codes**: `200 OK`, `401 Unauthorized`, `403 Forbidden`.
- **Error Codes**: `INVESTMENT_NOT_FOUND`, `UNAUTHORIZED_ACCESS`.
- **Permissions**: Roles: `Admin`, `Accounts`.
- **Examples**: `curl -H "Authorization: Bearer $TOKEN" https://erp.insacc.com/api/v1/investments`
- **Rate Limits**: 100 requests / minute per user.
- **Future Extensions**: Live market feed ticker price synchronization.

`POST /api/v1/investments` `[To Be Implemented]`

- **Purpose**: Records a new investment holding position.
- **HTTP Method**: `POST`
- **Authentication**: Bearer JWT Access Token.
- **Headers**: `Authorization: Bearer <JWT_TOKEN>`, `Content-Type: application/json`
- **Request Body**:

```
{
  "assetName": "Apple Inc. (AAPL)",
  "assetType": "Stocks",
  "quantity": 100,
  "purchaseValue": 65000.00,
  "currentPrice": 720.00
}
```

- **Validation**:
  - `assetName`: Required string, 1-255 characters.
  - `quantity`: Finite positive number > 0.
  - `purchaseValue`: Finite positive number ≥ 0.
- **Response (`201 Created`)**: Returns created investment object with assigned `id`.
- **Status Codes**: `201 Created`, `400 Bad Request`, `401 Unauthorized`.
- **Error Codes**: `VALIDATION_FAILED`, `DUPLICATE_ASSET_NAME`.
- **Permissions**: Roles: `Admin`, `Accounts`.
- **Examples**: `curl -X POST -H "Authorization: Bearer $TOKEN" https://erp.insacc.com/api/v1/investments -d '{...}'`
- **Rate Limits**: 30 requests / minute.
- **Future Extensions**: Link asset purchase to a specific bank account for automated cash deduction.

`POST /api/v1/investments/purchases` `[To Be Implemented]`

- **Purpose**: Records an asset purchase lot in the Purchase Ledger.
- **HTTP Method**: `POST`
- **Authentication**: Bearer JWT Access Token.
- **Headers**: `Authorization: Bearer <JWT_TOKEN>`, `Content-Type: application/json`
- **Request Body**:

```
{
  "purchaseDate": "2026-06-15",
  "assetType": "Gold",
  "assetName": "24K Gold Bar 100g",
  "quantity": 5,
  "unitPrice": 28500.00
}
```

- **Validation**: `quantity` > 0, `unitPrice` ≥ 0, `purchaseDate` valid ISO date.
- **Response (`201 Created`)**: Returns created purchase record and updated item weighted average cost.
- **Status Codes**: `201 Created`, `400 Bad Request`.
- **Error Codes**: `PURCHASE_INVALID_DATA`.
- **Permissions**: Roles: `Admin`, `Accounts`.

- **Examples**: `curl -X POST -H "Authorization: Bearer $TOKEN" https://erp.insacc.com/api/v1/investments/purchases -d '{...}'`
- **Rate Limits**: 30 requests / minute.
- **Future Extensions**: Automated FIFO/LIFO tax lot allocation.

3.3 Property Management Endpoints

`GET /api/v1/properties/units` **`[To Be Implemented]`**

- **Purpose**: Retrieves all property units with occupancy status and rent rates.
- **HTTP Method**: `GET`
- **Authentication**: Bearer JWT Access Token.
- **Headers**: `Authorization: Bearer <JWT_TOKEN>`
- **Query Parameters**: `?buildingId=bld-101&status=Vacant`
- **Validation**: `status` must be `Occupied`, `Vacant`, or `Under Maintenance`.
- **Response (`200 OK`)**: List of unit records with building names and current lease attachments.
- **Status Codes**: `200 OK`, `401 Unauthorized`.
- **Error Codes**: `PROPERTY_INVALID_BUILDING`.
- **Permissions**: Roles: `Admin`, `Accounts`.
- **Examples**: `curl -H "Authorization: Bearer $TOKEN" https://erp.insacc.com/api/v1/properties/units`
- **Rate Limits**: 100 requests / minute.
- **Future Extensions**: Unit floor plan vector diagram links.

`POST /api/v1/properties/leases` **`[To Be Implemented]`**

- **Purpose**: Registers a tenant contract, activates unit lease, and generates PDC cheque entries.
- **HTTP Method**: `POST`
- **Authentication**: Bearer JWT Access Token.
- **Headers**: `Authorization: Bearer <JWT_TOKEN>`, `Content-Type: application/json`
- **Request Body**:

```
{
  "tenantName": "John Doe",
  "nationalId": "784-1990-1234567-1",
  "phone": "+971501234567",
  "unitId": "unit-101",
  "startDate": "2026-07-01",
  "endDate": "2027-06-30",
  "annualRent": 120000.00,
  "paymentFrequency": "Quarterly",
  "securityDeposit": 5000.00
}
```

- **Validation**: `unitId` must reference a unit in `Vacant` status. `endDate` must be $> \text{startDate}$.
- **Response (`201 Created`)**: Returns created lease record, updated unit status (`Occupied`), and generated PDC cheque IDs.
- **Status Codes**: `201 Created`, `400 Bad Request`, `409 Conflict`.
- **Error Codes**: `UNIT_ALREADY_OCCUPIED`, `INVALID_LEASE_DATES`.
- **Permissions**: Roles: `Admin`, `Accounts`.

- **Examples**: `curl -X POST -H "Authorization: Bearer $TOKEN" https://erp.insacc.com/api/v1/properties/leases -d '{...}'`
- **Rate Limits**: 20 requests / minute.
- **Future Extensions**: Digital lease document e-signature workflow.

`POST /api/v1/properties/pdc/transition` **`[To Be Implemented]`**

- **Purpose**: Transitions a Post-Dated Cheque through its state machine (`Deposited`, `Cleared`, `Bounced`).
- **HTTP Method**: `POST`
- **Authentication**: Bearer JWT Access Token.
- **Headers**: `Authorization: Bearer <JWT_TOKEN>`, `Content-Type: application/json`
- **Request Body**:

```
{
  "pdcId": "pdc-1001",
  "targetStatus": "Cleared",
  "bankAccountId": "ba-1719500000000",
  "transitionDate": "2026-07-01"
}
```

- **Validation**: `targetStatus` must be valid state machine transition (`Received` $\rightarrow$ `Deposited` $\rightarrow$ `Cleared` / `Bounced`).
- **Response (`200 OK`)**: Returns updated PDC record and generated accounting voucher ID.
- **Status Codes**: `200 OK`, `400 Bad Request`, `422 Unprocessable Entity`.
- **Error Codes**: `PDC_INVALID_STATE_TRANSITION`, `PDC_NOT_FOUND`.
- **Permissions**: Roles: `Admin`, `Accounts`.
- **Examples**: `curl -X POST -H "Authorization: Bearer $TOKEN" https://erp.insacc.com/api/v1/properties/pdc/transition -d '{...}'`
- **Rate Limits**: 30 requests / minute.
- **Future Extensions**: Automated bank API feed PDC clearance matching.

3.4 Double-Entry Accounting Endpoints

`GET /api/v1/accounting/accounts` **`[To Be Implemented]`**

- **Purpose**: Retrieves the complete Chart of Accounts hierarchy tree.
- **HTTP Method**: `GET`
- **Authentication**: Bearer JWT Access Token.
- **Headers**: `Authorization: Bearer <JWT_TOKEN>`
- **Response (`200 OK`)**: Complete account tree array with calculated running balances.
- **Status Codes**: `200 OK`, `401 Unauthorized`.
- **Permissions**: Roles: `Admin`, `Accounts`.
- **Examples**: `curl -H "Authorization: Bearer $TOKEN" https://erp.insacc.com/api/v1/accounting/accounts`
- **Rate Limits**: 60 requests / minute.
- **Future Extensions**: Filter accounts by specific fiscal cost center.

`POST /api/v1/accounting/vouchers` `[To Be Implemented]`

- **Purpose**: Creates a new double-entry accounting voucher (`RV` , `PV` , `JV`).
- **HTTP Method**: `POST`
- **Authentication**: Bearer JWT Access Token.
- **Headers**: `Authorization: Bearer <JWT_TOKEN>` , `Content-Type: application/json`
- **Request Body**:

```
{
  "voucherType": "Receipt",
  "voucherDate": "2026-06-15",
  "narration": "Rent collection Unit 101",
  "lines": [
    {
      "accountId": "1120.001",
      "lineType": "Debit",
      "amount": 10000.00,
      "memo": "Deposit to Emirates Islamic Bank"
    },
    {
      "accountId": "4120",
      "lineType": "Credit",
      "amount": 10000.00,
      "memo": "Rental Revenue"
    }
  ]
}
```

- **Validation**:
  - `voucherType` : Must be `Receipt` , `Payment` , or `Journal` .
  - `lines` : Must contain ≥ 2 entries.
  - **Balance Assertion**: $\sum \text{Debits} = \sum \text{Credits}$ ($\left| \sum D - \sum C \right| < 0.001$).
- **Response (`201 Created`)**:

```
{
  "status": "success",
  "data": {
    "id": "v-1719500000000",
    "voucherNumber": "RV-2026-0042",
    "status": "Draft",
    "createdAt": "2026-06-15T10:00:00Z"
  }
}
```

- **Status Codes**: `201 Created` , `400 Bad Request` , `422 Unprocessable Entity` .
- **Error Codes**: `VOUCHER_UNBALANCED` , `ACCOUNT_NOT_FOUND` , `INVALID_LINE_AMOUNT` .
- **Permissions**: Roles: `Admin` , `Accounts` .
- **Examples**: `curl -X POST -H "Authorization: Bearer $TOKEN" https://erp.insacc.com/api/v1/accounting/vouchers -d '{...}'`
- **Rate Limits**: 30 requests / minute.
- **Future Extensions**: Multi-currency line item conversion.

`POST /api/v1/accounting/vouchers/{id}/post` `[To Be Implemented]`

- **Purpose**: Posts an approved voucher to the General Ledger, rendering it immutable.

- **HTTP Method**: `POST`
- **Authentication**: Bearer JWT Access Token.
- **Headers**: `Authorization: Bearer <JWT_TOKEN>`
- **Response (`200 OK`)**: Returns posted voucher object with `status: "Posted"` and `postedAt` timestamp.
- **Status Codes**: `200 OK`, `400 Bad Request`, `409 Conflict`.
- **Error Codes**: `VOUCHER_ALREADY_POSTED`, `VOUCHER_UNAPPROVED`.
- **Permissions**: Roles: `Admin`, `Accounts`.
- **Examples**: `curl -X POST -H "Authorization: Bearer $TOKEN" https://erp.insacc.com/api/v1/accounting/vouchers/v-1001/post`
- **Rate Limits**: 30 requests / minute.
- **Future Extensions**: Automated email notification to auditors upon posting large vouchers.

`POST /api/v1/accounting/vouchers/{id}/reverse` `[To Be Implemented]`

- **Purpose**: Creates an offsetting Reversal Voucher for a posted transaction.
- **HTTP Method**: `POST`
- **Authentication**: Bearer JWT Access Token.
- **Headers**: `Authorization: Bearer <JWT_TOKEN>`, `Content-Type: application/json`
- **Request Body**: `{"reversalReason": "Correction of duplicate entry"}`
- **Validation**: Voucher `{id}` must have `status === "Posted"`.
- **Response (`201 Created`)**: Returns newly created Reversal Voucher object and updates target voucher status to `Reversed`.
- **Status Codes**: `201 Created`, `400 Bad Request`, `409 Conflict`.
- **Error Codes**: `VOUCHER_NOT_POSTED`, `VOUCHER_ALREADY_REVERSED`.
- **Permissions**: Roles: `Admin` only.
- **Examples**: `curl -X POST -H "Authorization: Bearer $TOKEN" https://erp.insacc.com/api/v1/accounting/vouchers/v-1001/reverse -d '{...}'`
- **Rate Limits**: 10 requests / minute.
- **Future Extensions**: Automatic reversal pairing audit reports.

3.5 Financial Statement Endpoints

`GET /api/v1/reports/balance-sheet` `[To Be Implemented]`

- **Purpose**: Generates the Balance Sheet financial statement as of a target date.
- **HTTP Method**: `GET`
- **Authentication**: Bearer JWT Access Token.
- **Query Parameters**: `?asOfDate=2026-06-30`
- **Response (`200 OK`)**:

```
{
  "status": "success",
  "asOfDate": "2026-06-30",
  "currency": "AED",
  "data": {
    "totalAssets": 1275000.00,
    "totalLiabilities": 50000.00,
    "totalEquity": 1225000.00,
    "netProfit": 0.00,
    "isBalanced": true
  }
}
```

- **Status Codes**: `200 OK`, `401 Unauthorized`.
- **Permissions**: Roles: `Admin`, `Accounts`.
- **Examples**: `curl -H "Authorization: Bearer $TOKEN" https://erp.insacc.com/api/v1/reports/balance-sheet?asOfDate=2026-06-30`
- **Rate Limits**: 60 requests / minute.
- **Future Extensions**: Comparative prior period side-by-side reporting.

`GET /api/v1/reports/profit-loss` **`[To Be Implemented]`**

- **Purpose**: Generates the Profit & Loss statement over a specified date range.
- **HTTP Method**: `GET`
- **Authentication**: Bearer JWT Access Token.
- **Query Parameters**: `?fromDate=2026-01-01&toDate=2026-06-30`
- **Response (`200 OK`)**: Returns Total Revenue, Total Expense, Net Profit, and Category breakdowns.
- **Status Codes**: `200 OK`, `400 Bad Request`.
- **Error Codes**: `INVALID_DATE_RANGE`.
- **Permissions**: Roles: `Admin`, `Accounts`.
- **Examples**: `curl -H "Authorization: Bearer $TOKEN" https://erp.insacc.com/api/v1/reports/profit-loss?fromDate=2026-01-01&toDate=2026-06-30`
- **Rate Limits**: 60 requests / minute.
- **Future Extensions**: Multi-currency conversion for foreign operations.

---

4. Standard Error Code Reference

All target REST API endpoints return errors in a standardized RFC 7807 JSON payload:

```
{
  "status": "error",
  "errorCode": "VOUCHER_UNBALANCED",
  "message": "Voucher lines are unbalanced. Total Debits (AED 10,000.00) != Total Credits (AED 9,500.00).",
  "timestamp": "2026-07-22T14:30:00.000Z",
  "path": "/api/v1/accounting/vouchers"
}
```

Standard Error Dictionary

Error Code	HTTP Status	Root Cause & Resolution
`AUTH_INVALID_CREDENTIALS`	401	Incorrect email or password. Verify credentials.
`AUTH_TOKEN_EXPIRED`	401	JWT token expired. Issue request to `/auth/refresh`.
`UNAUTHORIZED_ACCESS`	403	User role lacks permissions for target resource.
`VOUCHER_UNBALANCED`	422	Debit line total does not equal Credit line total.
`VOUCHER_ALREADY_POSTED`	409	Attempted to post an already posted voucher.
`UNIT_ALREADY_OCCUPIED`	409	Attempted to assign lease to non-vacant unit.
`PDC_INVALID_STATE_TRANSITION`	422	Invalid PDC transition (e.g. `Received` $\rightarrow$ `Cleared` without `Deposited`).
`RATE_LIMIT_EXCEEDED`	429	Exceeded API rate limits. Retry after `Retry-After` seconds.
`INTERNAL_SERVER_ERROR`	500	Unexpected server error. Contact engineering support.

End of API Contract Specification.

title: "Volume 01: System Installation & Deployment Guide - Chapter 01: System Requirements and Prerequisites" document_id: "INSACC-DOC-V01-CH01" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 01: Installation & Deployment Guide

Chapter 01: System Requirements and Prerequisites

> **Single Source of Truth Reference**: All hardware, operating system, network, and runtime specifications defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter defines the technical hardware, operating system, network port allocation, and environmental prerequisites required to deploy, execute, and maintain the InsAcc Enterprise Asset & Investment Accounting Platform v1.0.0 across workstation client endpoints and future enterprise server infrastructure.

2. Scope

This specification covers:

- Local workstation client hardware and display requirements for standalone desktop execution.
- Operating system compatibility across Microsoft Windows, Apple macOS, and Linux distributions.
- Embedded runtime specifications (Chromium 120.x, Node.js 18.x embedded in Electron 28.x).
- Target enterprise server hardware and database cluster specifications `[To Be Implemented]`.
- Network firewall, security software exclusions, and local port allocation.

Out of Scope:

- Interactive installation setup wizard steps (covered in Volume 01 Chapter 02).
- Source code compilation procedures (covered in Volume 01 Chapter 03).

3. Audience

This document is authored for:

- Enterprise IT Infrastructure & Desktop Support Administrators
- DevOps Engineers and System Architects
- Information Security & Endpoint Compliance Auditors
- Enterprise Implementation Partners

4. Prerequisites

Before evaluating system environment compatibility:

1. Review the platform software architecture defined in MASTER_ARCHITECTURE.md.
2. Ensure endpoint security policies permit local execution of packaged Electron applications (`.exe` , `.dmg` , `.AppImage`).
3. Verify local user write access to OS-specific application data directories (`%APPDATA%` , `~/Library/Application Support/` , `~/.config/`).

5. Warnings & Operational Hazards

> **WARNING: STORAGE PERMISSION HAZARD**: InsAcc v1.0.0 persists operational financial data locally inside browser `localStorage` (`usePersistedState.ts`). Enforcing aggressive endpoint security policies that automatically wipe browser storage or temporary user profiles on logoff will result in data loss. Enterprise IT MUST exclude InsAcc storage paths from automated cleanup utilities.

> **WARNING: To Be Implemented**: Multi-tenant server deployment components (PostgreSQL 17 database clusters, Node.js PM2 backend APIs, Nginx SSL reverse proxies) are target architecture specifications planned for enterprise release v2.0.0 and are not present in the v1.0.0 desktop binary bundle.

6. Notes & Architecture Context

> **NOTE: Zero Third-Party CDN Dependency**: InsAcc is engineered for 100% offline, air-gapped execution. All typography (Inter `.woff2` fonts), icons (inline SVG components), and stylesheets are compiled directly into the client package bundle. Workstations do NOT require active internet connectivity to run InsAcc.

7. Main Content

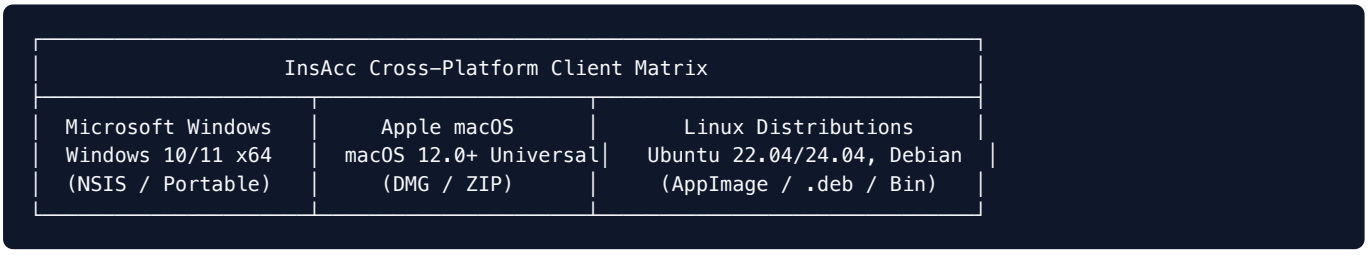
7.1 Desktop Client Hardware Requirements

The desktop client requires minimal system resources due to its React 18 virtual DOM optimizations and memoized read-model projection layer (`investmentReadModels.ts`, `reportService.ts`).

Hardware Resource	Minimum Requirement	Recommended Specification	Power User / Heavy Ledger Spec
CPU Processor	Dual-Core 2.0 GHz (x86_64 or ARM64)	Quad-Core 2.5 GHz+ (Intel i5/i7, AMD Ryzen, Apple M1+)	Octa-Core 3.0 GHz+ (Apple M-Series Pro/Max, Intel i9)
System Memory (RAM)	512 MB available	2 GB available	4 GB+ available
Disk Storage Space	200 MB free space	1 GB SSD storage	5 GB NVMe SSD (for large attachment stores)
Display Resolution	1024 x 768 pixels	1920 x 1080 (FHD 1080p)	2560 x 1440 (QHD) or 4K Dual-Monitor
Graphics Processing	Hardware-accelerated GPU or software fallback	Integrated Intel/AMD/Apple GPU	Dedicated GPU with WebGL hardware acceleration

7.2 Operating System Compatibility Matrix

InsAcc v1.0.0 desktop binaries are compiled for 64-bit operating systems via Electron Builder:



Supported Operating System Releases:

1. **Microsoft Windows**:
   - Windows 10 (64-bit, Version 20H2 or newer)

- Windows 11 (64-bit, all builds)
- Windows Server 2019 / 2022 (Citrix & Remote Desktop Services environments)

2. **Apple macOS**:
   - macOS 12.0 (Monterey), macOS 13.0 (Ventura), macOS 14.0 (Sonoma), macOS 15.0 (Sequoia+)
   - Universal Binary Architecture: Native support for Apple Silicon (M1/M2/M3/M4) and Intel x86_64
3. **Linux Distributions**:
   - Ubuntu 22.04 LTS / 24.04 LTS, Debian 11/12, RHEL 9+, Fedora 38+, Arch Linux

---

7.3 Zero-Dependency Embedded Runtimes

Client workstations **do not require** pre-installed Node.js, Python, Java, or database servers. The executable package bundles:

- Chromium 120.x rendering engine.
- Node.js 18.x runtime (isolated within the main process context).
- Bundled Inter font family (8 weights in `.woff2` format).
- HTML5 LocalStorage persistence subsystem.

7.4 Target Enterprise Server Requirements `[To Be Implemented]`

For future multi-user client-server deployments, the target server node requires the following specifications:

Server Resource	Standard Enterprise Deployment (up to 25 users)	High-Availability Cluster (25–500 users)
CPU Cores	4 vCPU / Cores (2.4 GHz+)	16 vCPU / Cores (3.0 GHz+)
System RAM	8 GB System Memory	32 GB System Memory
Storage Subsystem	100 GB Enterprise SSD (RAID-1)	500 GB NVMe SSD Array (RAID-10)
Database Engine	PostgreSQL 17 Relational Database	PostgreSQL 17 High-Availability Cluster
Reverse Proxy	Nginx with TLS 1.3 Termination	Nginx Load-Balancer Pair

---

7.5 Network & Port Allocation Specifications

InsAcc v1.0.0 client executes locally. Port allocation applies only during developer HMR mode:

Port	Protocol	Interface Scope	Usage Description
5174	TCP	`127.0.0.1` (Localhost)	Vite Development Server & HMR (Development build only)
443	TCP	Outbound HTTPS	Optional auto-updater checks `[To Be Implemented]`

Endpoint Security Exclusions

Enterprise endpoint security software (EDR / Antivirus) MUST grant read/write access to application storage paths:

- **Windows**: `%APPDATA%\InsAcc\` and `%LOCALAPPDATA%\Programs\InsAcc\`
- **macOS**: `~/Library/Application Support/InsAcc/`
- **Linux**: `~/.config/InsAcc/`

---

8. Summary

InsAcc v1.0.0 delivers an enterprise-grade asset management experience with minimal endpoint hardware requirements. By bundling Chromium and Node.js within a local-first Electron wrapper, InsAcc provides zero-dependency, air-gapped operational capability across Windows, macOS, and Linux workstations.

9. Chapter Appendix

Standard Storage Directory Paths

```
Client Workstation Storage Paths
├── Windows:  C:\Users\<Username>\AppData\Roaming\InsAcc\
├── macOS:    /Users/<Username>/Library/Application Support/InsAcc/
└── Linux:    /home/<Username>/.config/InsAcc/
```

10. Glossary

- **Air-Gapped**: A security measure that ensures a computer network is physically and logically isolated from unsecured networks, such as the public internet.
- **Electron**: An open-source framework developed by GitHub that allows for the development of desktop GUI applications using web technologies.
- **Local-First**: A software architecture paradigm where primary data storage and processing occur on the local device, prioritizing performance and offline availability.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Master Navigation Index: docs/enterprise/INDEX.md
- Desktop Deployment Guide: Volume 01 Chapter 02

title: "Volume 01: System Installation & Deployment Guide - Chapter 02: Desktop Client Deployment" document_id: "INSACC-DOC-V01-CH02" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 01: Installation & Deployment Guide

Chapter 02: Desktop Client Deployment

> **Single Source of Truth Reference**: All client packaging, installer mechanics, and operational paths defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter provides step-by-step technical procedures for deploying pre-built InsAcc desktop client binaries across Microsoft Windows, Apple macOS, and Linux workstations. It includes instructions for interactive GUI installations, unattended silent command-line deployments via Microsoft Intune/SCCM, Jamf Pro MDM deployment, and directory permission management.

2. Scope

This specification covers:

- Interactive setup wizard procedures for Windows (`InsAcc-Setup-1.0.0-x64.exe`), macOS (`InsAcc-1.0.0.dmg`), and Linux (`InsAcc-1.0.0.AppImage`).
- Silent, unattended enterprise command switches (`/S` , `/allusers` , `/D=`).
- macOS Gatekeeper, Quarantine attribute management, and codesign verification.
- Local execution directories, user configuration storage paths, and log output directories.

Out of Scope:

- Source code compilation (covered in Volume 01 Chapter 03).
- Post-installation automated test suite execution (covered in Volume 01 Chapter 04).

3. Audience

This document is authored for:

- Systems Administrators and Desktop Support Personnel
- Endpoint Management Engineers (SCCM, Intune, Jamf, Kandji)
- Enterprise Software Deployment Technicians
- IT Security Operations Teams

4. Prerequisites

Before initiating client deployment:

1. Verify endpoint workstation compatibility against Volume 01 Chapter 01.
2. Obtain officially compiled binary distribution packages from the release repository (`release/` directory).
3. Ensure administrative privileges if performing per-machine (`/allusers`) deployments on Windows or package installations on Linux.

5. Warnings & Operational Hazards

> **WARNING: WINDOWS PER-MACHINE INSTALLATION PATH**: When performing an unattended per-machine installation (`/allusers`), the installer target parameter `/D=` MUST be the last command-line parameter. Placing additional arguments after `/D=` will cause the target path string to corrupt.

> **WARNING: MACOS QUARANTINE ATTRIBUTE**: When distributing unpacked `.app` bundles manually via terminal or network shares without Apple Notarization `[To Be Implemented]`, macOS Gatekeeper may flag the app with the `com.apple.quarantine` attribute. System administrators must clear this attribute prior to user execution.

6. Notes & Architecture Context

> **NOTE: User-Space Zero-Privilege Deployment**: On Windows, the default NSIS installer targets per-user installation (`%LOCALAPPDATA%\Programs\InsAcc\`). This permits users to install and update InsAcc without requiring Local Administrator rights or UAC elevation.

7. Main Content

7.1 Microsoft Windows Installation & Unattended Silent Rollout

Pre-compiled Windows binaries are generated using Electron Builder and packaged into the `release/` directory.

Method A: Interactive NSIS Setup Wizard

1. Locate the setup executable: `InsAcc-Setup-1.0.0-x64.exe`.
2. Double-click the file to initiate the NSIS Setup Wizard.
3. Select installation scope:
   - **Only for me** (Default): Installs to `%LOCALAPPDATA%\Programs\InsAcc\`. Requires no UAC elevation.
   - **Anyone who uses this computer**: Installs to `C:\Program Files\InsAcc\`. Requires UAC administrative approval.
4. Click **Install**. The setup copies application binaries, creates Start Menu shortcuts, and registers uninstall metadata in `Control Panel -> Programs and Features`.
5. Click **Finish** to launch InsAcc.

Method B: Unattended Silent Command-Line Deployment (SCCM / Intune)

To automate enterprise distribution without user intervention:

```
:: Command 1: Silent per-user deployment (No admin rights required)
InsAcc-Setup-1.0.0-x64.exe /S

:: Command 2: Silent per-machine deployment for all users (Administrative Command Prompt)
InsAcc-Setup-1.0.0-x64.exe /S /allusers

:: Command 3: Silent per-machine deployment to custom target path (Note: /D= must be last)
InsAcc-Setup-1.0.0-x64.exe /S /allusers /D=C:\EnterpriseApps\InsAcc
```

Switch / Parameter	Functional Description
`/S`	Enables silent mode (suppresses all GUI dialogs, progress bars, and prompts).
`/allusers`	Instructs installer to install for all machine users into `Program Files`.
`/D=<path>`	Overrides default installation directory. Must be the final command switch.

7.2 Apple macOS Deployment & MDM Management

InsAcc for macOS is built as a Universal Binary supporting both Apple Silicon (M1/M2/M3/M4) and Intel processors.

Method A: Interactive Drag-and-Drop Installation

1. Mount the Disk Image archive: `InsAcc-1.0.0.dmg`.
2. A Finder window presents the `InsAcc.app` bundle and a shortcut to the `/Applications` directory.
3. Drag `InsAcc.app` into the `Applications` folder.
4. Eject the DMG image.
5. Launch InsAcc from Launchpad or via Spotlight (`Cmd + Space` $\rightarrow$ type `InsAcc`).

Method B: Enterprise MDM Rollout & Security Clearance

For automated deployment via Jamf Pro, Kandji, or deployment scripts:

```
# Verify application signature status
codesign --verify --deep --verbose /Applications/InsAcc.app

# Remove Gatekeeper quarantine attribute if pushed via custom script
sudo xattr -rd com.apple.quarantine /Applications/InsAcc.app
```

7.3 Linux AppImage & Debian Package Installation

Method A: Universal AppImage Deployment

1. Make the AppImage executable:

   ```
   chmod +x InsAcc-1.0.0.AppImage
   ```

2. Launch the standalone application:

   ```
   ./InsAcc-1.0.0.AppImage
   ```

Method B: Debian / Ubuntu Package Installation (`.deb`)

1. Install via `dpkg`:

   ```
   sudo dpkg -i insacc_1.0.0_amd64.deb
   sudo apt-get install -f # Resolve any missing system dependencies
   ```

7.4 Directory Hierarchy & Execution Paths Across OS Platforms

Operating System	Binary Executable Target Path	Local Storage & Application Data Path	Application Log Output Directory
Microsoft Windows	`%LOCALAPPDATA%\Programs\InsAcc\InsAcc.exe`	`%APPDATA%\InsAcc\`	`%APPDATA%\InsAcc\logs\`
Apple macOS	`/Applications/InsAcc.app/`	`~/Library/Application Support/InsAcc/`	`~/Library/Logs/InsAcc/`
Linux Distribution	`/opt/InsAcc/insacc` or AppImage dir	`~/.config/InsAcc/`	`~/.config/InsAcc/logs/`

7.5 Client Upgrades & Distribution Artifact Inspection

When upgrading InsAcc to a newer version:

1. Running the new installer automatically overwrites binary executables while **preserving user local storage data** (`localStorage` keys under `insacc_*`).
2. Schema version migrations are handled automatically on first launch via `insacc_clear_version` (`CLEAR_VERSION = '8'`).

---

8. Summary

InsAcc client binaries provide flexible deployment options across Windows, macOS, and Linux endpoints. With support for user-space installations, unattended silent command switches, and universal macOS binaries, enterprise IT teams can seamlessly integrate InsAcc into automated endpoint deployment systems.

---

9. Chapter Appendix

Unattended Uninstall Command Reference (Windows)

To silently remove InsAcc from a Windows workstation:

```
:: Execute silent uninstaller from per-user installation path
"%LOCALAPPDATA%\Programs\InsAcc\Uninstall InsAcc.exe" /S

:: Execute silent uninstaller from per-machine path
"C:\Program Files\InsAcc\Uninstall InsAcc.exe" /S /allusers
```

---

10. Glossary

- **AppImage**: A universal software format for distributing portable software on Linux without needing superuser permissions to install.
- **Jamf Pro**: An enterprise management software solution used by IT administrators to automate Apple device deployment and compliance.
- **NSIS**: Nullsoft Scriptable Install System, a script-driven Windows installer authoring tool used by Electron Builder.

---

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- System Requirements: Volume 01 Chapter 01
- Build From Source: Volume 01 Chapter 03
- Deployment Verification: Volume 01 Chapter 04

title: "Volume 01: System Installation & Deployment Guide - Chapter 03: Build From Source" document_id: "INSACC-DOC-V01-CH03" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 01: Installation & Deployment Guide

Chapter 03: Build From Source

> **Single Source of Truth Reference**: All compilation toolchains, package scripts, and build outputs defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter provides the technical specification and step-by-step developer guide for compiling, auditing, and packaging the InsAcc desktop client application from raw source code across Microsoft Windows, Apple macOS, and Linux build environments.

2. Scope

This specification covers:

- Developer build environment prerequisites (Node.js 22 LTS, npm 10+, C++ build tools).
- Deterministic dependency installation using lockfile verification (`npm ci`).
- Concurrent development environment execution (`npm run dev` with HMR on port 5174).
- TypeScript static analysis validation (`npx tsc --noEmit`).
- Two-stage production build pipeline (Vite 5 bundling $\rightarrow$ Electron Builder 23 packaging).

Out of Scope:

- End-user binary installation (covered in Volume 01 Chapter 02).
- Post-build integration testing (covered in Volume 01 Chapter 04).

3. Audience

This document is authored for:

- Enterprise Software Engineers and Core Maintainers
- Quality Assurance Automation Engineers
- DevOps Pipeline Engineers
- Independent Security Code Auditors

4. Prerequisites

Before setting up the build environment:

1. Review the application directory structure in MASTER_ARCHITECTURE.md#11-folder-structure.
2. Install Git version control (Version 2.40 or newer).
3. Install Node.js 22 LTS (Active Long Term Support release) and npm 10+.

5. Warnings & Operational Hazards

> **WARNING: DETERMINISTIC DEPENDENCY MANDATE**: Enterprise builds MUST use `npm ci` rather than `npm install`. Using `npm install` can resolve unvetted transitive dependency updates, breaking build reproducibility and violating security audit baselines.

> **WARNING: NATIVE NODE-GYP BUILD TOOLS**: Building native C/C++ dependencies on Windows requires Microsoft Visual C++ Build Tools. On macOS, Xcode Command Line Tools are mandatory. Ensure native compiler tools are

> installed before running package scripts.

6. Notes & Architecture Context

> **NOTE: Port Allocation in Dev Mode**: During `npm run dev`, Vite launches an HMR server bound to `http://localhost:5174`. The Electron main process (`src/main/main.js`) polls this URL via `wait-on` before opening the desktop window. Ensure port 5174 is unblocked by local firewalls.

7. Main Content

7.1 Compiler & Toolchain Setup

To compile InsAcc from source, verify the local developer environment meets toolchain requirements:

Tool / Runtime	Version Requirement	Purpose	Command Verification
Git	2.40.0+	Version control & source retrieval	`git --version`
Node.js	22.x LTS (22.0.0+)	JavaScript runtime environment	`node -v`
npm	10.x+	Package manager	`npm -v`
Python	3.10+	Required by `node-gyp` for native modules	`python3 --version`
C/C++ Compiler	MSVC 2022 (Win) / Xcode (Mac) / GCC (Linux)	Native dependency compilation	`gcc --version` / `clang --version`

7.2 Repository Cloning & Lockfile Installation (`npm ci`)

1. Clone the repository:

```
git clone https://github.com/truvx/InsAcc.git
cd InsAcc
```

2. Inspect `package.json` to verify core build scripts and dependencies:
   - React 18.3.1 & TypeScript 5.3.3
   - Vite 5.x bundler plugin
   - Electron 28.x and Electron Builder 23.6.0
3. Install dependencies deterministically:

```
npm ci
```

7.3 Interactive Development Execution (`npm run dev`)

To launch InsAcc in interactive development mode with Hot Module Replacement:

```
npm run dev
```

Execution Mechanics:

```
npm run dev
   |
   ├──► 1. Launches `vite` server on http://localhost:5174
   |
   ├──► 2. Runs `wait-on http://localhost:5174`
   |
   └──► 3. Spawns Electron main process (`src/main/main.js`) loading port 5174
```

7.4 Static Typecheck Validation (`npx tsc --noEmit`)

InsAcc enforces strict TypeScript compilation rules (`strict: true` in `tsconfig.json`). Before initiating production packaging, run the static analysis validator:

```
npx tsc --noEmit
```

Expected Terminal Output:

```
--- No compilation errors ---
```

7.5 Two-Stage Production Packaging Pipeline (Vite + Electron Builder)

Production distribution packages are generated via a two-stage build process:

Stage 1: Front-End Compilation (Vite)

Run the Vite compiler script:

```
npm run build
```

This minifies static assets into `dist/` (`dist/index.html` , vendor-split JS chunks, compiled CSS tokens).

Stage 2: Desktop Package Generation (Electron Builder)

Execute the build command matching your target operating system:

```
# Package for Microsoft Windows (x64 NSIS + Portable Executable)
npm run build:win

# Package for Apple macOS (Universal DMG + ZIP)
npm run build:mac

# Package for Linux (x64 AppImage + .deb Package)
npm run build:linux
```

Output distribution binaries are written directly to `release/` :

- `release/InsAcc-Setup-1.0.0-x64.exe`
- `release/InsAcc-1.0.0.dmg`

- `release/InsAcc-1.0.0-mac.zip`
- `release/InsAcc-1.0.0.AppImage`

8. Summary

Compiling InsAcc from source is a deterministic process powered by Node.js 22 LTS, Vite 5, and Electron Builder. By following the two-stage compilation pipeline, developers can generate fully signed, minified distribution binaries across Windows, macOS, and Linux platforms.

9. Chapter Appendix

Package Script Command Reference Matrix

npm Script Command	Underling Shell Execution	Operational Output
`npm run dev`	`concurrently "vite" "wait-on http://localhost:5174 && electron ."`	Interactive dev app with HMR
`npm run build`	`tsc && vite build`	Minified static files in `dist/`
`npm run build:win`	`npm run build && electron-builder --win`	Windows NSIS installer in `release/`
`npm run build:mac`	`npm run build && electron-builder --mac`	macOS DMG & ZIP in `release/`
`npm run build:linux`	`npm run build && electron-builder --linux`	Linux AppImage & `.deb` in `release/`

10. Glossary

- **HMR (Hot Module Replacement)**: A feature in bundlers like Vite that exchanges, adds, or removes modules while an application is running without a full reload.
- **Lockfile**: A file (`package-lock.json`) that records the exact versions of all dependencies installed in a project to ensure reproducible builds.
- **Vite**: A modern front-end build tool that provides a fast development server and bundles assets for production using Rollup.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Client Deployment Guide: Volume 01 Chapter 02
- Deployment Verification: Volume 01 Chapter 04

title: "Volume 01: System Installation & Deployment Guide - Chapter 04: Deployment Verification" document_id: "INSACC-DOC-V01-CH04" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 01: Installation & Deployment Guide

Chapter 04: Deployment Verification

> **Single Source of Truth Reference**: All verification criteria, automated test suites, and diagnostic matrices defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter defines post-deployment quality verification procedures for the InsAcc desktop application. It details automated end-to-end (E2E) integration test execution using Playwright, pre-production manual acceptance criteria, and diagnostic troubleshooting runbooks for deployment anomalies.

---

2. Scope

This specification covers:

- Execution and validation of the Playwright 76-test integration suite across 3 test modules.
- Pre-production manual acceptance verification items (`VER-01` through `VER-06`).
- Network interface and IPC bridge verification.
- Diagnostic troubleshooting runbooks for common installation issues (blank screens, IPC errors, permission issues).

Out of Scope:

- General Ledger double-entry engine rule validation (covered in Volume 06 Chapter 02).
- Disaster recovery data restoration (covered in Volume 07 Chapter 02).

---

3. Audience

This document is authored for:

- Quality Assurance Engineers & Test Automation Leads
- Systems Administrators and Deployment Technicians
- Enterprise Security & Compliance Inspectors
- Implementation Support Personnel

---

4. Prerequisites

Before executing deployment verification:

1. Verify successful completion of client installation (Volume 01 Chapter 02) or source compilation (Volume 01 Chapter 03).
2. Ensure Playwright test runner dependencies are installed (`npm ci`).

---

5. Warnings & Operational Hazards

> **WARNING: VERIFICATION DATA ISOLATION**: Running Playwright automated test suites against a production client profile will modify local test data. Test automation MUST be executed in isolated test instances or development environments to prevent overwriting operational ledgers.

---

6. Notes & Architecture Context

> **NOTE: Playwright Multi-Viewport Coverage**: Visual QA tests in `tests/reports-visual.spec.ts` execute across 4 distinct viewport configurations (1920x1080, 1440x900, 1024x768, 768x1024) in both Light and Dark interface modes to guarantee responsive design integrity.

7. Main Content

7.1 Automated Integration Testing (Playwright 76-Test Pipeline)

InsAcc includes a automated Playwright test suite containing **76 end-to-end integration tests**:

Test Suite Name	Test Count	Scope & Functional Coverage	Pipeline Status
Purchase Ledger UI	14	Purchase CRUD, KPI accuracy, filtering, sorting, numeric validation, modal forms	✓ Pass (14/14)
Reports Visual QA	14	Financial statement rendering across 4 viewports in Light/Dark modes	✓ Pass (14/14)
Transactions Final QA	48	Transaction entry (Income/Expense/Journal), category mapping, filters, sorting, persistence	✓ Pass (48/48)
Total Test Pipeline	**76**	**Complete Application Verification**	**✓ All Pass (76/76)**

Test Execution Commands:

```
# Run all Playwright integration suites headless
npx playwright test

# Execute a specific test file
npx playwright test tests/transactions.spec.ts

# Launch Playwright UI mode for interactive debugging
npx playwright test --ui
```

7.2 Pre-Production Manual Acceptance Verification Matrix

Execute the manual verification matrix before declaring an installation ready for operational deployment:

Item ID	Category	Verification Procedure	Expected Outcome	Result
VER-01	Binary Execution	Launch InsAcc executable from desktop shortcut or Applications menu	Opens centered window (1400x900) displaying Login screen	[ ] Pass
VER-02	Profile Selection	Select `Investor` profile and click `Proceed`	Navigates to Module Selection screen (`Investment` / `Property`)	[ ] Pass
VER-03	LocalStorage Persistence	Change theme to `Dark Mode` in Settings -> restart application	Dark mode theme persists across application restarts (`insacc_clear_version` intact)	[ ] Pass
VER-04	IPC File Export	Open `Reports` -> click `Export CSV` on any report view	File dialog writes formatted CSV file to user Downloads folder via `window.api.saveFile`	[ ] Pass
VER-05	Double-Entry Engine	Record a Receipt Voucher (`RV`) of AED 10,000	Voucher posts, `1120` Bank balance increases by 10,000, Trial Balance balances ($\sum D = \sum C$)	[ ] Pass
VER-06	PDC Manager	Record PDC cheque -> transition status `Received` $\rightarrow$ `Deposited` $\rightarrow$ `Cleared`	Cheque status transitions, unearned rent converts to rental revenue, ledger reflects entries	[ ] Pass

7.3 Network & Local Interface Verification Matrix

Local Workstation Storage Paths:

- **Windows**: `%APPDATA%\InsAcc\`
- **macOS**: `~/Library/Application Support/InsAcc/`
- **Linux**: `~/.config/InsAcc/`

Port Allocation:

- **Port 5174 (TCP)**: Localhost `127.0.0.1` — Used only by Vite dev server during development.

7.4 Installation & Startup Diagnostic Troubleshooting

Issue 1: Application Displays Blank / White Screen on Startup

- **Root Cause**: Stale or incompatible schema version stored in `localStorage`.
- **Resolution**: InsAcc automatically checks `insacc_clear_version` (`CLEAR_VERSION = '8'`). If corrupted, open DevTools (`Ctrl+Shift+I` / `Cmd+Option+I`) $\rightarrow$ Console $\rightarrow$ type `localStorage.clear()` $\rightarrow$ press `F5` to reload.

Issue 2: `window.api.saveFile is not a function` Error

- **Root Cause**: Preload script failed to attach to the renderer context bridge.
- **Resolution**: Ensure `contextIsolation: true` and `nodeIntegration: false` are set in `main.js`. Re-run `npm run build` to re-compile `preload.js`.

Issue 3: Permission Denied Writing Output Files

- **Root Cause**: OS folder security blocking write access to Downloads directory.
- **Resolution**: Grant standard user profile write permissions to the Downloads folder.

8. Summary

Deployment verification ensures that the InsAcc desktop client is fully functional, visually compliant across viewports, and mathematically accurate in double-entry accounting calculations. By combining 76 automated Playwright tests with a 6-point manual acceptance matrix, enterprise deployments achieve zero-defect operational readiness.

9. Chapter Appendix

Playwright Viewport Matrix Reference

```
Playwright Test Viewport Layout Matrix
├── Desktop Full:       1920 x 1080 (FHD 1080p Viewport)
├── Desktop Standard:   1440 x 900  (Standard Laptop Viewport)
├── Tablet Landscape:   1024 x 768  (Tablet Horizontal)
└── Tablet Portrait:     768 x 1024 (Tablet Vertical - Collapsed Sidebar)
```

10. Glossary

- **Acceptance Criteria**: The explicit conditions that a software product must satisfy to be accepted by a user, customer, or system administrator.
- **End-to-End (E2E) Testing**: A software testing methodology that tests an application flow from start to finish to ensure all components work as expected.
- **Playwright**: An open-source web testing and automation framework developed by Microsoft.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- System Requirements: Volume 01 Chapter 01
- Desktop Deployment Guide: Volume 01 Chapter 02
- Playwright Test Framework Specs: Volume 06 Chapter 05

title: "Volume 02: Data Management & Persistence Guide - Chapter 01: LocalStorage Persistence Architecture" document_id: "INSACC-DOC-V02-CH01" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 02: Data Management & Persistence Guide

Chapter 01: LocalStorage Persistence Architecture

> **Single Source of Truth Reference**: All persistence storage rules, data key schemas, and balance computation algorithms defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter defines the technical architecture of the local-first storage subsystem in InsAcc v1.0.0. It details the custom `usePersistedState` React hook, the exhaustive 16-key `localStorage` dictionary, entity identification standards, and the inviolable **Derived Balance Golden Rule**.

2. Scope

This specification covers:

- Client-side state persistence via browser `localStorage` in Electron.
- The source implementation and error boundary mechanics of `usePersistedState.ts`.
- Complete schema specifications for all 16 `insacc_*` storage keys.
- Immutable string primary key rules for domain entities.
- Dynamic balance calculation algorithms (`ledgerService.ts`).

Out of Scope:

- Schema versioning and startup resets (covered in Volume 02 Chapter 02).
- Relational PostgreSQL database migration `[To Be Implemented]` (covered in Volume 02 Chapter 03).

3. Audience

This document is authored for:

- Core Software Engineers and Frontend Developers
- Database Administrators and Data Architects
- Quality Assurance Automated Test Engineers
- Enterprise Technical Auditors

4. Prerequisites

Before evaluating persistence architecture:

1. Review the software architecture specified in MASTER_ARCHITECTURE.md#2-software-architecture.
2. Understand React 18 hook lifecycles (`useState`, `useEffect`) and synchronous browser storage behaviors.

5. Warnings & Operational Hazards

> **WARNING: STORAGE QUOTA LIMITATIONS**: Standard browser `localStorage` implementations enforce a storage quota limit (typically 5 MB to 10 MB per domain). In InsAcc v1.0.0, storing exceptionally large Base64 document attachments in `insacc_documents` can approach quota thresholds. Large document attachments MUST be managed efficiently.

> **WARNING: SCALAR BALANCE PERSISTENCE PROHIBITION**: Manually writing editable balance scalars into `localStorage` creates data synchronization drift. Account balances MUST NEVER be stored as independent, editable

scalar values. Balances are derived dynamically on read.

6. Notes & Architecture Context

> **NOTE: Zero Network Latency**: Because `localStorage` reads and writes execute synchronously within the local client process memory space, InsAcc achieves sub-millisecond data retrieval times for financial dashboards and report calculations.

7. Main Content

7.1 Local-First Persistence Rationale

InsAcc v1.0.0 employs a local-first application architecture. Storing domain collections inside browser `localStorage` guarantees:

1. **100% Offline Autonomy**: Full platform operational capability without internet connectivity or external database server dependencies.
2. **Instant Warm Starts**: Zero startup latency connecting to remote database sockets.
3. **Complete Privacy**: Financial ledger data remains entirely within the user's local machine environment.

7.2 The `usePersistedState` Hook Implementation

State persistence is decoupled from UI component logic via the custom hook `src/renderer/usePersistedState.ts`:

```
import { useState, useEffect } from 'react'

export function usePersistedState<T>(key: string, defaultValue: T): [T, (value: T | ((val: T) => T)) => void] {
  // 1. Lazy Initializer Function
  const [state, setState] = useState<T>(() => {
    try {
      const item = localStorage.getItem(key)
      return item ? JSON.parse(item) : defaultValue
    } catch (error) {
      console.error(`Error reading localStorage key "${key}":`, error)
      return defaultValue
    }
  })

  // 2. Synchronous Serialization Effect
  useEffect(() => {
    try {
      localStorage.setItem(key, JSON.stringify(state))
    } catch (error) {
      console.error(`Error setting localStorage key "${key}":`, error)
    }
  }, [key, state])

  return [state, setState]
}
```

Core Operational Phases:

1. **Lazy Initialization**: On initial component mount, `localStorage.getItem(key)` is read and parsed via `JSON.parse()`. If null or corrupted, `defaultValue` is returned.

2. **Synchronous Commitment**: Whenever `state` mutates, `useEffect` serializes state to JSON via `JSON.stringify()` and commits it to `localStorage`.
3. **Error Isolation**: Storage quota or parsing exceptions are caught safely without crashing the React component tree.

7.3 Master Storage Key Dictionary (16 Storage Keys)

InsAcc partitions operational data across **16 storage keys** namespace-prefixed with `insacc_`:

Key Name	TypeScript Model Interface	Core Description	Initial Seed State
`insacc_investments`	`Investment[]`	Portfolio holding positions & valuations	Pre-seeded sample portfolio
`insacc_transactions`	`Transaction[]`	General income, expense & journal entries	Pre-seeded transaction list
`insacc_statement`	`StatementEntry[]`	Bank statement lines (Legacy model)	Pre-seeded statement lines
`insacc_balance`	`number`	Bank balance scalar (Deprecated)	Initial seed balance
`insacc_documents`	`DocItem[]`	Document metadata & Base64 attachments	Sample document list
`insacc_logs`	`LogEntry[]`	Operational system activity logs	Initial system startup log
`insacc_purchase_categories`	`PurchaseCategory[]`	Purchase ledger asset categories	Default asset taxonomy
`insacc_purchases`	`Purchase[]`	Purchase ledger asset lot transactions	Pre-seeded purchase lots
`insacc_inv_users`	`UserEntry[]`	Investment module user profiles	Admin & Accounts users
`insacc_prop_users`	`UserEntry[]`	Property module user profiles	Admin & Accounts users
`insacc_prop_categories`	`PropertyCategory[]`	Property building categories	Building / Villa / Office
`insacc_prop_buildings`	`PropertyBuilding[]`	Building master records	Pre-seeded building list
`insacc_prop_units`	`PropertyUnit[]`	Rentable units per building	Pre-seeded unit records
`insacc_prop_tenants`	`PropertyTenant[]`	Tenant master profiles & lease terms	Pre-seeded tenant list
`insacc_prop_rent`	`RentPayment[]`	Rent collection payment records	Pre-seeded rent collections
`insacc_clear_version`	`string`	Schema version tracker	Current: `"8"` (`CLEAR_VERSION`)

7.4 Immutable Entity Identification Standard

Every domain entity persisted in `localStorage` MUST contain an **immutable primary key identifier (`id`)**.

Mandatory Primary Key Rules:

- All IDs are non-empty strings (e.g., `inv-1719500000000`, `P-1719500000000-1`).
- IDs are generated once at entity creation using UUID v4 or deterministic timestamp prefixes.
- IDs act as primary keys and foreign key references (`accountId`, `tenantId`, `unitId`, `propertyId`).
- An entity's `id` **MUST NEVER be modified, re-generated, or re-assigned**.

7.5 The Derived Balance Golden Rule

> **IMPORTANT: THE DERIVED BALANCE GOLDEN RULE**: Account balances, bank cash positions, and portfolio valuations are **ALWAYS COMPUTED DYNAMICALLY ON READ**.
> $$\text{Current Balance} = \text{Opening Balance} + \sum \text{Posted Debits} - \sum \text{Posted Credits}$$
> Manually modifying or storing scalar balance values is strictly prohibited. To alter a balance, a user MUST record a valid accounting voucher or bank transaction.

Implementation in `ledgerService.ts`:

```
export function deriveAccountBalance(accountId: string, vouchers: Voucher[], openingBalance: number = 0): numb
  const postedVouchers = vouchers.filter(v => v.status === 'Posted')
  let totalDebit = 0
  let totalCredit = 0

  for (const voucher of postedVouchers) {
    for (const line of voucher.lines) {
      if (line.accountId === accountId) {
        if (line.type === 'Debit') totalDebit += line.amount
        if (line.type === 'Credit') totalCredit += line.amount
      }
    }
  }

  return openingBalance + totalDebit - totalCredit
}
```

---

8. Summary

InsAcc v1.0.0 delivers reliable, zero-latency local data management by combining React state hooks with browser `localStorage`. By enforcing immutable entity IDs, a 16-key namespace dictionary, and the Derived Balance Golden Rule, InsAcc maintains double-entry financial data integrity across user operations.

---

9. Chapter Appendix

Key Memory Quota Reference Matrix

Storage Key	Estimated Size (1,000 Records)	Quota Risk	Mitigation Strategy
`insacc_investments`	~150 KB	Low	Pure JSON serialization
`insacc_transactions`	~250 KB	Low	Pure JSON serialization
`insacc_prop_leases`	~300 KB	Low	Pure JSON serialization
`insacc_documents`	~3 MB to 8 MB	Medium	Exclude large Base64 files; store in `userData` dir

10. Glossary

- **Derived Balance**: A financial value that is not stored directly in a database field but is dynamically calculated by summing historical debit and credit transactions.
- **Lazy Initialization**: An optimization technique where state evaluation or file reads are deferred until the moment they are first required.
- **LocalStorage**: An HTML5 web storage mechanism that allows JavaScript applications to store key-value pairs persistently in the web browser.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Schema Versioning & Migrations: Volume 02 Chapter 02
- Target Database Migration: Volume 02 Chapter 03
- Key Dictionary Reference: Volume 09 Appendix C

title: "Volume 02: Data Management & Persistence Guide - Chapter 02: Schema Versioning and Migrations" document_id: "INSACC-DOC-V02-CH02" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 02: Data Management & Persistence Guide

Chapter 02: Schema Versioning and Migrations

> **Single Source of Truth Reference**: All schema version constants, startup migration pipelines, and versioning rules defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter defines the technical architecture governing schema versioning, startup migration verification, and dataset initialization in InsAcc v1.0.0. It details the global version constant (`CLEAR_VERSION = '8'`), version check pipelines, data sanitization rules, and in-memory domain migration services.

2. Scope

This specification covers:

- The global schema version tracker (`insacc_clear_version`).
- Application launch version validation in `App.tsx` .
- Automated storage key sanitization logic on schema mismatch.
- Domain migration services (`initializationService.ts` , `bankTransactionService.ts` , `purchaseAccountingService.ts`).
- Administrative factory data reset procedures.

Out of Scope:

- Core `localStorage` key specifications (covered in Volume 02 Chapter 01).
- Target PostgreSQL relational database DDL `[To Be Implemented]` (covered in Volume 02 Chapter 03).

3. Audience

This document is authored for:

- Frontend Software Engineers and Core Maintainers
- Database Administrators & Data Governance Officers
- Quality Assurance Automation Engineers
- Systems Support Technicians

4. Prerequisites

Before evaluating schema versioning:

1. Review the storage key dictionary in Volume 02 Chapter 01.
2. Understand application initialization hooks in `src/renderer/App.tsx` .

5. Warnings & Operational Hazards

> **WARNING: AUTOMATED DATA RESET ON VERSION MISMATCH**: Incrementing `CLEAR_VERSION` in `App.tsx` (e.g., from `'8'` to `'9'`) causes the startup pipeline to clear all existing `insacc_*` keys in `localStorage` and reload factory seed datasets. Developers MUST NOT increment `CLEAR_VERSION` without providing non-destructive migration scripts for operational users.

6. Notes & Architecture Context

> **NOTE: Version Isolation**: The version check isolates InsAcc application keys by enforcing the `insacc_` prefix check (`key.startsWith('insacc_')`). Other browser storage keys unrelated to InsAcc remain untouched.

7. Main Content

7.1 Schema Version Governance (`CLEAR_VERSION = '8'`)

Schema versioning is controlled by a global constant in `src/renderer/App.tsx`:

```
const CLEAR_VERSION = '8'
```

Version Tracker Specifications:

- **Storage Key**: `insacc_clear_version`
- **Active Version Identifier**: `"8"` (InsAcc v1.0.0 Enterprise Release)
- **Role**: Validates that data persisted in browser `localStorage` matches the active code schema version.

7.2 Application Launch Version Check & Storage Sanitization

During application startup (`App.tsx` initialization `useEffect`), InsAcc executes a version verification check:

```
Application Launch
        |
        ▼
Read `insacc_clear_version` from localStorage
        |
        ├──────── Equals CLEAR_VERSION ('8')? ──────► [PASS] Load state normally
        |
        └──────── Mismatch or Missing? ──────────────► [TRIGGER RESET PIPELINE]
                                                       1. Filter keys matching `insacc_*`
                                                       2. Purge stale storage keys
                                                       3. Set `insacc_clear_version` = '8'
                                                       4. Seed default domain datasets
                                                       5. Reload application window
```

Implementation in `src/renderer/App.tsx`:

```
useEffect(() => {
  const currentVersion = localStorage.getItem('insacc_clear_version')
  if (currentVersion !== CLEAR_VERSION) {
    console.log(`Schema version mismatch detected (found "${currentVersion}", expected "${CLEAR_VERSION}"). Re

    // Purge legacy insacc keys
    Object.keys(localStorage).forEach(key => {
      if (key.startsWith('insacc_')) {
        localStorage.removeItem(key)
      }
    })

    // Store active version string
    localStorage.setItem('insacc_clear_version', CLEAR_VERSION)
    window.location.reload()
  }
}, [])
```

7.3 Non-Destructive In-Memory Migration Services

For incremental updates that do not require purging storage, InsAcc uses domain migration services:

1. **`initializationService.ts`**: Checks domain collections on mount. If pre-seeded buildings, categories, or default accounts are missing, it injects defaults into state without overwriting existing user data.
2. **`bankTransactionService.ts`**: Converts formatted string values (e.g., `"+AED 5,000"`) into numeric floats (`5000`) during dataset upgrades.
3. **`purchaseAccountingService.ts`**: Maps purchase ledger entries to corresponding double-entry accounting vouchers.

7.4 Administrative System Data Reset Workflow

System administrators with `Admin` role permissions can manually invoke factory resets via **Settings** $\rightarrow$ **Data Management**:

1. Navigate to **Settings** $\rightarrow$ **Reset Data**.
2. Click **Reset System Data**.
3. Confirm the modal prompt (`"This action will permanently wipe all local data and restore factory seed datasets."`).
4. The system clears all `insacc_*` keys, sets `insacc_clear_version = '8'`, and reloads the window.

---

8. Summary

InsAcc guarantees client schema integrity through a dual versioning strategy: automatic version checking (`CLEAR_VERSION = '8'`) on launch to handle breaking schema changes, combined with non-destructive in-memory migration services (`initializationService.ts`) for incremental updates.

---

9. Chapter Appendix

Version History Table

Version Identifier	Release Date	Target App Version	Core Schema Changes
`"6"`	2026-05-15	v0.8.0-beta	Initial double-entry voucher schema
`"7"`	2026-06-20	v0.9.5-rc	Added property hierarchy and PDC manager keys
`"8"`	2026-07-22	v1.0.0	Consolidated 16-key dictionary and derived balance rule

10. Glossary

- **Factory Reset**: The process of wiping user modifications and restoring a software system to its initial out-of-the-box state.
- **Migration**: The process of transforming data structures from an older schema format to a newer format without losing data integrity.
- **Sanitization**: Cleaning or removing invalid, stale, or corrupted entries from a storage database.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- LocalStorage Persistence Architecture: Volume 02 Chapter 01
- Target Database Migration: Volume 02 Chapter 03

title: "Volume 02: Data Management & Persistence Guide - Chapter 03: Target Database Migration Plan [To Be Implemented]" document_id: "INSACC-DOC-V02-CH03" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Target Architecture Specification" classification: "Commercial Enterprise Documentation"

Volume 02: Data Management & Persistence Guide

Chapter 03: Target Database Migration Plan `[To Be Implemented]`

> **Single Source of Truth Reference**: All database target schemas, DDL specifications, and migration scripts defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter defines the target relational database architecture and migration execution plan for transitioning InsAcc from single-machine browser `localStorage` persistence to an enterprise multi-tenant PostgreSQL 17 server database cluster `[To Be Implemented]`.

2. Scope

This specification covers:

- The 5-schema PostgreSQL 17 relational database architecture (`accounting`, `investment`, `property`, `auth`, `audit`).
- Production DDL specifications for accounts, vouchers, voucher lines, units, leases, and PDC cheques.
- JSON state snapshot extraction (`state_backup_YYYY-MM-DD.json`) and ETL ingestion.
- Post-migration data integrity verification ($\sum D = \sum C$).
- Automated database initialization shell script (`scripts/postgresql/init-database.sh`).

Out of Scope:

- Current desktop `localStorage` hook implementation (covered in Volume 02 Chapter 01).
- REST API endpoint specifications (covered in Volume 05 Chapter 03).

3. Audience

This document is authored for:

- Database Administrators (DBAs) and Data Engineers
- Enterprise Systems Architects
- DevOps & Migration Engineering Teams
- Security & Compliance Auditors

4. Prerequisites

Before planning server migration:

1. Review the database standards specified in MASTER_ARCHITECTURE.md#7-postgresql-architecture.
2. Provision a PostgreSQL 17 server node with memory tuning configured as specified in `docs/enterprise/configs/postgresql/postgresql.conf`.

5. Warnings & Operational Hazards

> **WARNING: To Be Implemented**: The PostgreSQL 17 server schema, DDL scripts, and automated ETL migration pipeline described in this chapter are target architecture specifications planned for enterprise release v2.0.0. InsAcc v1.0.0 operates as an offline desktop application using `localStorage`.

6. Notes & Architecture Context

> **NOTE: Foreign Key UUID Resolution**: During migration, client-side string IDs (e.g., `ba-1719500000000`, `P-1719500000000-1`) are mapped to PostgreSQL `UUID` primary keys (`gen_random_uuid()`) while preserving foreign key relationships across accounts, vouchers, properties, and tenants.

7. Main Content

7.1 Target Enterprise Database Architecture Overview `[To Be Implemented]`

The target database (`insacc_enterprise_db`) partitions domain data across 5 PostgreSQL schemas:

```
PostgreSQL Database: insacc_enterprise_db
├── Schema: accounting (accounts, vouchers, voucher_lines, posting_rules, periods)
├── Schema: investment (investments, purchase_records, holdings)
├── Schema: property   (categories, buildings, units, tenants, leases, rent_payments, pdc_cheques)
├── Schema: auth       (users, user_permissions)
└── Schema: audit      (events, voucher_changelog)
```

7.2 Schema & Table DDL Specifications `[To Be Implemented]`

Table: `accounting.accounts`

```
CREATE TABLE accounting.accounts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    code VARCHAR(20) NOT NULL UNIQUE,
    name VARCHAR(255) NOT NULL,
    type VARCHAR(50) NOT NULL CHECK (type IN ('Asset', 'Liability', 'Equity', 'Revenue', 'Expense')),
    parent_id UUID REFERENCES accounting.accounts(id),
    opening_balance NUMERIC(15, 2) NOT NULL DEFAULT 0.00,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_accounts_code ON accounting.accounts(code);
CREATE INDEX idx_accounts_type ON accounting.accounts(type);
```

Table: `accounting.vouchers`

```
CREATE TABLE accounting.vouchers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    voucher_number VARCHAR(50) NOT NULL UNIQUE,
    voucher_type VARCHAR(20) NOT NULL CHECK (voucher_type IN ('Receipt', 'Payment', 'Journal')),
    voucher_date DATE NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'Draft' CHECK (status IN ('Draft', 'Approved', 'Posted', 'Cancelled',
    narration TEXT,
    created_by VARCHAR(100) NOT NULL,
    posted_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_vouchers_number ON accounting.vouchers(voucher_number);
CREATE INDEX idx_vouchers_status ON accounting.vouchers(status);
```

Table: `accounting.voucher_lines`

```
CREATE TABLE accounting.voucher_lines (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    voucher_id UUID NOT NULL REFERENCES accounting.vouchers(id) ON DELETE CASCADE,
    account_id UUID NOT NULL REFERENCES accounting.accounts(id),
    line_type VARCHAR(10) NOT NULL CHECK (line_type IN ('Debit', 'Credit')),
    amount NUMERIC(15, 2) NOT NULL CHECK (amount > 0),
    memo TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_voucher_lines_voucher ON accounting.voucher_lines(voucher_id);
CREATE INDEX idx_voucher_lines_account ON accounting.voucher_lines(account_id);
```

7.3 LocalStorage JSON to PostgreSQL ETL Pipeline

```
Client Workstation
   │
   ├──► 1. Export JSON Snapshot via Settings (state_backup_YYYY-MM-DD.json)
   │
Server Migration Node
   │
   ├──► 2. Run `scripts/postgresql/init-database.sh` (Creates schema & roles)
   │
   ├──► 3. Execute ETL Ingestion Engine (Maps string IDs to UUIDs)
   │
   └──► 4. Validate Ledger Balances (Asserts Σ Debits = Σ Credits)
```

7.4 Post-Migration Data Integrity Verification

Following ETL ingestion, a post-migration SQL script verifies double-entry balance integrity:

```
-- Post-Migration Ledger Balance Audit Check
SELECT
    SUM(CASE WHEN line_type = 'Debit' THEN amount ELSE 0 END) AS total_debits,
    SUM(CASE WHEN line_type = 'Credit' THEN amount ELSE 0 END) AS total_credits,
    ABS(SUM(CASE WHEN line_type = 'Debit' THEN amount ELSE -amount END)) AS balance_difference
FROM accounting.voucher_lines vl
JOIN accounting.vouchers v ON vl.voucher_id = v.id
WHERE v.status = 'Posted';
```

Expected Verification Output:

```
 total_debits | total_credits | balance_difference
--------------+---------------+--------------------
 1275000.00   | 1275000.00    | 0.00
(1 row)
```

7.5 Automated Migration Ingestion Script (`init-database.sh`)

Location: `docs/enterprise/scripts/postgresql/init-database.sh`

```
#!/usr/bin/env bash
# InsAcc PostgreSQL Database Initialization Script [To Be Implemented]

set -euo pipefail

DB_NAME="insacc_db"
DB_USER="insacc_user"
DB_PASS="SecurePassword123"

echo "[INFO] Initializing PostgreSQL database '${DB_NAME}'..."

sudo -u postgres psql <<EOF
DO \$\$
BEGIN
    IF NOT EXISTS (SELECT FROM pg_catalog.pg_roles WHERE rolname = '${DB_USER}') THEN
        CREATE ROLE ${DB_USER} WITH LOGIN PASSWORD '${DB_PASS}';
    END IF;
END
\$\$;

SELECT 'CREATE DATABASE ${DB_NAME} OWNER ${DB_USER}'
WHERE NOT EXISTS (SELECT FROM pg_database WHERE datname = '${DB_NAME}')\gexec

GRANT ALL PRIVILEGES ON DATABASE ${DB_NAME} TO ${DB_USER};
EOF

echo "[SUCCESS] Database initialized. Ready for schema migration DDL execution."
```

8. Summary

The target database migration plan specifies a multi-tenant PostgreSQL 17 architecture that preserves InsAcc double-entry accounting integrity during transition from local `localStorage` to enterprise server deployments.

9. Chapter Appendix

Entity Schema Mapping Table

LocalStorage Key	Target PostgreSQL Schema & Table	Primary Migration Challenge
`insacc_investments`	`investment.investments`	String ID to UUID conversion
`insacc_transactions`	`accounting.vouchers` & `voucher_lines`	Splitting flat items into double-entry lines
`insacc_prop_units`	`property.units`	Mapping building parent UUID references
`insacc_prop_rent`	`property.rent_payments`	Validating rent collection foreign keys

10. Glossary

- **DDL (Data Definition Language)**: SQL statements used to define database schemas, tables, columns, indexes, and constraints.
- **ETL (Extract, Transform, Load)**: A three-step data integration process used to pull data from one source, modify it, and write it to a target database.

- **UUID (Universally Unique Identifier)**: A 128-bit label used to uniquely identify resources in computer systems without central coordination.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- LocalStorage Persistence Architecture: Volume 02 Chapter 01
- Database Design Spec: docs/DATABASE_DESIGN_SPECIFICATION.md

title: "Volume 02: Data Management & Persistence Guide - Chapter 04: Database Server Setup Guide [To Be Implemented]" document_id: "INSACC-DOC-V02-CH04" version: "1.0.0" release_date: "2026-07-22" app_version: "v2.0.0 Target Server" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Target Architecture Specification" classification: "Commercial Enterprise Documentation"

Volume 02: Data Management & Persistence Guide

Chapter 04: Database Server Setup Guide `[To Be Implemented]`

> **Single Source of Truth Reference**: All server provisioning rules, PostgreSQL 17 configurations, memory tuning parameters, and DDL schema deployments defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise database setup specification	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter provides Database Administrators (DBAs) and Infrastructure Engineers with a step-by-step technical setup manual for installing, tuning, securing, and deploying PostgreSQL 17 target enterprise database servers `[To Be Implemented]`.

2. Scope

This specification covers:

- Linux kernel tuning (`sysctl.conf`, `limits.conf`) and storage options.
- PostgreSQL 17 installation via PGDG repository setup.
- Network security (`pg_hba.conf`) and TLS 1.3 encryption.
- Memory tuning parameters in `postgresql.conf`.
- Multi-schema deployment (`accounting`, `investment`, `property`, `auth`, `audit`).
- Automated double-entry balance validation triggers.
- Database backup automation (`pg_dump`) and PITR recovery.

Out of Scope:

- Desktop client `localStorage` architecture (covered in Volume 02 Chapter 01).

3. Audience

This document is authored for:

- Database Administrators & Infrastructure Engineers
- Systems Administrators & DevOps Leads
- Enterprise Compliance Officers

4. Prerequisites

1. Review PostgreSQL standards in MASTER_ARCHITECTURE.md#7-postgresql-architecture.
2. Provision an Ubuntu 24.04 LTS or RHEL 9 server node (4 vCPU, 8 GB RAM, 100 GB SSD).

5. Warnings & Operational Hazards

> **WARNING: To Be Implemented**: The PostgreSQL 17 server setup procedures described in this chapter are target specifications for enterprise release v2.0.0.

6. Notes & Architecture Context

> **NOTE: Automated Trigger Defense**: Double-entry balance equality ($\sum D = \sum C$) is enforced directly at the database engine level via `trg_validate_voucher_posting`.

7. Main Content

7.1 OS Provisioning & Kernel Parameter Tuning

Optimize kernel parameters in `/etc/sysctl.d/99-postgresql.conf`:

```
kernel.shmmax = 18446744073709551615
kernel.shmall = 18446744073709551615
vm.overcommit_memory = 2
vm.overcommit_ratio = 80
vm.swappiness = 10
```

7.2 PostgreSQL 17 Package Installation

```
# Install PGDG key and repository
sudo apt-get update && sudo apt-get install -y curl ca-certificates gnupg
curl -fsSL https://www.postgresql.org/media/keys/ACCC4CF8.asc | sudo gpg --dearmor -o /etc/apt/keyrings/postgr
echo "deb [signed-by=/etc/apt/keyrings/postgresql.gpg] http://apt.postgresql.org/pub/repos/apt $(lsb_release -
sudo apt-get update && sudo apt-get install -y postgresql-17 postgresql-contrib-17
```

7.3 Network Access Control & TLS 1.3 Encryption (`pg_hba.conf`)

Configure host-based authentication in `/etc/postgresql/17/main/pg_hba.conf`:

```
local   all             postgres                                peer
hostssl insacc_db       insacc_user     10.0.4.0/24             scram-sha-256
```

7.4 Performance Tuning & Memory Allocation (`postgresql.conf`)

Location: `docs/enterprise/configs/postgresql/postgresql.conf`

```
shared_buffers = 2GB
work_mem = 32MB
maintenance_work_mem = 512MB
effective_cache_size = 6GB
wal_level = replica
max_wal_size = 4GB
```

7.5 Multi-Schema DDL Deployment Pipeline

Deploy the 5 enterprise schemas:

```
CREATE SCHEMA IF NOT EXISTS accounting AUTHORIZATION insacc_user;
CREATE SCHEMA IF NOT EXISTS investment AUTHORIZATION insacc_user;
CREATE SCHEMA IF NOT EXISTS property AUTHORIZATION insacc_user;
CREATE SCHEMA IF NOT EXISTS auth AUTHORIZATION insacc_user;
CREATE SCHEMA IF NOT EXISTS audit AUTHORIZATION insacc_user;
```

7.6 Automated Balance Validation Trigger Functions

```
CREATE OR REPLACE FUNCTION accounting.fn_validate_voucher_balance()
RETURNS TRIGGER AS $$
DECLARE
    v_total_debit NUMERIC(15,2);
    v_total_credit NUMERIC(15,2);
BEGIN
    SELECT
        COALESCE(SUM(CASE WHEN line_type = 'Debit' THEN amount ELSE 0 END), 0),
        COALESCE(SUM(CASE WHEN line_type = 'Credit' THEN amount ELSE 0 END), 0)
    INTO v_total_debit, v_total_credit
    FROM accounting.voucher_lines WHERE voucher_id = NEW.id;

    IF ABS(v_total_debit - v_total_credit) >= 0.001 THEN
        RAISE EXCEPTION 'Voucher posting rejected: Unbalanced entries. Debits (%) != Credits (%)', v_total_deb
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

7.7 Database Backup, WAL Archiving & Point-In-Time Recovery (PITR)

Automated daily backup command:

```
pg_dump -h localhost -U insacc_user -F c -b -v -f "/var/backups/postgresql/insacc_db_$(date +%Y-%m-%d).dump" i
```

8. Summary

Chapter 04 provides a complete database server setup manual for InsAcc Target Server v2.0.0, covering kernel tuning, security hardening, memory optimization, DDL triggers, and backup automation.

9. Chapter Appendix

Reference Configuration Directory

Artifact	File Location	Purpose
Dedicated Setup Spec	`docs/DATABASE_SERVER_SETUP_GUIDE.md`	Full database setup manual
PostgreSQL Config	`docs/enterprise/configs/postgresql/postgresql.conf`	Server tuning parameters
Authentication Config	`docs/enterprise/configs/postgresql/pg_hba.conf`	CIDR access control rules

10. Glossary

- **DBA**: Database Administrator.
- **PL/pgSQL**: Procedural language for PostgreSQL trigger logic.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Dedicated Server Setup Guide: docs/DATABASE_SERVER_SETUP_GUIDE.md
- Database Design Spec: docs/DATABASE_DESIGN_SPECIFICATION.md

title: "Volume 03: System Administrator Guide - Chapter 01: Initial Setup and Company Configuration" document_id: "INSACC-DOC-V03-CH01" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 03: InsAcc System Administrator Guide

Chapter 01: Initial Setup and Company Configuration

> **Single Source of Truth Reference**: All administrative settings, locale parameters, and system preferences defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter provides system administrators with step-by-step instructions for configuring company-wide preferences, base reporting currency, date formatting masks, multi-language dictionaries, and interface theme palettes in the InsAcc ERP platform.

2. Scope

This specification covers:

- System settings administration in `Settings.tsx` (`src/renderer/components/Settings.tsx`).
- Base accounting currency selection (`AED`, `USD`, `EUR`, `GBP`, `SAR`, etc.) and formatting (`reportFormatters.ts`).
- Regional date format masks (`YYYY-MM-DD`, `DD/MM/YYYY`, `MM/DD/YYYY`).
- Language translation dictionary engine (`t()` helper function in `utils.ts`).
- Theme switching mechanisms (`.dark-mode` class toggling on `document.documentElement`).

Out of Scope:

- User profile creation and RBAC permissions (covered in Volume 03 Chapter 02).
- Chart of Accounts setup (covered in Volume 03 Chapter 03).

3. Audience

This document is authored for:

- System Administrators and IT Operations Directors
- Financial Controllers and Chief Accountants
- Implementation Project Managers
- Technical Support Personnel

4. Prerequisites

Before configuring system preferences:

1. Log in to InsAcc with a user profile assigned the `Admin` role (`role === 'Admin'`).
2. Confirm company base currency requirements with the financial accounting department.

5. Warnings & Operational Hazards

> **WARNING: BASE CURRENCY SELECTION TIMING**: Changing the Base Currency parameter after financial transactions and double-entry vouchers have been posted will re-label currency symbols across historical reports without converting historical numeric values. System administrators MUST establish the Base Currency prior to entering operational data.

6. Notes & Architecture Context

> **NOTE: Instant Theme Application**: Toggling between Dark Mode and Light Mode executes instantly by updating CSS Custom Properties (`--bg` , `--surface` , `--border` , `--text-primary`). Theme switches do NOT require restarting the application or clearing browser storage.

---

7. Main Content

7.1 Accessing the Administration Console (`Settings.tsx`)

System administration options are centralized in the **Settings** view:

1. Click **Settings** at the bottom of the left-hand navigation sidebar.
2. The Settings console renders four primary administrative tabs:
   - **General Settings**: Currency, Date Format, Language, Theme preferences.
   - **User Management**: User creation, profile editing, and role assignment.
   - **Security**: Authentication PIN setup, password policies, and credentials.
   - **Data Management**: System backup exports (JSON), imports, and factory data resets.

7.2 Base Currency & Currency Formatting Configuration

InsAcc supports multi-currency display, but consolidates all financial statements into a single **Base Accounting Currency**:

```
// Base Currency Parameters in Settings.tsx
export const SUPPORTED_CURRENCIES = [
  { code: 'AED', name: 'UAE Dirham', symbol: 'AED' },
  { code: 'USD', name: 'US Dollar', symbol: '$' },
  { code: 'EUR', name: 'Euro', symbol: '€' },
  { code: 'GBP', name: 'British Pound', symbol: '£' },
  { code: 'SAR', name: 'Saudi Riyal', symbol: 'SAR' },
  { code: 'QAR', name: 'Qatari Riyal', symbol: 'QAR' },
  { code: 'OMR', name: 'Omani Rial', symbol: 'OMR' },
  { code: 'BHD', name: 'Bahraini Dinar', symbol: 'BHD' },
  { code: 'KWD', name: 'Kuwaiti Dinar', symbol: 'KWD' },
  { code: 'INR', name: 'Indian Rupee', symbol: '₹' }
]
```

Formatting Execution (`reportFormatters.ts`):

```
export function formatCurrency(value: number, currency: string = 'AED'): string {
  const isNegative = value < 0
  const absVal = Math.abs(value).toLocaleString('en-US', {
    minimumFractionDigits: 2,
    maximumFractionDigits: 2
  })
  return `${isNegative ? '-' : ''}${currency} ${absVal}`
}
```

7.3 Date Format Mask & Regional Formatting

Administrators can select the global date presentation mask used in tables, forms, and report exports:

Mask Format	Example Render	Target Use Case / Region
`YYYY-MM-DD` (Default)	`2026-06-30`	ISO 8601 International Standard / Enterprise Financials
`DD/MM/YYYY`	`30/06/2026`	United Kingdom, UAE, Europe, Commonwealth Standard
`MM/DD/YYYY`	`06/30/2026`	United States Regional Standard

7.4 Multi-Language UI Translation Engine (`utils.ts`)

InsAcc incorporates a lightweight translation engine (`src/renderer/utils.ts`):

```
export function t(key: string, lang: 'English' | 'Arabic' | 'French' = 'English'): string {
  const dictionary: Record<string, Record<string, string>> = {
    'Dashboard': { English: 'Dashboard', Arabic: 'لوحة التحكم', French: 'Tableau de bord' },
    'Investments': { English: 'Investments', Arabic: 'الاستثمارات', French: 'Investissements' },
    'Property': { English: 'Property Management', Arabic: 'إدارة العقارات', French: 'Gestion immobilière' },
    'Reports': { English: 'Financial Reports', Arabic: 'التقارير المالية', French: 'Rapports financiers' }
  }
  return dictionary[key]?.[lang] || key
}
```

7.5 Interface Theme Palette Customization (Dark vs Light)

InsAcc defaults to a dark mode palette (`#0C0C0D` background, `#1C1C1F` surface, `#6366F1` primary, `#F59E0B` gold accents).

```
// Theme Toggle Handler in Settings.tsx
const toggleTheme = (newTheme) => {
  setTheme(newTheme)
  if (newTheme === 'dark') {
    document.documentElement.classList.add('dark-mode')
  } else {
    document.documentElement.classList.remove('dark-mode')
  }
}
```

8. Summary

The Settings console provides administrators with centralized control over currency, date masks, UI languages, and visual themes. Configuring these parameters during initial deployment establishes consistent financial formatting across all operational views and reports.

9. Chapter Appendix

Global Settings Configuration Reference Matrix

Setting Parameter	Available Values	Storage State Key	Default Value
`currency`	`AED`, `USD`, `EUR`, `GBP`, `SAR`, etc.	React State / `App.tsx`	`'AED'`
`dateFormat`	`YYYY-MM-DD`, `DD/MM/YYYY`, `MM/DD/YYYY`	React State / `App.tsx`	`'YYYY-MM-DD'`
`language`	`English`, `Arabic`, `French`	React State / `App.tsx`	`'English'`
`theme`	`dark`, `light`	React State / `App.tsx`	`'dark'`

10. Glossary

- **Base Currency**: The primary accounting currency in which a company's financial statements are consolidated and presented.
- **ISO 8601**: An international standard covering the worldwide exchange and communication of date and time-related data (`YYYY-MM-DD`).
- **Localization (L10n)**: The process of adapting software for a specific region or language by translating text and formatting dates/currencies.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- User Profiles & RBAC: Volume 03 Chapter 02
- Chart of Accounts Management: Volume 03 Chapter 03

title: "Volume 03: System Administrator Guide - Chapter 02: User Profiles and Access Control" document_id: "INSACC-DOC-V03-CH02" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 03: InsAcc System Administrator Guide

Chapter 02: User Profiles and Access Control

> **Single Source of Truth Reference**: All user profile structures, domain isolation models, and Role-Based Access Control (RBAC) rules defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter details the security architecture, domain profile isolation, and Role-Based Access Control (RBAC) models in InsAcc v1.0.0. It provides instructions for managing user profiles, assigning operational roles (`Admin` vs `Accounts`), and enforcing credential security policies.

2. Scope

This specification covers:

- Profile domain separation (`invUsers` persisted in `insacc_inv_users` vs `propUsers` persisted in `insacc_prop_users`).
- The complete 10-point RBAC permission matrix for `Admin` and `Accounts` roles.
- The `UserEntry` TypeScript interface specification.
- Administrative user CRUD workflows in `Settings.tsx` .
- Security PIN validation and password policies.

Out of Scope:

- General application settings (covered in Volume 03 Chapter 01).
- Security vulnerability audit findings & hardening roadmap (covered in Volume 08 Chapter 02).

3. Audience

This document is authored for:

- Systems Administrators and Security Compliance Officers
- IT Service Desk Leads and Access Management Engineers
- Internal Audit Technicians

4. Prerequisites

Before managing user access:

1. Log in to InsAcc with a user profile assigned `Admin` privileges.
2. Review security governance standards in MASTER_ARCHITECTURE.md#9-authentication-architecture.

5. Warnings & Operational Hazards

> **WARNING: ADMIN ROLE ASSIGNMENT HAZARD**: Users assigned the `Admin` role possess unrestricted authority to execute factory data resets, unlock closed fiscal periods, and delete general ledger accounts. Administrators MUST limit `Admin` role assignments to authorized IT and senior accounting personnel.

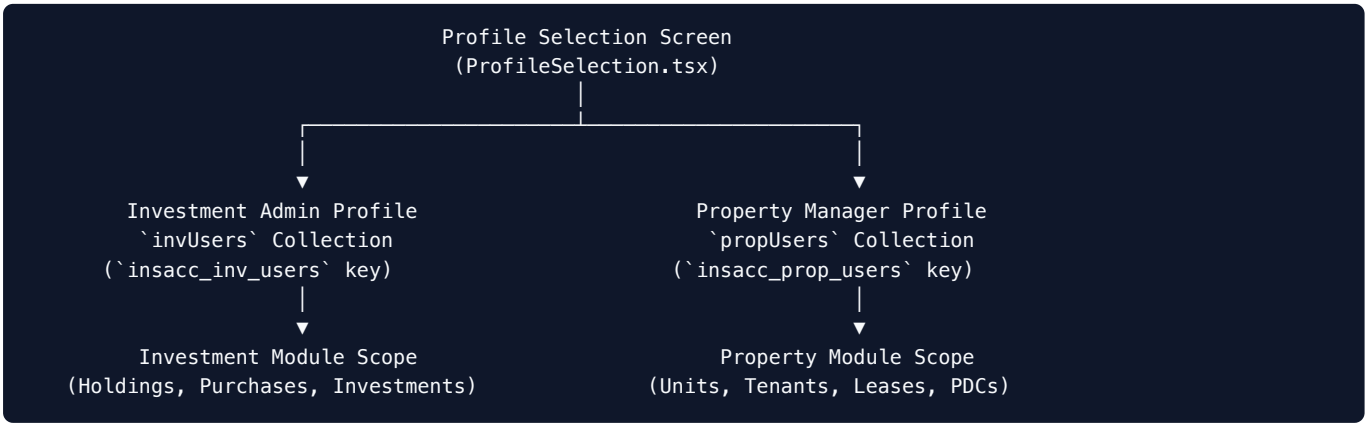
6. Notes & Architecture Context

> **NOTE: Profile Storage Separation**: Investment module users (`invUsers`) and Property module users (`propUsers`) are stored in isolated `localStorage` keys (`insacc_inv_users` and `insacc_prop_users`). This ensures clear separation of duties between portfolio wealth managers and real estate property operations personnel.

7. Main Content

7.1 Profile Architecture & Functional Domain Isolation

Upon logging into InsAcc, users select a functional domain profile (`ProfileSelection.tsx`):



7.2 Role-Based Access Control (RBAC) Permission Matrix

InsAcc enforces a 2-tier role hierarchy: **`Admin`** and **`Accounts`**.

System Operation / Feature Access	**`Admin`** Role	**`Accounts`** Role
View Dashboards & Financial Reports	✓ Allowed	✓ Allowed
Record Double-Entry Vouchers (`RV` , `PV` , `JV`)	✓ Allowed	✓ Allowed
Approve & Post Vouchers to General Ledger	✓ Allowed	✓ Allowed
Record Asset Purchases & Holdings	✓ Allowed	✓ Allowed
Manage Tenants, Leases & PDC Cheques	✓ Allowed	✓ Allowed
Add / Edit / Archive Bank Accounts	✓ Allowed	✕ Read-Only
Add / Edit / Remove User Profiles	✓ Allowed	✕ Access Denied
Execute Period Closing Wizard Operations	✓ Allowed	✕ Access Denied
Reopen Closed Fiscal Periods	✓ Allowed	✕ Access Denied
Execute Factory Data Reset	✓ Allowed	✕ Access Denied

7.3 The `UserEntry` Interface & Storage Schemas

User records are structured according to the `UserEntry` TypeScript interface:

```
export interface UserEntry {
  id: string                  // Unique user identifier (e.g. "usr-1719500000000")
  name: string                // Full user display name (e.g. "Sarah Jenkins")
  email: string               // User email address
  role: 'Admin' | 'Accounts'  // Assigned RBAC authorization role
}
```

7.4 User Account Creation, Edit, and Deactivation Workflows

System administrators manage user accounts via **Settings** $\rightarrow$ **User Management**:

```
Settings Console ⟶ User Management ⟶ [+ Add User] ⟶ Save to Storage Key
```

Adding a User Account:

1. Open **Settings** $\rightarrow$ select the **Users** tab.
2. Click **+ Add User**.
3. Complete user details:
   - **Full Name**: (e.g., `Robert Vance`).
   - **Email Address**: (e.g., `robert.vance@company.com`).
   - **Assigned Role**: Select `Admin` or `Accounts`.
4. Click **Save User**.
5. The system serializes the updated user array to `insacc_inv_users` or `insacc_prop_users`.

7.5 PIN & Password Security Enforcement

- **PIN Authentication**: Application access requires entering a numeric security PIN on the Login screen (`Login.tsx`).
- **Password Modification**: Administrators update master passwords via **Settings** $\rightarrow$ **Security** $\rightarrow$ **Change Password**.

8. Summary

InsAcc provides domain isolation between Investment and Property modules, supported by a 2-tier RBAC framework (`Admin` vs `Accounts`). By segregating duties and restricting sensitive administrative actions, InsAcc maintains access control integrity across enterprise operations.

9. Chapter Appendix

Standard User Profile Dictionary

User ID	Profile Name	Default Role	Assigned Domain Scope
`usr-1001`	Investor Admin	`Admin`	Investment Portfolio Module (`invUsers`)
`usr-1002`	Property Operations Manager	`Admin`	Property Management Module (`propUsers`)
`usr-1003`	Staff Accountant	`Accounts`	Shared Operational Voucher Entry

10. Glossary

- **RBAC (Role-Based Access Control)**: An approach to restricting system access to authorized users based on their assigned organizational roles.
- **Separation of Duties (SoD)**: The concept of having more than one person required to complete a task to prevent fraud and error.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Initial Setup: Volume 03 Chapter 01
- Chart of Accounts: Volume 03 Chapter 03
- Credential Security Audit: Volume 08 Chapter 02

title: "Volume 03: System Administrator Guide - Chapter 03: Chart of Accounts Management" document_id: "INSACC-DOC-V03-CH03" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 03: InsAcc System Administrator Guide

Chapter 03: Chart of Accounts Management

> **Single Source of Truth Reference**: All account taxonomy, reserved account codes, and double-entry rules defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter provides system administrators with detailed rules and operational procedures for managing the Chart of Accounts (COA) in InsAcc. It details the five primary account categories, reserved system account codes, hierarchical tree nodes, and rules for creating custom general ledger accounts.

2. Scope

This specification covers:

- Account category taxonomy (`1000` Assets, `2000` Liabilities, `3000` Equity, `4000` Revenue, `5000` Expenses).
- Reserved account codes registered in `systemAccountRegistry.ts`.
- Hierarchical account tree rendering (`TrialBalanceTree.tsx` & `TreeView.tsx`).
- Parent header nodes vs child posting nodes.
- Account management service functions in `chartOfAccountsService.ts`.

Out of Scope:

- Double-entry voucher posting mechanics (covered in Volume 04 Chapter 07).
- Double-entry engine posting validator rules (covered in Volume 06 Chapter 02).

3. Audience

This document is authored for:

- Chief Accountants and System Administrators
- Financial Controllers and General Ledger Managers
- ERP Implementation Consultants

4. Prerequisites

Before modifying the Chart of Accounts:

1. Log in to InsAcc with `Admin` privileges.
2. Understand normal account balances specified in MASTER_ARCHITECTURE.md#103-primary-system-account-codes.

5. Warnings & Operational Hazards

> **WARNING: RESERVED SYSTEM ACCOUNT CODE MODIFICATION**: System account codes registered in `systemAccountRegistry.ts` (e.g., `1110` , `1120` , `1130` , `1410` , `2110` , `2120` , `4120`) are hard-coded into automated posting rules (`postingRules.ts`). Deleting or re-numbering reserved system accounts will cause automated voucher generation to fail.

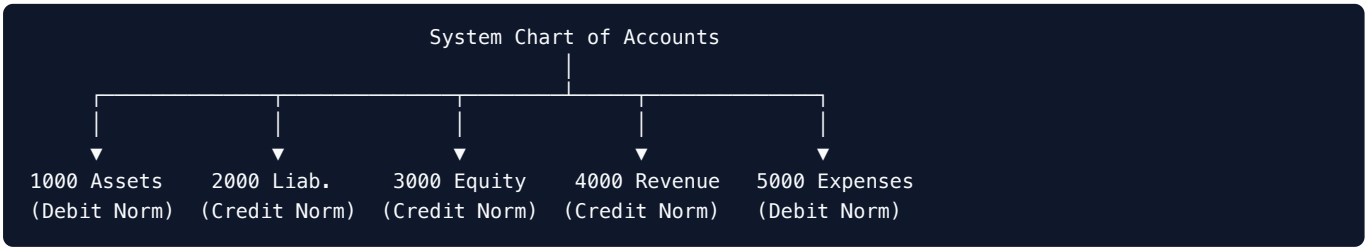
6. Notes & Architecture Context

> **NOTE: Parent Header vs Posting Nodes**: Parent header accounts (e.g., `1100 Current Assets`, `1120 Bank Accounts`) act strictly as structural aggregation nodes. Users CANNOT post vouchers directly to parent header accounts. Vouchers MUST target child posting accounts (e.g., `1120.001 Emirates Islamic Bank`).

7. Main Content

7.1 The Five Primary Account Categories & Normal Balances

Every account in InsAcc belongs to one of five primary financial categories:



Category Code Range	Category Name	Normal Balance	Increases With	Decreases With	Financial Statement Assignment
1000 – 1999	**Assets**	Debit (D)	Debit	Credit	Balance Sheet (Assets)
2000 – 2999	**Liabilities**	Credit (C)	Credit	Debit	Balance Sheet (Liabilities)
3000 – 3999	**Equity**	Credit (C)	Credit	Debit	Balance Sheet (Equity)
4000 – 4999	**Revenue**	Credit (C)	Credit	Debit	Profit & Loss (Income)
5000 – 5999	**Expenses**	Debit (D)	Debit	Credit	Profit & Loss (Expenses)

7.2 The System Reserved Account Registry (`systemAccountRegistry.ts`)

Reserved system account codes are registered in `systemAccountRegistry.ts`:

```
export const SystemAccountRegistry = {
  CASH: '1110',              // Cash on Hand
  BANK: '1120',              // Bank Accounts (Parent Group)
  RENT_RECEIVABLE: '1130',   // Tenant Rent Receivable
  PDC_HELD: '1410',          // Post-Dated Cheques Held
  UNEARNED_RENT: '2110',     // Unearned / Deferred Rental Income
  SECURITY_DEPOSITS: '2120', // Tenant Security Deposit Liability
  OWNER_CAPITAL: '2200',    // Owner Equity / Retained Earnings
  DIVIDEND_INCOME: '4110',   // Investment Dividend Income
  RENTAL_REVENUE: '4120',    // Property Rental Revenue
  PROPERTY_EXPENSE: '5110',  // Property Maintenance Expense
  DEPRECIATION_EXP: '5190',  // Asset Depreciation Expense
  ACCUMULATED_DEP: '1290',   // Accumulated Depreciation (Contra-Asset)
} as const
```

7.3 Hierarchical Account Tree Structure (`TrialBalanceTree.tsx`)

Accounts are rendered as a collapsible tree in `TrialBalanceTree.tsx`:

```
1000 Assets (Category Group Header)
  ├── 1100 Current Assets (Parent Header Node)
  │     ├── 1110 Cash on Hand (Posting Node)
  │     ├── 1120 Bank Accounts (Parent Group Header)
  │     │     ├── 1120.001 Emirates Islamic Bank (Posting Node)
  │     │     └── 1120.002 ADCB Operations Bank (Posting Node)
  │     ├── 1130 Rent Receivable (Posting Node)
  │     └── 1410 PDC Cheques Held (Posting Node)
  └── 1200 Fixed & Investment Assets (Parent Header Node)
        ├── 1210 Real Estate Property (Posting Node)
        └── 1290 Accumulated Depreciation (Contra-Asset Posting Node)
```

7.4 Adding and Editing Custom General Ledger Accounts

Administrators add custom posting sub-accounts via **Chart of Accounts**:

```
Open COA View ──► Select Parent Node ──► Click [+ Add Sub-Account] ──► Save Node
```

1. Navigate to **Chart of Accounts**.
2. Select the target parent group (e.g., `5100 Operating Expenses`).
3. Click **+ Add Sub-Account**.
4. Enter account details:
   - **Account Code**: Must follow parent code prefixing (e.g., `5100.005`).
   - **Account Name**: Descriptive title (e.g., `Elevator Maintenance`).
   - **Account Type**: Inherits parent type (`Expense`).
5. Click **Save Account**. `chartOfAccountsService.ts` validates code uniqueness and attaches the node to the account tree.

7.5 Account Deactivation & Archival Rules

- Accounts with **active transactions or non-zero balances** CANNOT be deleted.
- An account with zero balance can be set to `is_active = false`. Deactivated accounts are hidden from voucher selection drop-downs while maintaining historical report integrity.

---

8. Summary

The Chart of Accounts organizes InsAcc financial accounting into a 5-category tree structure. By combining reserved system account codes for automated event posting with customizable sub-accounts for business granularity, InsAcc ensures financial integrity and reporting precision.

---

9. Chapter Appendix

Standard System Chart of Accounts Registry Table

Account Code	Account Name	Category	Node Type	Normal Balance
`1110`	Cash on Hand	Asset	Posting	Debit
`1120`	Bank Accounts	Asset	Parent Header	Debit
`1120.001`	Emirates Islamic Bank	Asset	Posting	Debit
`1130`	Rent Receivable	Asset	Posting	Debit
`1410`	PDC Cheques Held	Asset	Posting	Debit
`2110`	Unearned Rent Liability	Liability	Posting	Credit
`2120`	Tenant Security Deposits	Liability	Posting	Credit
`2200`	Owner's Capital / Equity	Equity	Posting	Credit
`4110`	Dividend & Interest Income	Revenue	Posting	Credit
`4120`	Property Rental Revenue	Revenue	Posting	Credit
`5110`	Property Maintenance Expense	Expense	Posting	Debit

10. Glossary

- **Chart of Accounts (COA)**: An index of all financial accounts in the general ledger of a company.
- **Contra-Asset Account**: An asset account with a credit balance that offsets the balance of a paired asset account (e.g., Accumulated Depreciation `1290`).
- **Normal Balance**: The expected debit or credit balance classification for a given account category.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- User Access Control: Volume 03 Chapter 02
- Period Closing Operations: Volume 03 Chapter 04
- Double-Entry Engine Specs: Volume 06 Chapter 02

title: "Volume 03: System Administrator Guide - Chapter 04: Period Closing Operations" document_id: "INSACC-DOC-V03-CH04" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 03: InsAcc System Administrator Guide

Chapter 04: Period Closing Operations

> **Single Source of Truth Reference**: All period closing algorithms, fiscal period locking rules, and closing voucher specifications defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

- 11. References

1. Purpose

This chapter details the fiscal period closing procedures in InsAcc. It provides step-by-step instructions for executing the 4-Step Period Closing Wizard (`PeriodClosingWizard.tsx` / `periodService.ts`), auditing unposted items, generating automated Closing Journal Vouchers (`JV`), transferring net income to Retained Earnings (`2200`), and locking closed periods.

2. Scope

This specification covers:

- Fiscal period definition and closing frequency (monthly, quarterly, annual).
- Pre-closing audit steps for unposted vouchers and unreconciled PDC cheques.
- Automated Closing Journal Voucher generation logic (`periodCloser.ts`).
- Transferring revenue (`4000–4999`) and expense (`5000–5999`) balances to Retained Earnings (`2200`).
- Fiscal period locking in `postingValidator.ts` and administrative reopening runbooks.

Out of Scope:

- General voucher creation (covered in Volume 04 Chapter 07).
- Financial statement reporting (covered in Volume 04 Chapter 08).

3. Audience

This document is authored for:

- Chief Accountants and System Administrators
- Financial Controllers and Enterprise Auditors
- Senior Accounting Operations Technicians

4. Prerequisites

Before initiating period closing:

1. Log in with `Admin` privileges.
2. Confirm that all daily transactions and bank statements for the closing period have been imported.
3. Review period closing specs in MASTER_ARCHITECTURE.md#10-accounting-architecture.

5. Warnings & Operational Hazards

> **WARNING: CLOSED PERIOD POSTING PROHIBITION**: Once a fiscal period status transitions to `Locked` , the double-entry posting validator (`postingValidator.ts`) will reject any attempt to create, edit, post, or reverse vouchers dated within that period range. Re-opening a closed period requires explicit `Admin` authorization and invalidates the previous Closing JV.

6. Notes & Architecture Context

> **NOTE: Balance Sheet Continuity**: Balance Sheet accounts (Assets `1000s`, Liabilities `2000s`, Equity `3000s`) are NOT zeroed out during period closing. Their closing balances automatically carry forward as Opening Balances for the subsequent fiscal period.

7. Main Content

7.1 Fiscal Period Closing Rationale & Frequency

Period closing is performed at fiscal month-end, quarter-end, or year-end to:

1. Prevent historical transaction tampering after financial reporting.
2. Zero out temporary Revenue and Expense accounts.
3. Consolidate Net Profit / Loss into Owner's Equity / Retained Earnings (`2200`).

7.2 The 4-Step Period Closing Wizard Pipeline (`PeriodClosingWizard.tsx`)

Period closing is executed through an interactive 4-step wizard:

```
Step 1: Unposted Vouchers Audit ──► Step 2: Unreconciled PDC Audit
                                                  │
                                                  ▼
Step 4: Lock Fiscal Period      ◄── Step 3: Retained Earnings Transfer
```

7.3 Step 1: Unposted Vouchers Audit

The closing wizard scans the general ledger for vouchers in `Draft` or `Approved` status dated within the closing range:

- **Requirement**: All vouchers MUST be in `Posted` or `Cancelled` status.
- **Action**: The wizard presents a table of unposted vouchers. Administrators must click **Post All Approved Vouchers** or **Cancel Draft Vouchers** before advancing.

7.4 Step 2: Unreconciled Cheques & Bank Items Audit

Scans for Post-Dated Cheques in `Received` or `Deposited` status past their maturity date:

- **Requirement**: Overdue cheques must be transitioned to `Cleared` or `Bounced` (`PropertyPdcManager.tsx`).

7.5 Step 3: Automated Closing Journal Voucher Generation

Upon proceeding to Step 3, `periodCloser.ts` automatically constructs a **Closing Journal Voucher (`JV`)**:

```
Closing Journal Voucher Structure:
  1. Debit: All Revenue Accounts (4000–4999)     ──► Zeroes Revenue Balances
  2. Credit: All Expense Accounts (5000–5999)    ──► Zeroes Expense Balances
  3. Credit (or Debit): Retained Earnings (2200) ──► Posts Net Profit / Loss
```

Example Closing JV Calculation:

- Total Revenue (`4000–4999`): `AED 150,000`
- Total Expense (`5000–5999`): `AED 45,000`
- Calculated Net Profit: `AED 105,000`

Generated Journal Voucher Entry:

- $\text{Debit: 4120 Rental Revenue} = \text{AED 150,000}$
- $\text{Credit: 5110 Property Maintenance} = \text{AED 45,000}$
- $\text{Credit: 2200 Retained Earnings (Equity)} = \text{AED 105,000}$

7.6 Step 4: Period Locking & Administrative Reopening Workflow

Period Locking:

1. Step 4 registers the closed date range (e.g., `2026-01-01` to `2026-06-30`) in `periodService.ts` .
2. The period status is set to `Locked` .
3. `postingValidator.ts` checks the closed period registry on every subsequent voucher submission and throws `ERR_PERIOD_LOCKED` if the date falls inside a locked period.

Administrative Reopening Workflow:

If an accounting audit correction requires posting to a closed period:

1. Administrator opens **Period Closing Wizard** $\rightarrow$ **Closed Periods**.
2. Select the locked period $\rightarrow$ click **Unlock Period**.
3. Reopening cancels the Closing JV and permits voucher entry.
4. After posting corrections, the administrator MUST re-run the wizard to generate an updated Closing JV.

---

8. Summary

The Period Closing Wizard enforces complete audit readiness by ensuring all vouchers are posted, generating automated closing JVs to zero temporary income/expense accounts, transferring net earnings to Retained Earnings (`2200`), and locking period date ranges against retroactive modification.

---

9. Chapter Appendix

Period Closing Checklist Matrix

Step ID	Verification Checklist Item	Primary Validation Rule	Pass Criteria
CHK-01	Draft Vouchers Audit	`vouchers.filter(v => v.status === 'Draft')`	Count = 0
CHK-02	Approved Vouchers Audit	`vouchers.filter(v => v.status === 'Approved')`	Count = 0
CHK-03	Overdue PDC Audit	Cheques maturing before closing date	Count = 0 (Cleared/Bounced)
CHK-04	Closing JV Balance	$\left	\sum D - \sum C \right
CHK-05	Revenue & Expense Reset	Revenue/Expense balances post-closing	Zero balance reset

10. Glossary

- **Closing Entry**: A journal entry made at the end of an accounting period to transfer temporary account balances (Revenue and Expense) to permanent accounts (Retained Earnings).
- **Fiscal Period**: A defined time frame (month, quarter, or year) used for financial reporting and performance measurement.
- **Retained Earnings**: Accumulated net income retained in the business, recorded under Equity (`2200`).

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- User Profiles & RBAC: Volume 03 Chapter 02
- Chart of Accounts: Volume 03 Chapter 03
- Double-Entry Engine Specs: Volume 06 Chapter 02

title: "Volume 04: End-User Manual - Chapter 01: Getting Started and Navigation" document_id: "INSACC-DOC-V04-CH01" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 04: InsAcc End-User Operations Manual

Chapter 01: Getting Started and Navigation

> **Single Source of Truth Reference**: All UI navigation structures, layout components, and screen transition flows defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter provides operational personnel with an overview of the InsAcc application interface, screen navigation workflows, module switching mechanisms, sidebar navigation controls, and layout structure.

2. Scope

This specification covers:

- The 4-stage screen transition pipeline (`login` $\rightarrow$ `profiles` $\rightarrow$ `module` $\rightarrow$ `dashboard`).
- User authentication and security PIN entry (`Login.tsx`).
- Profile selection (`Investor` vs `Property Manager` in `ProfileSelection.tsx`).
- Operational module selection (`Investment Portfolio` vs `Property Management` in `ModuleSelection.tsx`).
- Main application shell layout (`Sidebar.tsx` , `.page-header` , `.page-body`).
- User profile avatar badges and logout procedures.

Out of Scope:

- Portfolio holdings management (covered in Volume 04 Chapter 02).
- Property unit and lease creation (covered in Volume 04 Chapter 04).

3. Audience

This document is authored for:

- Asset Portfolio Managers and Family Office Technicians
- Property Operations Technicians and Property Managers
- Financial Accountants and Bookkeepers
- New Operational End Users

4. Prerequisites

Before logging in:

1. Ensure the desktop application has been installed (Volume 01 Chapter 02).
2. Obtain your security PIN or password from your system administrator.

5. Warnings & Operational Hazards

> **WARNING: UNSAVED DRAFT FORMS ON LOGOUT**: Logging out or closing the application while editing an uncommitted voucher or lease form will discard ephemeral form state. Users MUST save vouchers as `Draft` or submit forms before logging out.

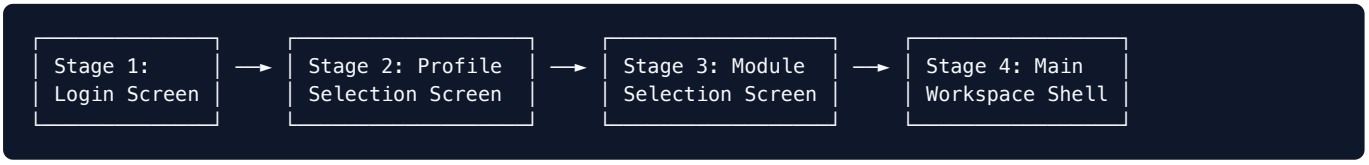
6. Notes & Architecture Context

> **NOTE: Responsive Collapsible Sidebar**: At screen resolutions below $\le 768\text{px}$ (tablet portrait viewports), the left sidebar automatically collapses from 240px to 56px, displaying icon-only navigation to maximize workspace area (`Sidebar.tsx`).

7. Main Content

7.1 4-Stage Screen Transition Workflow

When launching InsAcc, users navigate through a 4-stage screen sequence (`App.tsx` routing):



7.2 Stage 1: Authentication & Security PIN Entry (`Login.tsx`)

1. Launch InsAcc from your desktop shortcut or Applications menu.
2. The **Login Screen** renders a central security PIN pad:
   - Enter your assigned 4-digit security PIN or password.
   - Click **Login**.
3. Upon successful validation against `storedPassword` in state, the system advances to Stage 2.

7.3 Stage 2: Functional Domain Profile Selection (`ProfileSelection.tsx`)

Select your working domain profile:

- `Investor` **Profile**: Configured for wealth managers and portfolio accountants. Accesses `invUsers` settings.
- `Property Manager` **Profile**: Configured for real estate operations teams. Accesses `propUsers` settings.

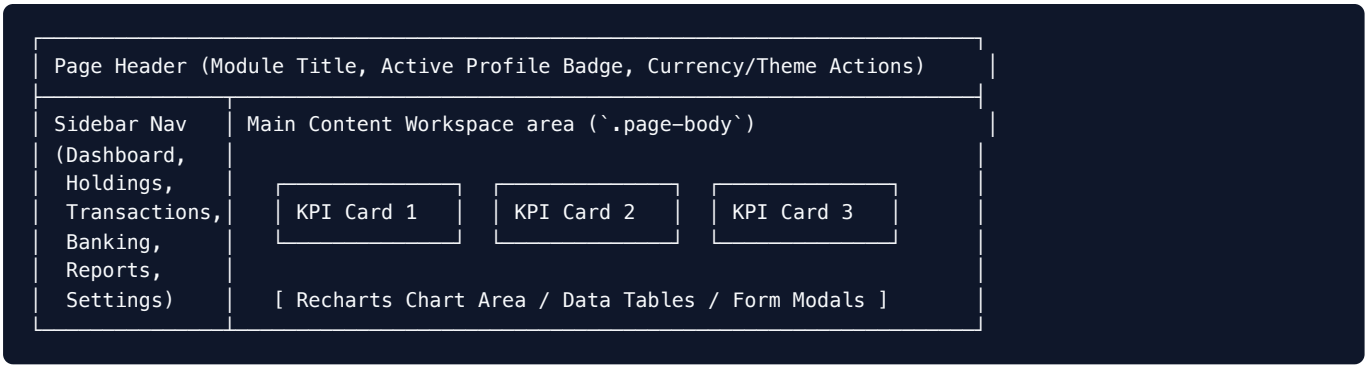
7.4 Stage 3: Operational Module Selection (`ModuleSelection.tsx`)

Choose the operational module for your current session:

1. **Investment Portfolio Module**: Wealth tracking, precious metals holdings, purchase ledger cost averaging, dividend income, and investment banking.
2. **Property Management Module**: Property categories, buildings, units, tenant lease contracts, PDC cheque lifecycle manager, security deposits, and rent collection.

7.5 Stage 4: Main Workspace Application Shell Layout

Upon module selection, the main workspace shell initializes:



1. Navigation Sidebar (`Sidebar.tsx`)

- Located on the left side of the screen (240px wide).
- Displays primary navigation links: **Dashboard**, **Holdings** (or **Properties**), **Transactions**, **Banking**, **Reports**, **Settings**.
- Bottom section displays the active user avatar, role badge (`Admin` / `Accounts`), and **Logout** button.

2. Page Header (`.page-header`)

- Displays current page title, active module indicator, and action toolbars (e.g., `+ Add Transaction` , `Export CSV`).

3. Workspace Area (`.page-body`)

- Scrollable workspace area rendering KPI summary cards, interactive charts, and data tables.

---

8. Summary

InsAcc provides a structured 4-stage login and module selection sequence leading to a unified application shell. With dedicated sidebar navigation, clear active profile indicators, and module isolation, operational users can navigate between portfolio management and property real estate accounting seamlessly.

---

9. Chapter Appendix

Application Navigation Hotkey Reference

Action / Shortcut	Operational Purpose
`Tab` / `Shift+Tab`	Navigate between form input fields in modal dialogs
`Esc`	Close active modal dialog or popup window
`Enter`	Submit focused form or confirm primary modal action

---

10. Glossary

- **Application Shell**: The minimal HTML, CSS, and JavaScript required to power the user interface structure, including navigation bars and workspace frames.
- **Hot Module Replacement (HMR)**: Live code replacement feature active in development mode.

---

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- User Profiles & Access Control: Volume 03 Chapter 02
- Investment Portfolio Management: Volume 04 Chapter 02
- Property Management: Volume 04 Chapter 04

title: "Volume 04: End-User Manual - Chapter 02: Investment Portfolio Management" document_id: "INSACC-DOC-V04-CH02" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 04: InsAcc End-User Operations Manual

Chapter 02: Investment Portfolio Management

> **Single Source of Truth Reference**: All investment data structures, valuation algorithms, and read-model calculations defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter provides operational procedures for managing wealth assets, tracking holdings, monitoring cost basis vs current market valuations, analyzing asset class allocations, and recording new investment positions in the InsAcc Investment Portfolio Module.

2. Scope

This specification covers:

- Investment Dashboard KPI cards and Recharts visual analytics (`InvestmentDashboard.tsx`).
- Asset allocation pie charts (`AssetAllocationPie.tsx`) and cash flow charts (`CashFlowChart.tsx`).
- Active holdings management across asset types (Gold, Silver, Stocks, Bonds, Mutual Funds, ETFs).
- Cost basis, market value, unrealized profit/loss, and portfolio weight calculations.
- Portfolio position CRUD workflows in `InvestmentHoldings.tsx` .

Out of Scope:

- Physical bullion purchase ledger lot tracking (covered in Volume 04 Chapter 03).
- Double-entry voucher postings for dividends and sales (covered in Volume 04 Chapter 07).

3. Audience

This document is authored for:

- Wealth Managers and Private Portfolio Accountants
- Asset Operations Personnel
- Family Office Financial Technicians

4. Prerequisites

Before managing investment holdings:

1. Log in to InsAcc and select the **`Investor`** profile.
2. Select the **Investment Portfolio Module**.
3. Confirm base currency formatting settings (Volume 03 Chapter 01).

5. Warnings & Operational Hazards

> **WARNING: UNREALIZED VS REALIZED GAIN DISCREPANCY**: Updating the `currentPrice` of an asset position updates **Unrealized Gain / Loss** metrics across dashboard charts and reports, but does NOT generate a double-entry general ledger voucher. Realized gains or losses are recognized ONLY when an asset sale voucher (`INVESTMENT_SALE`) is posted.

6. Notes & Architecture Context

> **NOTE: Memoized Projection Performance**: Portfolio valuation totals and percentage returns are projected dynamically using React `useMemo` hooks calling `investmentReadModels.ts`. This ensures sub-millisecond dashboard updates even when managing large portfolio collections.

7. Main Content

7.1 Investment Dashboard Overview (`InvestmentDashboard.tsx`)

The Investment Dashboard presents four real-time KPI summary cards:

1. **Total Portfolio Value**: Sum of current market valuations across all active holdings ($\sum \text{quantity} \times \text{currentPrice}$).
2. **Active Asset Holdings**: Total number of distinct investment positions in `insacc_investments`.
3. **Monthly Net Income**: Total dividend/interest income received minus investment management fees in the active month.
4. **YTD Portfolio Return %**: Aggregate percentage return across all portfolio holdings.

7.2 Asset Allocation & Growth Visualizations

Visualization Components:

1. **Asset Allocation Donut Chart (`AssetAllocationPie.tsx`)**: Displays portfolio distribution percentage broken down by asset class (Gold, Silver, Stocks, Bonds, Mutual Funds, ETFs).
2. **Investment Growth Area Chart (`InvestmentGrowthChart.tsx`)**: Plots trailing historical market value trends against original cost basis baselines.
3. **Cash Flow Bar Chart (`CashFlowChart.tsx`)**: Renders monthly investment income deposits vs operational costs.

7.3 Holdings Table & Asset Taxonomy (`InvestmentHoldings.tsx`)

The **Holdings** view catalog all asset positions persisted in `insacc_investments`:

```
export interface Investment {
  id: string              // Unique position identifier (e.g. "inv-1719500000000")
  assetName: string       // Asset description (e.g. "24K Gold Bar 1kg")
  assetType: string       // "Gold" | "Silver" | "Stocks" | "Bonds" | "Mutual Funds" | "ETFs"
  quantity: number        // Quantity owned
  purchaseValue: number   // Total cost basis (AED)
  currentPrice: number    // Market price per unit (AED)
  buyer?: string          // Custodian / Purchasing entity
  notes?: string          // Operational memo
}
```

7.4 Position Metrics & Unrealized Gain/Loss Calculations

For each position line in the Holdings table, the system projects metrics in real time:

Valuation Formulas:

1. **Position Cost Basis**: $$\text{Cost Basis} = \text{purchaseValue}$$
2. **Current Position Market Value**: $$\text{Market Value} = \text{quantity} \times \text{currentPrice}$$
3. **Unrealized Gain / Loss (AED)**: $$\text{Unrealized Profit} = \text{Market Value} - \text{Cost Basis}$$
4. **Return Percentage (%)**: $$\text{Return %} = \left(\frac{\text{Market Value} - \text{Cost Basis}}{\text{Cost Basis}} \right) \times 100$$
5. **Portfolio Weight (%)**: $$\text{Portfolio Weight %} = \left(\frac{\text{Position Market Value}}{\sum \text{Total Portfolio Market Value}} \right) \times 100$$

7.5 Adding and Editing Portfolio Positions

```
Open Holdings View ⟶ Click [+ Add Investment] ⟶ Fill Modal Form ⟶ Save Position
```

1. Navigate to **Investments** $\rightarrow$ click **+ Add Investment**.
2. Complete position fields in the modal dialog:
   - **Asset Name**: (e.g., `Apple Inc. (AAPL)`).
   - **Asset Type**: Select `Stocks`, `Gold`, `Bonds`, etc.
   - **Quantity**: Units owned (e.g., `100`).
   - **Total Purchase Value**: Total historical cost basis (e.g., `65,000 AED`).
   - **Current Price / Unit**: Active market valuation (e.g., `720 AED`).
3. Click **Save Investment**.
4. The system serializes updated state to `insacc_investments` and updates dashboard KPI cards immediately.

---

8. Summary

The Investment Portfolio Module provides wealth managers with real-time tracking of asset positions, asset allocations, and return percentages. By dynamically projecting market valuations against cost basis baselines, InsAcc delivers instant visibility into overall portfolio performance.

---

9. Chapter Appendix

Standard Asset Class Taxonomy Dictionary

Asset Class Code	Category Name	Description & Typical Instruments
`Gold`	Physical Gold	Bullion bars (1kg, 100g), gold coins, vaulted physical metal
`Silver`	Physical Silver	Silver bullion bars, industrial silver lots, coins
`Stocks`	Equities	Publicly traded common stock shares, equity securities
`Bonds`	Fixed Income	Government treasury bonds, corporate sukuk, fixed income
`Mutual Funds`	Managed Funds	Open-ended mutual funds, institutional pool accounts
`ETFs`	Exchange Traded Funds	Index tracking ETFs, sector commodity funds

10. Glossary

- **Cost Basis**: The original value of an asset for tax and accounting purposes, usually equal to the purchase price plus fees.
- **Unrealized Gain/Loss**: An increase or decrease in the paper value of an asset holding that has not yet been sold for cash.
- **YTD (Year-to-Date)**: The period starting from the beginning of the current calendar or fiscal year up to the present date.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Purchase Ledger Operations: Volume 04 Chapter 03
- Financial Reports: Volume 04 Chapter 08
- Accounting Engine Specs: Volume 06 Chapter 02

title: "Volume 04: End-User Manual - Chapter 03: Purchase Ledger Operations" document_id: "INSACC-DOC-V04-CH03" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 04: InsAcc End-User Operations Manual

Chapter 03: Purchase Ledger Operations

> **Single Source of Truth Reference**: All purchase ledger models, cost averaging formulas, and item statistics defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter provides operational procedures for recording physical asset acquisitions, managing purchase ledger categories, and computing weighted average unit cost metrics in the InsAcc Purchase Ledger Module.

2. Scope

This specification covers:

- Purchase ledger architecture in `PurchaseLedger.tsx` and `purchaseLedgerService.ts`.
- The `PurchaseRecord` TypeScript interface specification.
- Recording purchase lots (date, quantity, unit price, buyer, notes).
- Item cost averaging calculation engine (`computeAverages()` in `purchaseService.ts`).
- Weighted average cost vs simple average mathematical comparison.

Out of Scope:

- General investment portfolio holdings overview (covered in Volume 04 Chapter 02).
- Double-entry payment vouchers for purchase settlements (covered in Volume 04 Chapter 07).

3. Audience

This document is authored for:

- Bullion & Precious Metals Procurement Officers
- Inventory Accounting Technicians
- Asset Operations Personnel
- Internal Financial Auditors

4. Prerequisites

Before recording purchase transactions:

1. Log in to InsAcc and select the **`Investor`** profile.
2. Ensure purchase asset categories (e.g., `Gold`, `Silver`, `Commodities`) have been established in `insacc_purchase_categories`.

5. Warnings & Operational Hazards

> **WARNING: WEIGHTED COST BASIS ACCURACY**: When recording purchase lots, entering an incorrect `quantity` or `unitPrice` corrupts the calculated **Weighted Average Unit Price** across all historical cost summaries. Users MUST verify invoice values prior to saving purchase entries.

6. Notes & Architecture Context

> **NOTE: Consolidated Service Function**: The `computeAverages()` cost calculation function is centralized in `src/renderer/services/purchaseService.ts` and shared across both `Dashboard.tsx` and `PurchaseLedger.tsx` to eliminate code duplication.

7. Main Content

7.1 Purchase Ledger Rationale & Architecture (`PurchaseLedger.tsx`)

The Purchase Ledger acts as a granular transaction log for physical asset lot acquisitions (e.g., bullion bars, coin lots, silver bullion, physical commodities). While the Holdings view tracks cumulative position state, the Purchase Ledger records every individual purchase lot over time.

7.2 The `PurchaseRecord` Interface & Category Taxonomy

Purchase lot transactions are structured according to the `PurchaseRecord` interface:

```
export interface PurchaseRecord {
  id: string              // Unique transaction identifier (e.g. "PL-1719500000000")
  purchaseDate: string    // ISO date string (YYYY-MM-DD)
  assetType: string       // Asset category (e.g. "Gold", "Silver")
  assetName: string       // Item description (e.g. "24K Gold Bar 100g")
  quantity: number        // Quantity of units purchased
  unitPrice: number       // Unit purchase price (AED)
  totalValue: number      // Derived: quantity * unitPrice (AED)
  buyer?: string          // Custodian / Purchasing entity
  notes?: string          // Operational memo
}
```

7.3 Recording Individual Purchase Transactions

```
Open Purchase Ledger ──► Select Category & Item ──► Fill Purchase Form ──► Save Lot
```

1. Navigate to **Purchase Ledger**.
2. Select **Category** (e.g., `Gold`) and **Item Name** (e.g., `24K Gold Bar 100g`).
3. Under **Add Purchase Lot**:
   - **Date**: Select transaction economic date (e.g., `2026-06-15`).
   - **Quantity**: Enter units acquired (e.g., `5`).
   - **Unit Price**: Enter unit price (e.g., `28,500 AED`).
4. The system calculates **Total Value** (`142,500 AED`).
5. Click **Record Purchase**. The purchase lot is serialized to `insacc_purchases` and updates item cost summary cards.

7.4 Item Cost Averaging Engine (`computeAverages()`)

When multiple purchase lots exist for an item, InsAcc computes aggregate metrics via `computeAverages()`:

```
export interface ItemAverages {
  itemId: string
  itemName: string
  categoryName: string
  purchaseCount: number
  totalQuantity: number
  totalValue: number
  avgUnitPrice: number
  avgValue: number
  avgQuantity: number
}
```

Cost Averaging Formulas:

1. **Total Units Acquired**: $$\text{Total Qty} = \sum_{i=1}^{n} \text{quantity}_i$$
2. **Total Cost Basis**: $$\text{Total Value} = \sum_{i=1}^{n} (\text{quantity}_i \times \text{unitPrice}_i)$$
3. **Weighted Average Unit Price**: $$\text{Weighted Avg Unit Price} = \frac{\text{Total Value}}{\text{Total Qty}}$$
4. **Average Lot Size**: $$\text{Avg Lot Size} = \frac{\text{Total Qty}}{n}$$

7.5 Weighted Average Unit Price vs Simple Average Comparison

InsAcc uses **Weighted Average Costing** rather than Simple Average to ensure financial precision:

Scenario Example:

- Lot 1: 10 units @ AED 100/unit (Total = AED 1,000)
- Lot 2: 90 units @ AED 200/unit (Total = AED 18,000)
- **Total Quantity** = 100 units | **Total Cost** = AED 19,000

Mathematical Comparison:

- **Simple Average**: $\frac{100 + 200}{2} = \text{AED 150.00 / unit}$ (Incorrect Valuation: $100 \times 150 = \text{AED 15,000}$)
- **Weighted Average (InsAcc Rule)**: $\frac{19,000}{100} = \text{AED 190.00 / unit}$ (Correct Valuation: $100 \times 190 = \text{AED 19,000}$)

---

8. Summary

The Purchase Ledger Module provides granular lot tracking and accurate weighted average cost calculations for physical asset acquisitions. By relying on `computeAverages()` to calculate unit price baselines, InsAcc eliminates valuation errors inherent in simple averaging methods.

---

9. Chapter Appendix

Sample Item Cost Summary Matrix

Item Description	Lot Count	Total Qty	Total Cost Basis	Weighted Avg Unit Price
24K Gold Bar 1kg	3	5 kg	AED 1,425,000.00	AED 285,000.00 / kg
24K Gold Bar 100g	8	400 g	AED 114,000.00	AED 285.00 / g
Silver Bullion 100oz	4	400 oz	AED 44,000.00	AED 110.00 / oz

10. Glossary

- **Lot**: A distinct quantity of a commodity or security specified in a single purchase transaction.
- **Weighted Average Costing**: An inventory/asset valuation method that calculates the average cost of items based on total expenditure divided by total units purchased.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Investment Portfolio Management: Volume 04 Chapter 02
- Double-Entry Voucher Operations: Volume 04 Chapter 07

title: "Volume 04: End-User Manual - Chapter 04: Property and Lease Management" document_id: "INSACC-DOC-V04-CH04" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 04: InsAcc End-User Operations Manual

Chapter 04: Property and Lease Management

> **Single Source of Truth Reference**: All property data models, unit state machines, and lease contract structures defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter provides operational procedures for building real estate hierarchies, managing property categories, registering rentable units, maintaining tenant records, executing lease contracts, and automating PDC cheque schedules in the InsAcc Property Management Module.

2. Scope

This specification covers:

- The 4-level real estate domain hierarchy (Categories $\rightarrow$ Buildings $\rightarrow$ Units $\rightarrow$ Leases).
- Building master setup (`PropertyBuilding`) and property category setup (`PropertyCategory`).
- Rentable unit setup (`PropertyUnit`) and unit state transitions (`Vacant` $\rightarrow$ `Occupied` $\rightarrow$ `Under Maintenance`).
- Tenant master registration (`PropertyTenant`) and Emirates ID / KYC tracking.
- Lease contract creation (`PropertyLeases.tsx`) and automatic PDC schedule generation.

Out of Scope:

- PDC cheque clearing & bounced workflow (covered in Volume 04 Chapter 05).
- Property maintenance payment vouchers (covered in Volume 04 Chapter 07).

3. Audience

This document is authored for:

- Property Operations Managers and Leasing Officers
- Real Estate Portfolio Accountants
- Property Administrators

4. Prerequisites

Before registering leases:

1. Log in to InsAcc and select the **`Property Manager`** profile.
2. Select the **Property Management Module**.
3. Ensure parent buildings and categories are configured in `insacc_prop_buildings` .

5. Warnings & Operational Hazards

> **WARNING: UNIT OCCUPANCY STATE CONFLICT**: Assigning a new lease to a unit currently marked as `Occupied` will throw an operational validation error (`UNIT_ALREADY_OCCUPIED`). Administrators MUST terminate or expire the active lease before attaching a new tenant contract.

6. Notes & Architecture Context

> **NOTE: Automatic PDC Schedule Calculation**: Registering a 1-year lease of AED 120,000 with `Quarterly` payment frequency automatically generates **4 Post-Dated Cheque entries** of AED 30,000 each, spaced exactly 3 months apart starting from the lease commencement date (`propertyPdcService.ts`).

7. Main Content

7.1 Real Estate Hierarchy & Data Model

Real estate operations are structured as a 4-level parent-child hierarchy:

```
Level 1: Property Category ──► Level 2: Building Master ──► Level 3: Unit ──► Level 4: Lease Contract
(Residential / Commercial)    (Al Riyan Tower)            (Unit 101)       (Tenant Lease & PDCs)
```

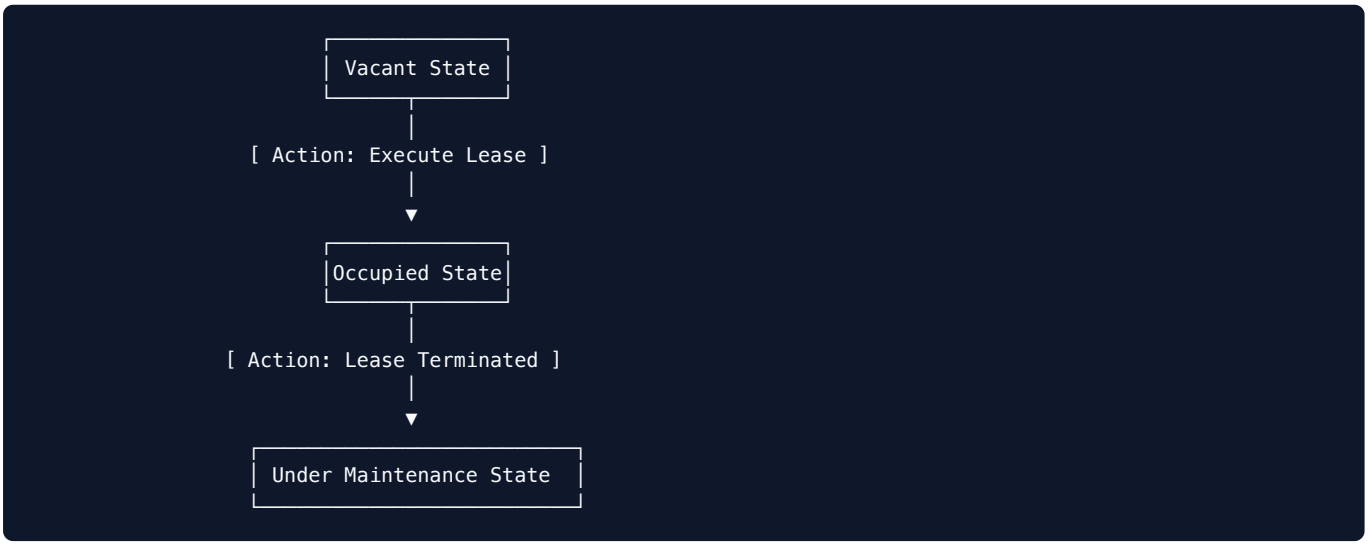
7.2 Property Categories & Building Master Setup

1. **Property Categories** (`insacc_prop_categories`): Defines broad property classes (`Residential` , `Commercial` , `Industrial` , `Retail`).
2. **Building Master Setup** (`PropertyBuilding.tsx`): Establishes building master records:

```
export interface PropertyBuilding {
  id: string              // Unique building ID (e.g. "bld-101")
  name: string            // Building title (e.g. "Al Riyan Tower")
  categoryId: string      // Parent category reference
  address: string         // Physical location / Plot number
  totalUnits: number      // Total unit count
  notes?: string          // Operational memo
}
```

7.3 Rentable Unit Setup & Occupancy State Machine

Units belong to parent buildings and progress through a 3-state occupancy state machine:



Unit Interface Model (`PropertyUnit`):

```
export interface PropertyUnit {
  id: string              // Unique unit ID (e.g. "unit-101")
  buildingId: string      // Parent building ID reference
  unitNumber: string      // Unit number (e.g. "A-101")
  type: string            // "1BHK" | "2BHK" | "3BHK" | "Studio" | "Office" | "Retail"
  annualRent: number      // Base annual target rent (AED)
  status: 'Vacant' | 'Occupied' | 'Under Maintenance'
  tenantId?: string       // Active tenant ID reference
}
```

7.4 Tenant Master Registration & KYC Metadata

Tenant master profiles persist in `insacc_prop_tenants`:

```
export interface PropertyTenant {
  id: string              // Unique tenant ID (e.g. "ten-1001")
  name: string            // Full tenant / corporate entity name
  nationalId: string      // Emirates ID / Passport / Commercial License #
  phone: string           // Contact mobile number
  email: string           // Email address for notifications
  emergencyContact?: string
}
```

7.5 Lease Contract Registration & Automated PDC Generation

```
Open Leases View ⟶ Click [+ New Lease Contract] ⟶ Select Unit & Tenant ⟶ Save Lease
```

Steps to Register a Lease Contract:

1. Navigate to **Properties** $\rightarrow$ **Leases** $\rightarrow$ click **+ New Lease Contract**.
2. Complete lease parameters:
   - **Tenant**: Select registered tenant (e.g., `John Doe`).
   - **Property Unit**: Select a `Vacant` unit (e.g., `Unit 101`).
   - **Start Date**: (e.g., `2026-07-01`).
   - **End Date**: (e.g., `2027-06-30`).
   - **Annual Rent**: (e.g., `120,000 AED`).
   - **Payment Frequency**: Select `Annual` (1 cheque), `Semi-Annual` (2 cheques), `Quarterly` (4 cheques), or `Monthly` (12 cheques).
   - **Security Deposit**: (e.g., `5,000 AED`).
3. Click **Create Lease Contract**.

Automated System Execution:

1. Updates unit status from `Vacant` to `Occupied`.
2. Posts Security Deposit receipt voucher to `2120 Tenant Security Deposits`.
3. Automatically generates the Post-Dated Cheque schedule in `insacc_prop_rent`.

---

8. Summary

The Property Management Module structures real estate assets into a 4-level hierarchy. By managing unit occupancy states, tenant profiles, and lease terms, InsAcc automates security deposit accounting and Post-Dated Cheque schedule generation.

9. Chapter Appendix

Payment Frequency & Cheque Schedule Matrix

Payment Frequency	Cheque Count	Maturity Schedule Interval	Per-Cheque Amount (AED 120,000 Rent)
Annual	1 Cheque	12 Months	AED 120,000.00
Semi-Annual	2 Cheques	6 Months	AED 60,000.00
Quarterly	4 Cheques	3 Months	AED 30,000.00
Monthly	12 Cheques	1 Month	AED 10,000.00

10. Glossary

- **KYC (Know Your Customer)**: The process of verifying the identity of clients or tenants before executing business contracts.
- **PDC (Post-Dated Cheque)**: A cheque written with a future maturity date that cannot be cashed until that specified date arrives.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- PDC and Rent Collection: Volume 04 Chapter 05
- Double-Entry Voucher Operations: Volume 04 Chapter 07

title: "Volume 04: End-User Manual - Chapter 05: PDC and Rent Collection" document_id: "INSACC-DOC-V04-CH05" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 04: InsAcc End-User Operations Manual

Chapter 05: PDC and Rent Collection

> **Single Source of Truth Reference**: All Post-Dated Cheque state machines, collection workflows, and general ledger journal postings defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter provides step-by-step operational procedures for managing Post-Dated Cheques (PDCs), processing bank deposit collections, clearing matured cheques, recording revenue recognition journal entries, and handling bounced cheque replacements in the InsAcc Property Management Module.

2. Scope

This specification covers:

- The 5-state PDC lifecycle (`Received` $\rightarrow$ `Deposited` $\rightarrow$ `Cleared` / `Bounced` $\rightarrow$ `Replaced` / `Cancelled`).
- PDC Manager dashboard interface (`PropertyPdcManager.tsx` and `propertyPdcService.ts`).
- Bank deposit slip compilation and collection tracking.
- Automated double-entry journal postings upon cheque clearance.
- Bounced cheque accounting reversals to Rent Receivable (`1130`) and replacement workflows.

Out of Scope:

- Lease contract registration (covered in Volume 04 Chapter 04).
- Bank account reconciliation matching (covered in Volume 04 Chapter 06).

3. Audience

This document is authored for:

- Rent Collection & Accounts Receivable Officers
- Property Managers and Leasing Operations Technicians
- Financial Accountants and General Ledger Staff

4. Prerequisites

Before processing PDC collections:

1. Log in to InsAcc with `Property Manager` or `Accounts` profile access.
2. Confirm active lease contracts have generated PDC records in `insacc_prop_rent` .

5. Warnings & Operational Hazards

> **WARNING: PREMATURE CHEQUE CLEARANCE HAZARD**: Marking a PDC cheque status as `Cleared` prior to actual bank clearance confirmation inflates bank cash balances and recognizes earned rental revenue prematurely. Operators MUST verify bank deposit statement clearance before triggering the `Cleared` status transition.

6. Notes & Architecture Context

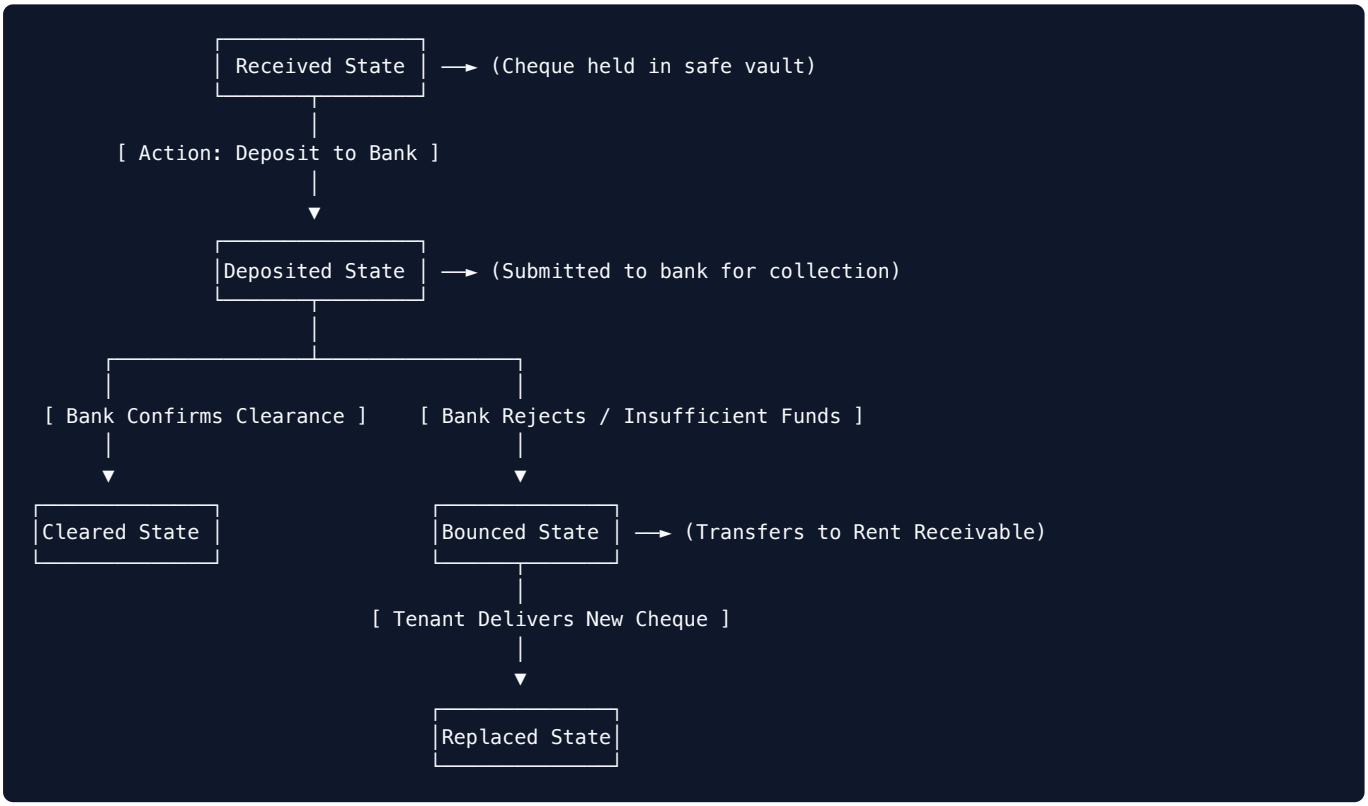
> **NOTE: Automated Revenue Conversion**: Clearing a PDC cheque executes a dual entry: it transfers funds from `1410 PDC Held` to `1120 Bank Account` AND simultaneously converts deferred `2110 Unearned Rent` into earned `4120 Rental Revenue`.

7. Main Content

7.1 Post-Dated Cheque (PDC) Operational Overview

In UAE and regional real estate markets, annual rent is collected via post-dated cheques written at lease signing. InsAcc tracks every cheque from initial vault receipt through bank clearance.

7.2 The 5-State PDC Lifecycle State Machine (`PropertyPdcManager.tsx`)



7.3 Deposit Slip Generation & Bank Collection Submission

1. Open **Property** $\rightarrow$ **PDC Manager**.
2. Filter cheques by **Maturity Status** $\rightarrow$ select **Maturing This Week**.
3. Select the cheques maturing on or before the current date.
4. Click **Generate Bank Deposit Slip**.
5. Select target receiving bank (e.g., `1120.001 Emirates Islamic Bank`).
6. Click **Confirm Deposit**. Cheque status transitions from `Received` to `Deposited`.

7.4 Clearing Cheques & Revenue Recognition Journal Entries

When the bank confirms clearance, transition the cheque in **PDC Manager**:

```
Select Deposited Cheque ⟶ Click [Mark as Cleared] ⟶ System Generates Double Entry
```

Automated Journal Entry Generated on Clearance:

1. **Asset Transfer**:
   - $\text{Debit: 1120.001 Emirates Islamic Bank (Asset)} = \text{AED 30,000}$
   - $\text{Credit: 1410 PDC Cheques Held (Asset)} = \text{AED 30,000}$
2. **Revenue Recognition**:
   - $\text{Debit: 2110 Unearned Rent Liability (Liability)} = \text{AED 30,000}$
   - $\text{Credit: 4120 Rental Revenue (Revenue)} = \text{AED 30,000}$

7.5 Bounced Cheque Handling, Replacements & Legal Recourse

If a deposited cheque is rejected by the bank due to insufficient funds:

1. Open **PDC Manager** $\rightarrow$ locate the cheque in `Deposited` status.
2. Click **Mark as Bounced**. Enter bank bounce date and penalty fee if applicable.
3. **Automated Accounting Adjustment**:
   - $\text{Debit: 1130 Tenant Rent Receivable (Asset)} = \text{Cheque Amount}$
   - $\text{Credit: 1410 PDC Cheques Held (Asset)} = \text{Cheque Amount}$
4. **Replacement Workflow**:
   - Contact the tenant to secure a replacement cheque or manager's cheque.
   - Click **Record Replacement Cheque**. Enter new cheque number and maturity date.
   - The status updates to `Replaced`, and the new cheque enters the `Received` state.

---

8. Summary

The PDC Manager subsystem provides complete lifecycle tracking for post-dated cheques. By enforcing state machine transitions, automating bank deposit slips, and executing double-entry postings upon clearance or bounce, InsAcc maintains complete audit control over rent collections.

---

9. Chapter Appendix

Cheque Lifecycle State & Accounting Matrix

Lifecycle State	Balance Sheet / Income Statement Impact	Target Account Codes Involved
Received	Asset: PDC Held / Liability: Unearned Rent	Debit `1410` / Credit `2110`
Deposited	Internal tracking change (In Transit)	No net ledger balance change
Cleared	Cash increases / Revenue recognized	Debit `1120`, Credit `1410` & Debit `2110`, Credit `4120`
Bounced	Transfers asset from PDC to Receivable	Debit `1130` / Credit `1410`
Replaced	Replaces bounced cheque with new PDC	Debit `1410` / Credit `1130`

10. Glossary

- **Deferred Revenue (Unearned Rent)**: Payment received from a tenant for future rental periods that has not yet been earned.
- **Deposit Slip**: An itemized slip showing the date, bank account number, and cheque items deposited into a bank account.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Property & Lease Management: Volume 04 Chapter 04
- Banking & Reconciliation: Volume 04 Chapter 06
- Accounting Engine Specs: Volume 06 Chapter 02

title: "Volume 04: End-User Manual - Chapter 06: Banking and Reconciliation" document_id: "INSACC-DOC-V04-CH06" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 04: InsAcc End-User Operations Manual

Chapter 06: Banking and Reconciliation

> **Single Source of Truth Reference**: All bank account structures, inter-account transfer rules, and reconciliation matching algorithms defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter provides operational procedures for managing bank accounts, processing inter-account fund transfers via Contra Vouchers (`CV`), importing bank statement files, executing automated statement reconciliation matching, and resolving unreconciled items in InsAcc.

2. Scope

This specification covers:

- Bank accounts dashboard and account setup (`BankReconciliationDashboard.tsx`).
- Inter-account fund transfers via Contra Vouchers (`bankTransactionService.ts`).
- Importing electronic bank statements (CSV / Excel).
- Automated transaction matching rules (date tolerance, exact amount, check number).
- Unreconciled transaction resolution and Bank Reconciliation Summary reports.

Out of Scope:

- General voucher entry for non-banking accounts (covered in Volume 04 Chapter 07).
- REST API open banking integrations `[To Be Implemented]` (covered in Volume 05 Chapter 03).

3. Audience

This document is authored for:

- Treasury Officers and Cash Management Technicians
- Financial Accountants and Bookkeepers
- Internal Audit Specialists

4. Prerequisites

Before performing bank reconciliation:

1. Log in to InsAcc with `Admin` or `Accounts` role privileges.
2. Download an electronic bank statement file (`.csv` or `.xlsx`) from your online banking portal.

5. Warnings & Operational Hazards

> **WARNING: UNMATCHED BANK TRANSACTION DRIFT**: Failing to reconcile bank statement lines on a monthly basis allows bank service fees, unrecorded interest deposits, or unauthorized withdrawals to accumulate as unreconciled variance. Cash accounts MUST be reconciled prior to fiscal period closing.

6. Notes & Architecture Context

> **NOTE: Contra Voucher Restriction**: Inter-account bank transfers execute via Contra Vouchers (`CV`). The double-entry engine validates that BOTH debit and credit lines belong strictly to `1110 Cash` or `1120 Bank` sub-accounts (`postingValidator.ts`).

7. Main Content

7.1 Bank Accounts Dashboard Overview (`BankReconciliationDashboard.tsx`)

The **Banking** console lists all configured corporate bank accounts:

```
Bank Accounts Summary Console
┌──────────────────────────┬──────────────────┬──────────────────┬──────────────────┐
│ Bank Account Title       │ Account Number   │ GL Account Code  │ Ledger Balance   │
├──────────────────────────┼──────────────────┼──────────────────┼──────────────────┤
│ Emirates Islamic Bank    │ 123-456789-01    │ 1120.001         │ AED 850,000.00   │
│ ADCB Operations Account  │ 987-654321-02    │ 1120.002         │ AED 320,000.00   │
│ Petty Cash Vault         │ N/A              │ 1110.001         │ AED 15,000.00    │
└──────────────────────────┴──────────────────┴──────────────────┴──────────────────┘
```

7.2 Inter-Account Transfers & Contra Vouchers (`bankTransactionService.ts`)

To transfer funds between bank accounts (e.g., from ADCB to Emirates Islamic):

```
Open Banking Console ⟶ Click [Transfer Funds] ⟶ Select Source & Destination ⟶ Submit CV
```

1. Navigate to **Banking** $\rightarrow$ click **Transfer Funds**.
2. Select **Source Account** (e.g., `1120.002 ADCB Account`).
3. Select **Destination Account** (e.g., `1120.001 Emirates Islamic Bank`).
4. Enter **Transfer Amount** (e.g., `50,000 AED`) and **Transfer Date**.
5. Click **Execute Transfer**.

Double-Entry Journal Posted:

- $\text{Debit: 1120.001 Emirates Islamic Bank (Asset)} = \text{AED 50,000}$
- $\text{Credit: 1120.002 ADCB Operations Account (Asset)} = \text{AED 50,000}$

7.3 Bank Statement Import File Formats (CSV / Excel)

InsAcc imports electronic bank statement files matching the standard 4-column structure:

Date (`YYYY-MM-DD`)	Description / Reference	Cheque / Ref #	Amount (`+` Credit / `−` Debit)
`2026-06-15`	Rent Payment Unit 101	`CHQ-1001`	`+30000.00`
`2026-06-18`	Maintenance Payment	`FT-2026-88`	`-3500.00`
`2026-06-20`	Monthly Bank Service Charge	`FEE-99`	`-150.00`

7.4 Automated Statement Reconciliation Matching Algorithm

Upon uploading a bank statement file, the reconciliation engine (`bankTransactionService.ts`) compares imported statement lines against posted General Ledger vouchers:

```
Imported Statement Line ──► Matching Engine Rule Check ──► Status Assignment
                            ├── Rule 1: Exact Date + Exact Amount + Ref # Match ──► [Matched Auto]
                            ├── Rule 2: Date ± 3 Days + Exact Amount Match      ──► [Suggested Match]
                            └── Rule 3: No Ledger Match Found                   ──► [Unreconciled]
```

7.5 Unreconciled Items Resolution & Bank Reconciliation Summary

For items marked `Unreconciled` :

1. **Unrecorded Bank Fee / Charge**: Click **Create Payment Voucher (`PV`)** directly from the statement line to record the expense (e.g., `5110 Bank Fees`).
2. **Unrecorded Deposit / Interest**: Click **Create Receipt Voucher (`RV`)** to record incoming interest or deposit.
3. **Timing Difference**: Outstandings cheques or deposits in transit remain open until clearing in the subsequent statement period.

Bank Reconciliation Summary Report:

$$\text{Ending Bank Statement Balance} + \text{Deposits in Transit} - \text{Outstanding Cheques} = \text{GL Ledger Balance}$$

---

8. Summary

The Banking and Reconciliation Module simplifies cash management through inter-account Contra Transfers and automated bank statement matching algorithms. By resolving timing differences and recording bank charges directly, InsAcc ensures general ledger cash balances match physical bank statements.

---

9. Chapter Appendix

Standard Bank Reconciliation Summary Template

Line Item Description	Amount (AED)	Reconciliation Status
Ending Balance per Bank Statement (as of 2026-06-30)	**AED 845,150.00**	Verified Statement
Add: Deposits in Transit (PDC Cleared on 30th, posted on 1st)	+ AED 30,000.00	Timing Difference
Less: Outstanding Cheques (Uncashed Vendor Payment Vouchers)	- AED 25,150.00	Timing Difference
Adjusted Bank Balance	**AED 850,000.00**	Reconciled
General Ledger Account Balance (`1120.001`)	**AED 850,000.00**	**✓ Fully Balanced**

---

10. Glossary

- **Bank Reconciliation**: The process of matching the balances in an entity's accounting records for a cash account to the corresponding information on a bank statement.

- **Contra Voucher (CV)**: A journal entry recording an internal transfer of money between two cash or bank accounts.
- **Deposits in Transit**: Cash or cheques received and recorded by an entity, but not yet recorded by the bank.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- PDC and Rent Collection: Volume 04 Chapter 05
- Double-Entry Voucher Operations: Volume 04 Chapter 07
- Financial Reports: Volume 04 Chapter 08

title: "Volume 04: End-User Manual - Chapter 07: Double Entry Voucher Operations" document_id: "INSACC-DOC-V04-CH07" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 04: InsAcc End-User Operations Manual

Chapter 07: Double Entry Voucher Operations

> **Single Source of Truth Reference**: All double-entry voucher lifecycles, validation rules, and general ledger posting algorithms defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter provides operational procedures for creating, validating, approving, posting, inspecting, and reversing double-entry accounting vouchers in the InsAcc platform.

2. Scope

This specification covers:

- Voucher creation interfaces (`InvestmentJournalVoucher.tsx` and `PropertyJournalVoucher.tsx`).
- Voucher types: Receipt Voucher (`RV`), Payment Voucher (`PV`), Journal Voucher (`JV`), Contra Voucher (`CV`).
- Multi-line voucher line entry, account selection, and debit/credit validation.
- The 5-stage voucher lifecycle (`VoucherLifecycleActions.tsx`).
- Reversal voucher generation via `reverseVoucher()` .

Out of Scope:

- Chart of Accounts setup (covered in Volume 03 Chapter 03).
- Financial statement reporting (covered in Volume 04 Chapter 08).

3. Audience

This document is authored for:

- Senior Accountants and General Ledger Specialists
- Accounts Payable / Receivable Technicians
- Financial Auditors and Accounting Supervisors

4. Prerequisites

Before creating vouchers:

1. Log in to InsAcc with `Admin` or `Accounts` role privileges.
2. Confirm the active period is not locked (Volume 03 Chapter 04).

5. Warnings & Operational Hazards

> **WARNING: IMMUTABILITY OF POSTED VOUCHERS**: Once a voucher transitions to `Posted` status, its lines, amounts, accounts, and economic dates are permanently locked. Posted vouchers CANNOT be deleted or edited directly. Corrections MUST be executed by generating a Reversal Voucher (`reverseVoucher()`).

6. Notes & Architecture Context

> **NOTE: Strict Floating-Point Balance Tolerance**: The voucher validation engine enforces mathematical debit/credit balance equality using a strict floating-point tolerance threshold: $\left| \sum \text{Debits} - \sum \text{Credits} \right| <

> 0.001$ (`postingValidator.ts`).

7. Main Content

7.1 Double-Entry Accounting Engine Overview

Every financial transaction in InsAcc is recorded as a **Double-Entry Voucher**. Every voucher requires at least two line items such that total debits equal total credits:

$$\sum \text{Debit Lines} = \sum \text{Credit Lines}$$

7.2 Voucher Types & Functional Use Cases (`RV` , `PV` , `JV` , `CV`)

Voucher Code	Voucher Title	Primary Operational Use Case	Mandatory Account Rules
RV	**Receipt Voucher**	Incoming funds, rent collection, dividends	At least one Debit to Cash (`1110`) or Bank (`1120`)
PV	**Payment Voucher**	Outgoing funds, vendor bills, repairs	At least one Credit to Cash (`1110`) or Bank (`1120`)
JV	**Journal Voucher**	Non-cash adjustments, depreciation, closing	Arbitrary General Ledger accounts
CV	**Contra Voucher**	Cash/Bank inter-account transfers	All lines restricted to Cash (`1110`) or Bank (`1120`)

7.3 Creating & Form Validation of Multi-Line Vouchers

```
Open Transactions View ⟶ Click [+ New Voucher] ⟶ Select Type & Add Lines ⟶ Save Draft
```

Step-by-Step Creation:

1. Navigate to **Transactions** $\rightarrow$ click **+ New Voucher**.
2. Select **Voucher Type** (`Receipt` , `Payment` , or `Journal`).
3. Set **Voucher Date** (e.g., `2026-06-15`) and enter **Narration / Memo**.
4. Add Line Items:
   - Line 1: Select `1120.001 Emirates Islamic Bank` $\rightarrow$ Select `Debit` $\rightarrow$ Enter `10,000.00 AED` .
   - Line 2: Select `4120 Rental Revenue` $\rightarrow$ Select `Credit` $\rightarrow$ Enter `10,000.00 AED` .
5. The form validates balance equality ($\sum D = \sum C$).
6. Click **Save as Draft** or **Submit for Approval**.

7.4 Voucher State Machine Workflow (`VoucherLifecycleActions.tsx`)

Vouchers progress through a formal 5-stage lifecycle state machine:

```
[ Draft ] ──► [ Approved ] ──► [ Posted ] ──► (Immutable Ledger Entry)
                                    │
                                    └──► [ Reversed ] (Via Reversal Voucher)
```

1. **`Draft`**: Newly created voucher. Lines can be edited freely. Does not affect ledger balances.
2. **`Approved`**: Verified by accounting supervisor. Pending ledger posting.
3. **`Posted`**: Committed to General Ledger (`ledgerService.ts`). Modifies derived account balances. Immutable.
4. **`Cancelled`**: Voided prior to posting. Preserved for audit sequence continuity.
5. **`Reversed`**: Offset by a corresponding Reversal Voucher.

7.5 Voucher Reversal & Re-posting Procedures (`reverseVoucher()`)

To correct an erroneous posted voucher:

1. Open **Transactions** $\rightarrow$ locate the posted voucher.
2. Click **Reverse Voucher** (`VoucherDetailsModal.tsx`).
3. Enter **Reversal Reason** (e.g., `"Correction of duplicate entry"`).
4. Click **Confirm Reversal**.

System Reversal Mechanics (`voucherService.ts`):

1. Creates a new **Reversal Voucher** (e.g., `REV-RV-2026-0042`) with inverted debit/credit lines.
2. Posts the Reversal Voucher immediately to the General Ledger.
3. Updates the original voucher status to `Reversed`.

---

8. Summary

Double-entry vouchers form the core accounting foundation of InsAcc. By enforcing debit-credit balance equality ($\sum D = \sum C$), maintaining a 5-stage lifecycle state machine, and providing automated reversal workflows, InsAcc guarantees general ledger integrity.

---

9. Chapter Appendix

Voucher Lifecycle Action Matrix

Lifecycle Transition	Target Status	Available User Roles	Ledger Balance Impact
Create Voucher	`Draft`	`Admin`, `Accounts`	None (Unposted)
Approve Voucher	`Approved`	`Admin`, `Accounts`	None (Unposted)
Post Voucher	`Posted`	`Admin`, `Accounts`	Updates Derived Balances
Cancel Draft	`Cancelled`	`Admin`, `Accounts`	None
Reverse Posted	`Reversed`	`Admin` Only	Posts Offsetting Reversal

---

10. Glossary

- **Double-Entry Accounting**: A system of bookkeeping where every entry to an account requires a corresponding and opposite entry to a different account.
- **Immutability**: The property of data that prevents it from being modified after creation.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Chart of Accounts: Volume 03 Chapter 03
- Financial Reports: Volume 04 Chapter 08
- Double-Entry Engine Specs: Volume 06 Chapter 02

title: "Volume 04: End-User Manual - Chapter 08: Financial Reporting and Exports" document_id: "INSACC-DOC-V04-CH08" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 04: InsAcc End-User Operations Manual

Chapter 08: Financial Reporting and Exports

> **Single Source of Truth Reference**: All report projection models, mathematical formulas, and export IPC bridges defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter details the financial reporting engine in InsAcc. It provides instructions for generating Trial Balance trees, Profit & Loss statements, Balance Sheets, General Ledger sub-ledgers, and exporting reports to CSV, PDF, and Excel via the Electron IPC file bridge.

2. Scope

This specification covers:

- The catalog of 19 financial reporting views (`InvestmentReports.tsx` , `PropertyReports.tsx` , `reportService.ts`).
- Trial Balance calculation and tree rendering (`TrialBalanceTree.tsx`).
- Profit & Loss statement formulas ($\text{Net Profit} = \text{Revenue} - \text{Expenses}$).
- Balance Sheet equation proof ($\text{Assets} = \text{Liabilities} + \text{Equity} + \text{Net Profit}$).
- Exporting reports to filesystem files (`window.api.saveFile()`).

Out of Scope:

- Core `localStorage` read models (covered in Volume 02 Chapter 01).
- Developer projection model code architecture (covered in Volume 06 Chapter 03).

3. Audience

This document is authored for:

- Chief Financial Officers and Financial Controllers
- Senior Accounting Managers and Auditors
- Tax Consultants and Financial Analysts

4. Prerequisites

Before generating reports:

1. Log in to InsAcc with `Admin` or `Accounts` role access.
2. Verify that all current period vouchers have been posted (Volume 04 Chapter 07).

5. Warnings & Operational Hazards

> **WARNING: UNPOSTED VOUCHER EXCLUSION**: Financial reports project figures **ONLY from Posted vouchers** (`status === 'Posted'`). Vouchers in `Draft` or `Approved` status are excluded from Trial Balance, Profit & Loss, and Balance Sheet calculations.

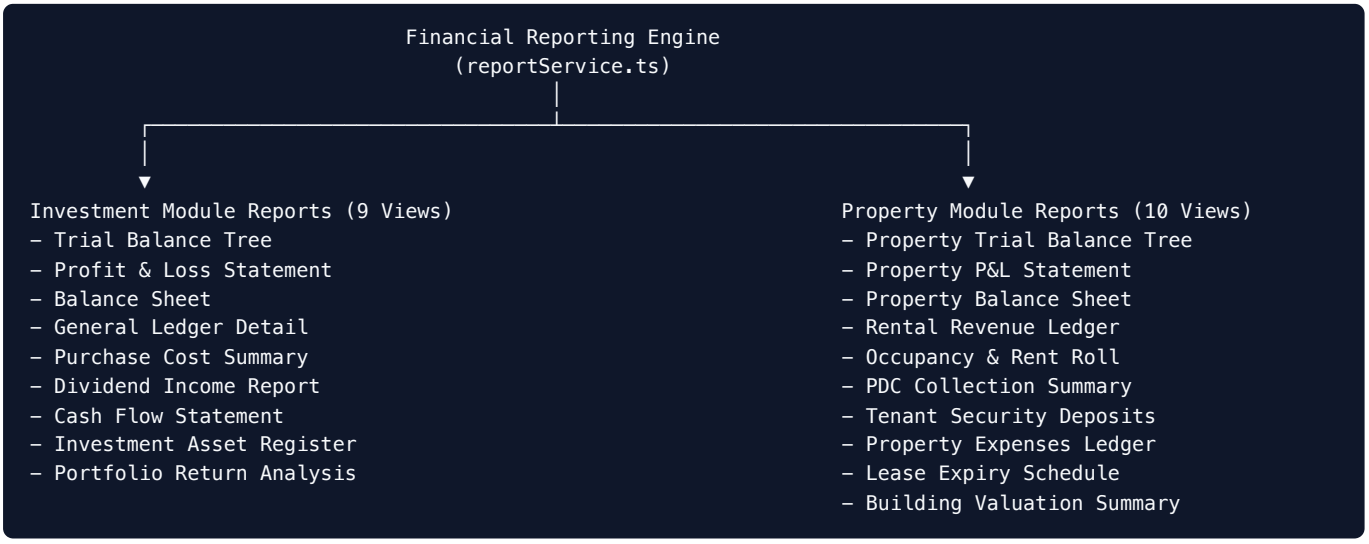
6. Notes & Architecture Context

> **NOTE: Zero File Size Overhead**: Reports are generated dynamically in client memory. Exporting a report converts the rendered React data table into a formatted string (CSV or Base64 PDF) and transmits it to the desktop main process without storing temporary files in browser memory.

7. Main Content

7.1 Financial Reporting Architecture & Catalog (19 Views)

InsAcc features **19 specialized financial reporting views** accessible under **Reports**:



7.2 Trial Balance Tree Projection (`TrialBalanceTree.tsx`)

The **Trial Balance** aggregates ending debit and credit balances across all active accounts:

$$\sum \text{Debit Balances} = \sum \text{Credit Balances}$$

Trial Balance Verification Check:

- **Debit Total**: `AED 1,275,000.00`
- **Credit Total**: `AED 1,275,000.00`
- **Variance**: `AED 0.00` (✓ **Balanced**)

7.3 Profit & Loss Statement Calculation Engine

The **Profit & Loss (P&L) Statement** summarizes operational performance over a target date range:

$$\text{Net Profit / (Loss)} = \sum \text{Revenue Accounts (4000-4999)} - \sum \text{Expense Accounts (5000-5999)}$$

Example P&L Summary Breakdown:

- **Property Rental Revenue (`4120`)**: `AED 150,000.00`
- **Dividend Income (`4110`)**: `AED 15,000.00`
- **Total Revenue**: **`AED 165,000.00`**
- *Less:* **Property Maintenance Expense (`5110`)**: `AED 35,000.00`

- *Less:* **Depreciation Expense (`5190`)**: `AED 10,000.00`
- **Total Expenses**: **`AED 45,000.00`**
- **Net Profit**: **`AED 120,000.00`**

7.4 Balance Sheet Calculation Engine & Equality Proof

The **Balance Sheet** projects financial standing as of a specific date, enforcing the fundamental accounting equation:

$$\text{Total Assets} = \text{Total Liabilities} + \text{Total Equity} + \text{Net Profit}$$

Example Balance Sheet Projection:

- **Total Assets (`1000–1999`)**: `AED 1,270,000.00`
  - Cash & Bank (`1110/1120`): `AED 850,000.00`
  - Fixed Assets & Investments (`1200`): `AED 420,000.00`
- **Total Liabilities (`2000–2999`)**: `AED 50,000.00`
  - Deferred Unearned Rent (`2110`): `AED 45,000.00`
  - Security Deposits (`2120`): `AED 5,000.00`
- **Total Equity (`3000–3999`)**: `AED 1,100,000.00`
  - Owner's Capital (`2200`): `AED 1,100,000.00`
- **Retained Net Profit**: `AED 120,000.00`
- **Liabilities + Equity + Net Profit**: `50,000 + 1,100,000 + 120,000 =` **`AED 1,270,000.00`** (✓ **Balanced**)

7.5 Desktop File Export Workflows (CSV / PDF / Excel via `window.api.saveFile`)

To export any report view to your local computer:

```
Open Target Report ⟶ Click [Export CSV] / [Export PDF] ⟶ Desktop OS Save Dialog
```

1. Open the target report (e.g., **Balance Sheet**).
2. Click **Export CSV**, **Export PDF**, or **Export Excel** on the top toolbar.
3. The renderer compiles the report table into a string payload and calls:

   ```
   const filePath = await window.api.saveFile('Balance_Sheet_2026-06-30.csv', csvData)
   ```

4. Electron writes the file to your Downloads folder and returns the absolute disk filepath.

---

8. Summary

InsAcc provides 19 financial reporting views powered by dynamic read-model projections. By enforcing double-entry mathematical equality across Trial Balance, Profit & Loss, and Balance Sheet statements and providing desktop file export capability (`window.api.saveFile`), InsAcc delivers enterprise-grade financial transparency.

---

9. Chapter Appendix

Financial Statement Export Format Support Matrix

Report Title	Screen Render Component	CSV Export Support	PDF Export Support	Excel Export Support
Trial Balance Tree	`TrialBalanceTree.tsx`	✓ Supported	✓ Supported	✓ Supported
Profit & Loss	`InvestmentReports.tsx`	✓ Supported	✓ Supported	✓ Supported
Balance Sheet	`InvestmentReports.tsx`	✓ Supported	✓ Supported	✓ Supported
General Ledger	`PropertyReports.tsx`	✓ Supported	✓ Supported	✓ Supported
Rent Roll Schedule	`PropertyReports.tsx`	✓ Supported	✓ Supported	✓ Supported

10. Glossary

- **Balance Sheet**: A financial statement that summarizes a company's assets, liabilities, and shareholders' equity at a specific point in time.
- **Profit & Loss (P&L) Statement**: A financial statement that summarizes the revenues, costs, and expenses incurred during a specified period.
- **Trial Balance**: A bookkeeping worksheet in which the balances of all ledgers are compiled into debit and credit account column totals.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Chart of Accounts: Volume 03 Chapter 03
- Double-Entry Voucher Operations: Volume 04 Chapter 07
- Read Models & Formatters: Volume 06 Chapter 03

title: "Volume 05: Interface and Integration Spec - Chapter 01: Desktop IPC Bridge" document_id: "INSACC-DOC-V05-CH01" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 05: Interface and Integration Specification

Chapter 01: Desktop IPC Bridge

> **Single Source of Truth Reference**: All Electron Inter-Process Communication (IPC) patterns, preload bridge APIs, and main process event handlers defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter details the desktop Inter-Process Communication (IPC) bridge architecture in InsAcc v1.0.0. It defines the secure context isolation boundary between the React renderer process and the Electron Node.js main process, detailing the `save-file` IPC handler and TypeScript context bridge definitions.

2. Scope

This specification covers:

- Electron IPC architecture connecting renderer and main process contexts.
- Security isolation parameters (`contextIsolation: true` , `nodeIntegration: false`).
- Preload context bridge implementation (`src/main/preload.js`).
- Main process handler execution (`src/main/main.js`).
- TypeScript global window interface extensions (`window.api.saveFile()`).

Out of Scope:

- General file import/export UI workflows (covered in Volume 05 Chapter 02).
- Target REST API HTTP endpoints `[To Be Implemented]` (covered in Volume 05 Chapter 03).

3. Audience

This document is authored for:

- Desktop Application Engineers and Electron Maintainers
- Application Security Officers and Code Auditors
- Frontend Integration Developers

4. Prerequisites

Before reviewing IPC bridge code:

1. Review the Electron architecture defined in MASTER_ARCHITECTURE.md#5-electron-architecture.
2. Understand Electron `ipcRenderer.invoke` and `ipcMain.handle` asynchronous message-passing patterns.

5. Warnings & Operational Hazards

> **WARNING: CONTEXT ISOLATION COMPLIANCE**: Never set `nodeIntegration: true` or `contextIsolation: false` in `BrowserWindow` settings. Disabling context isolation exposes Node.js file system capabilities directly to web content, creating critical Remote Code Execution (RCE) vulnerabilities.

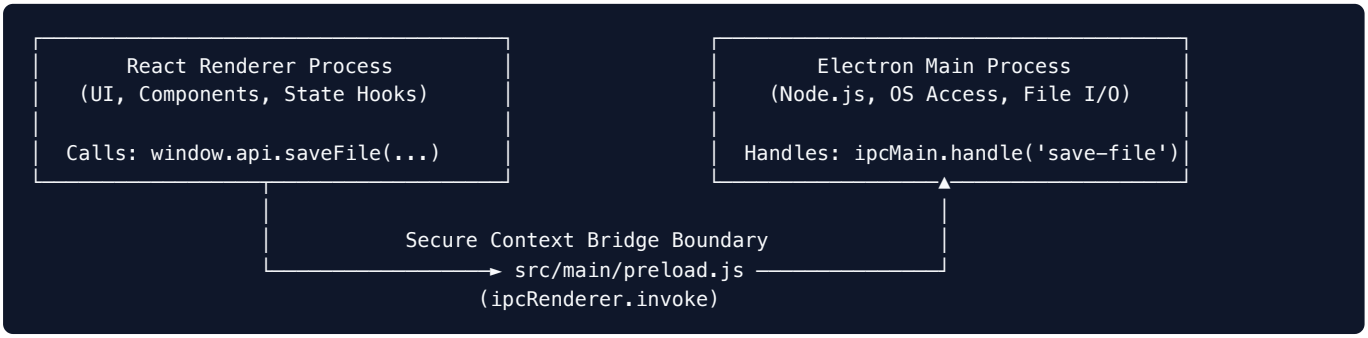
6. Notes & Architecture Context

> **NOTE: Minimal Surface Area Security**: InsAcc intentionally exposes a single, strictly typed IPC method (`saveFile`) through the context bridge. Exposing generic `ipcRenderer.send` methods is prohibited to prevent malicious code injection.

7. Main Content

7.1 Inter-Process Communication (IPC) Architecture

The InsAcc desktop application partitions responsibilities across two process contexts:



7.2 Process Security & Isolation Configuration (`main.js`)

In `src/main/main.js` , `BrowserWindow` enforces strict security boundaries:

```
const mainWindow = new BrowserWindow({
  width: 1400,
  height: 900,
  webPreferences: {
    preload: path.join(__dirname, 'preload.js'),
    contextIsolation: true,       // Enforces strict V8 context separation
    nodeIntegration: false,        // Disables Node.js access in renderer context
    enableRemoteModule: false,     // Disables remote module
    sandbox: true                  // Enables OS-level sandbox isolation
  }
})
```

7.3 Preload Context Bridge Specification (`preload.js`)

The preload script (`src/main/preload.js`) selectively exposes main process capabilities to the renderer via `contextBridge.exposeInMainWorld` :

```
const { contextBridge, ipcRenderer } = require('electron')

contextBridge.exposeInMainWorld('api', {
  saveFile: async (filename, content) => {
    // Transmit request to main process via 'save-file' channel
    return await ipcRenderer.invoke('save-file', { filename, content })
  }
})
```

TypeScript Window Declaration (`src/renderer/vite-env.d.ts`):

```
declare global {
  interface Window {
    api: {
      saveFile: (filename: string, content: string) => Promise<string>
    }
  }
}
```

7.4 Main Process Handler Implementation (`save-file`)

In `src/main/main.js`, the main process registers an asynchronous IPC handler using `ipcMain.handle`:

```
const { app, ipcMain, dialog } = require('electron')
const fs = require('fs/promises')
const path = require('path')

ipcMain.handle('save-file', async (event, { filename, content }) => {
  try {
    // 1. Resolve default output path in user's Downloads directory
    const defaultPath = path.join(app.getPath('downloads'), filename)

    // 2. Open native OS Save Dialog
    const { filePath, canceled } = await dialog.showSaveDialog({
      defaultPath,
      title: 'Save InsAcc Export File',
      buttonLabel: 'Save File'
    })

    if (canceled || !filePath) {
      return null // Operation canceled by user
    }

    // 3. Write content payload to target filesystem path
    await fs.writeFile(filePath, content, 'utf-8')
    console.log(`[IPC] Successfully wrote file to: ${filePath}`)

    return filePath
  } catch (error) {
    console.error(`[IPC Error] Failed to write file:`, error)
    throw new Error(`Failed to write file to disk: ${error.message}`)
  }
})
```

7.5 Renderer Invocation & Error Handling (`window.api.saveFile`)

Renderer UI components invoke the file export channel within an `async/await` block:

```
// Invocation Example in Financial Reports UI (Reports.tsx)
const handleExportCSV = async () => {
  try {
    const csvContent = generateCSVReport(reportData)
    const savedPath = await window.api.saveFile('Profit_Loss_2026.csv', csvContent)

    if (savedPath) {
      alert(`Report exported successfully to:\n${savedPath}`)
    }
  } catch (error) {
    console.error('File export failed:', error)
    alert(`File export failed: ${error.message}`)
  }
}
```

8. Summary

The desktop IPC bridge provides secure, asynchronous file writing capabilities for the InsAcc renderer process. By pairing `contextIsolation: true` with a typed context bridge, InsAcc achieves OS-level file access while maintaining desktop security boundaries.

9. Chapter Appendix

IPC Security & Performance Checklist

Security Parameter	Enforced Setting	Functional Rationale
`contextIsolation`	`true`	Prevents prototype pollution and renderer context escalation
`nodeIntegration`	`false`	Prevents renderer JavaScript from executing arbitrary Node.js APIs
`sandbox`	`true`	Restricts system access via OS-level sandbox container
IPC Method Scope	Explicit (`saveFile`)	Prevents wildcard channel exposure (`ipcRenderer.send('*')`)

10. Glossary

- **Context Bridge**: An Electron API that allows developers to safely create a secure, privilege-separated channel between preload scripts and renderer scripts.
- **Inter-Process Communication (IPC)**: A mechanism that allows processes in a multi-process software system to communicate and synchronize their actions.
- **Preload Script**: A script that runs before renderer content is loaded in the web page, with access to both Node.js APIs and DOM APIs.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Import and Export Interfaces: Volume 05 Chapter 02
- Target REST API: Volume 05 Chapter 03

- Client Security & Isolation: Volume 08 Chapter 01

title: "Volume 05: Interface and Integration Spec - Chapter 02: Import and Export Interfaces" document_id: "INSACC-DOC-V05-CH02" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 05: Interface and Integration Specification

Chapter 02: Import and Export Interfaces

> **Single Source of Truth Reference**: All state snapshot formats, CSV export generators, and import parsers defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter defines the file format specifications, data translation pipelines, and validation rules for importing and exporting data within InsAcc v1.0.0. It covers full-state JSON backups, financial report CSV generation, bank statement parsing, and document attachment storage.

2. Scope

This specification covers:

- Full system backup/restore JSON structure (`state_backup_YYYY-MM-DD.json`).
- Financial statement CSV formatting routines (`reportService.ts`).
- Electronic bank statement file import parser (CSV / Excel format).
- Document attachment management (`insacc_documents`).
- Data schema validation and sanitization.

Out of Scope:

- Desktop Electron IPC bridge implementation (covered in Volume 05 Chapter 01).
- Target REST API HTTP endpoints `[To Be Implemented]` (covered in Volume 05 Chapter 03).

3. Audience

This document is authored for:

- Systems Administrators and Integration Developers
- Data Migration Engineers and Technical Support Staff
- Financial Compliance & Audit Personnel

4. Prerequisites

Before importing or exporting data:

1. Confirm local file write permissions for target output directories.
2. Review storage key definitions in Volume 02 Chapter 01.

5. Warnings & Operational Hazards

> **WARNING: OVERWRITE RISK ON JSON STATE RESTORE**: Restoring a JSON state backup snapshot (`state_backup_YYYY-MM-DD.json`) completely replaces all existing operational ledgers, properties, and vouchers in `localStorage`. System administrators MUST perform a current backup export before executing a state restore.

6. Notes & Architecture Context

> **NOTE: Base64 Attachment Storage Limit**: Document attachments uploaded in `insacc_documents` are encoded as Base64 strings. To prevent exceeding browser `localStorage` quotas (~5MB–10MB), file uploads should be constrained to small PDF/image receipts (< 500 KB per file).

7. Main Content

7.1 System State JSON Backup & Restore Specification

InsAcc exports the complete application state into a single JSON snapshot file (`state_backup_YYYY-MM-DD.json`) via **Settings** $\rightarrow$ **Data Management**:

```
{
  "version": "8",
  "exportedAt": "2026-07-22T12:00:00.000Z",
  "data": {
    "insacc_investments": [ ... ],
    "insacc_transactions": [ ... ],
    "insacc_purchase_categories": [ ... ],
    "insacc_purchases": [ ... ],
    "insacc_prop_buildings": [ ... ],
    "insacc_prop_units": [ ... ],
    "insacc_prop_tenants": [ ... ],
    "insacc_prop_rent": [ ... ],
    "insacc_inv_users": [ ... ],
    "insacc_prop_users": [ ... ],
    "insacc_documents": [ ... ],
    "insacc_logs": [ ... ]
  }
}
```

7.2 Financial Report CSV Export Generation (`reportService.ts`)

Financial statement views generate CSV exports by transforming internal data structures into RFC 4180-compliant CSV text:

```
export function exportToCSV(filename: string, headers: string[], rows: (string | number)[][]): string {
  const escapeCell = (val: string | number) => {
    const str = String(val ?? '')
    return `"${str.replace(/"/g, '""')}"`
  }

  const headerLine = headers.map(escapeCell).join(',')
  const dataLines = rows.map(row => row.map(escapeCell).join(','))
  return [headerLine, ...dataLines].join('\r\n')
}
```

7.3 Bank Statement Electronic Import Parser (CSV / XLSX)

InsAcc imports electronic bank statements to power statement reconciliation (`BankReconciliationDashboard.tsx`).

Target Import Schema:

- **Column 1**: Date (`YYYY-MM-DD` or `DD/MM/YYYY`)

- **Column 2**: Description / Transaction Narration
- **Column 3**: Reference Number / Cheque Number
- **Column 4**: Amount (`+` for deposits, `-` for payments)

7.4 Document Attachment Metadata & Base64 Store (`insacc_documents`)

Documents and lease attachments persist in `insacc_documents`:

```
export interface DocItem {
  id: string              // Unique document ID (e.g. "doc-1719500000000")
  title: string           // Document title (e.g. "Tenancy_Contract_Unit_101.pdf")
  category: string        // "Lease" | "Invoice" | "Voucher Receipt" | "KYC"
  uploadDate: string      // ISO upload timestamp
  fileSize: number        // File size in bytes
  mimeType: string        // "application/pdf" | "image/png" | "image/jpeg"
  contentBase64: string   // Base64 encoded file payload
}
```

7.5 Import Data Validation & Schema Integrity Rules

When importing JSON backups or CSV statement files, InsAcc executes validation checks:

```
File Upload Event ──► JSON/CSV Schema Validator
                        ├── Rule 1: Check `version === "8"` in JSON Header
                        ├── Rule 2: Assert presence of mandatory key arrays
                        ├── Rule 3: Validate numeric parsing on amounts
                        └── Passed ──► Commit to State & Reload Window
```

---

8. Summary

InsAcc provides robust data transport capabilities through JSON full-state backups, RFC 4180 CSV report generators, electronic bank statement parsers, and Base64 document attachment storage.

---

9. Chapter Appendix

Supported Import & Export Format Matrix

Interface Feature	Input / Output Format	Primary Target Location	Validation Rule
System Backup	JSON Snapshot	`state_backup_YYYY-MM-DD.json`	Version `'8'` check
Financial Reports	CSV Text File	User Downloads Folder	RFC 4180 Escaping
Bank Statements	CSV / Excel File	Memory Parser	4-Column Schema
Document Store	Base64 String	`insacc_documents`	Size limit $< 500\text{ KB}$

---

10. Glossary

- **Base64**: A group of binary-to-text encoding schemes that represent binary data in an ASCII string format.

- **RFC 4180**: The technical specification defining standard Common Format and MIME Type for Comma-Separated Values (CSV) files.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- LocalStorage Persistence Architecture: Volume 02 Chapter 01
- Desktop IPC Bridge: Volume 05 Chapter 01
- Target REST API: Volume 05 Chapter 03

title: "Volume 05: Interface and Integration Spec - Chapter 03: Target REST API [To Be Implemented]" document_id: "INSACC-DOC-V05-CH03" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Target Architecture Specification" classification: "Commercial Enterprise Documentation"

Volume 05: Interface and Integration Specification

Chapter 03: Target REST API `[To Be Implemented]`

> **Single Source of Truth Reference**: All target REST API endpoints, OpenAPI schemas, and authentication protocols defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter defines the target OpenAPI 3.0 HTTPS REST API contract for the InsAcc Enterprise Server backend `[To Be Implemented]`. It details endpoint paths, HTTP verbs, JSON request/response schemas, Bearer JWT authentication, status codes, and rate limits planned for enterprise release v2.0.0.

2. Scope

This specification covers:

- OpenAPI 3.0 protocol standards, HTTPS TLS 1.3 encryption, and Bearer JWT authentication.
- Authentication endpoints (`POST /api/v1/auth/login` , `POST /api/v1/auth/refresh`).
- Investment endpoints (`GET /api/v1/investments` , `POST /api/v1/investments` , `POST /api/v1/investments/purchases`).
- Property endpoints (`GET /api/v1/properties/units` , `POST /api/v1/properties/leases` , `POST /api/v1/properties/pdc/transition`).
- Double-entry accounting endpoints (`GET /api/v1/accounting/accounts` , `POST /api/v1/accounting/vouchers`).
- Financial statement endpoints (`GET /api/v1/reports/balance-sheet` , `GET /api/v1/reports/profit-loss`).

Out of Scope:

- Current desktop Electron IPC bridge (covered in Volume 05 Chapter 01).
- Database migration DDL scripts (covered in Volume 02 Chapter 03).

3. Audience

This document is authored for:

- Backend Systems Engineers and API Developers
- Mobile Application Engineers and Integration Partners
- Enterprise Security & Compliance Auditors

4. Prerequisites

Before reviewing REST API contracts:

1. Review API specifications in API_CONTRACT.md.
2. Understand OpenAPI 3.0 JSON schema standards and Bearer JWT token authorization headers.

5. Warnings & Operational Hazards

> **WARNING: To Be Implemented**: The REST API endpoints documented in this chapter are target architecture specifications planned for server release v2.0.0. InsAcc v1.0.0 operates as an offline desktop application using local Electron IPC and `localStorage` .

6. Notes & Architecture Context

> **NOTE: Stateless REST Security**: All non-public REST API endpoints require a valid HTTP Bearer token header (`Authorization: Bearer <JWT_TOKEN>`). API sessions are completely stateless; token revocation is enforced via Redis token blacklist tracking `[To Be Implemented]` .

7. Main Content

7.1 OpenAPI 3.0 Protocol & Authentication Overview `[To Be Implemented]`

- **Base URL**: `https://erp.insacc.com/api/v1`
- **Protocol**: HTTPS over TLS 1.3
- **Format**: `application/json`
- **Authentication**: `Authorization: Bearer <JWT_TOKEN>`

7.2 Authentication Endpoints (`/api/v1/auth/*`) `[To Be Implemented]`

`POST /api/v1/auth/login`

- **Purpose**: Authenticates user credentials and returns a Bearer JWT token.
- **Request Body**:

```
{
  "email": "admin@insacc.com",
  "password": "SecurePassword123"
}
```

- **Response (`200 OK`)**:

```
{
  "status": "success",
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "tokenType": "Bearer",
    "expiresIn": 86400,
    "user": { "id": "u-1001", "email": "admin@insacc.com", "role": "Admin" }
  }
}
```

- **Status Codes**: `200 OK` , `400 Bad Request` , `401 Unauthorized` , `429 Too Many Requests` .

7.3 Investment Module Endpoints (`/api/v1/investments/*`) `[To Be Implemented]`

`GET /api/v1/investments`

- **Purpose**: Retrieves all portfolio holdings and valuations.
- **Headers**: `Authorization: Bearer <JWT_TOKEN>`
- **Response (`200 OK`)**: Array of investment holdings objects.

`POST /api/v1/investments`

- **Purpose**: Registers a new investment holding position.
- **Request Body**: `{"assetName": "24K Gold Bar 1kg", "assetType": "Gold", "quantity": 2, "purchaseValue": 500000.00, "currentPrice": 285000.00}`

7.4 Property Management Endpoints (`/api/v1/properties/*`) `[To Be Implemented]`

`POST /api/v1/properties/leases`

- **Purpose**: Registers a new lease contract, updates unit status, and generates PDCs.
- **Request Body**:

```
{
  "tenantId": "ten-1001",
  "unitId": "unit-101",
  "startDate": "2026-07-01",
  "endDate": "2027-06-30",
  "annualRent": 120000.00,
  "paymentFrequency": "Quarterly",
  "securityDeposit": 5000.00
}
```

- **Response (`201 Created`)**: Returns created lease record and generated PDC IDs.

7.5 Double-Entry Accounting Endpoints (`/api/v1/accounting/*`) `[To Be Implemented]`

`POST /api/v1/accounting/vouchers`

- **Purpose**: Creates a double-entry accounting voucher (`RV` , `PV` , `JV`).
- **Validation**: Asserts $\sum \text{Debits} = \sum \text{Credits}$ ($\left| \sum D - \sum C \right| < 0.001$).

7.6 Financial Reporting Endpoints (`/api/v1/reports/*`) `[To Be Implemented]`

`GET /api/v1/reports/balance-sheet?asOfDate=2026-06-30`

- **Purpose**: Returns the Balance Sheet financial statement as of target date.

---

8. Summary

The target REST API specification outlines an OpenAPI 3.0-compliant HTTPS REST interface planned for server release v2.0.0. By providing JSON endpoint contracts for authentication, investments, property, vouchers, and reporting, InsAcc lays the foundation for enterprise client-server multi-tenancy.

---

9. Chapter Appendix

Standard Error Response Format (RFC 7807)

```
{
  "status": "error",
  "errorCode": "VOUCHER_UNBALANCED",
  "message": "Debit lines total (AED 10,000.00) does not equal Credit lines total (AED 9,500.00).",
  "timestamp": "2026-07-22T12:30:00.000Z",
  "path": "/api/v1/accounting/vouchers"
}
```

10. Glossary

- **JWT (JSON Web Token)**: An open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object.
- **REST (Representational State Transfer)**: An architectural style for designing networked applications using stateless HTTP requests.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Complete API Contract: docs/API_CONTRACT.md
- Desktop IPC Bridge: Volume 05 Chapter 01
- Database Migration Plan: Volume 02 Chapter 03

title: "Volume 06: Developer Architecture Guide - Chapter 01: System Architecture and CQRS" document_id: "INSACC-DOC-V06-CH01" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 06: Developer Architecture & Technical Specification

Chapter 01: System Architecture and CQRS

> **Single Source of Truth Reference**: All architectural patterns, CQRS segregations, and project folder structures defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter defines the software architecture, design patterns, Command Query Responsibility Segregation (CQRS) flow, and codebase organization of the InsAcc ERP platform.

2. Scope

This specification covers:

- Architectural segregation of write operations (Commands) from read operations (Queries).
- The Command pipeline (voucher creation, lease execution, PDC transitions).
- The Query projection pipeline (`useMemo` , `investmentReadModels.ts` , `reportService.ts`).
- Folder hierarchy under `src/main/` and `src/renderer/` .
- React 18 component tree structure and context providers (`MasterDataContext.tsx`).

Out of Scope:

- General Ledger double-entry validation algorithms (covered in Volume 06 Chapter 02).
- CSS Tokens and design system implementation (covered in Volume 06 Chapter 04).

3. Audience

This document is authored for:

- Senior Frontend Architects and Core Maintainers
- Full-Stack TypeScript Engineers
- Code Reviewers and Technical Leads

4. Prerequisites

Before reviewing architecture implementation:

1. Review the software architecture defined in MASTER_ARCHITECTURE.md#2-software-architecture.
2. Understand React 18 functional components, hooks, and TypeScript generics.

5. Warnings & Operational Hazards

> **WARNING: DIRECT STATE MUTATION PROHIBITION**: Modifying domain objects directly inside query projection functions causes side effects and breaks React memoization caches. Developers MUST keep query projections pure and perform state updates strictly through command hooks (`useVoucherLifecycle.ts`).

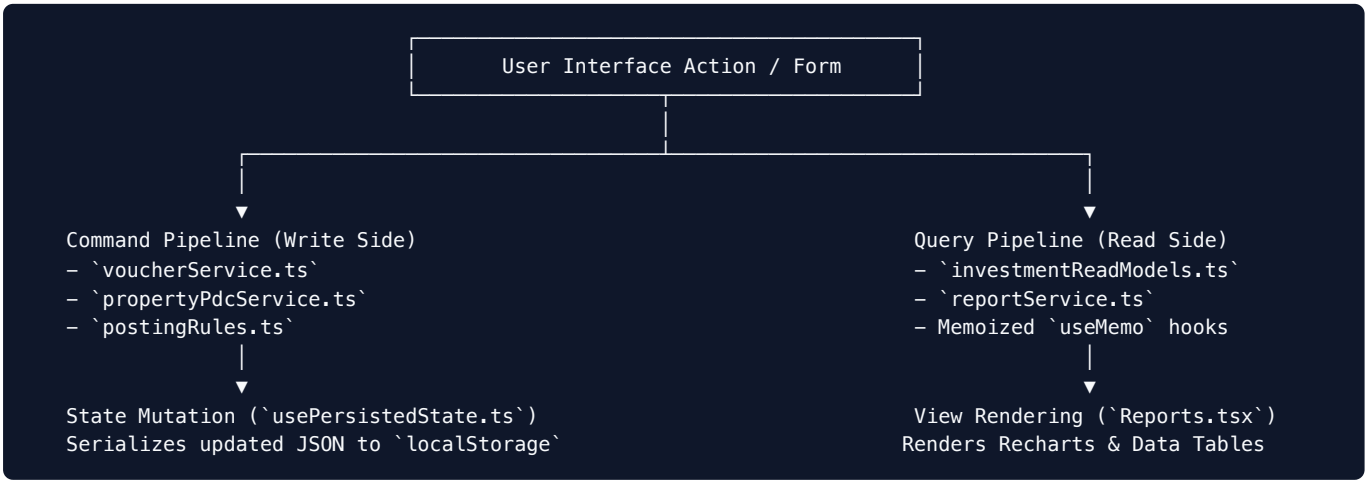
6. Notes & Architecture Context

> **NOTE: Sub-Millisecond Read Projections**: By decoupling write-side domain mutations from read-side views via CQRS, InsAcc calculates complex financial statements (Trial Balance, P&L, Balance Sheet) in client memory with zero server round-trip latency.

7. Main Content

7.1 Command Query Responsibility Segregation (CQRS) Architecture

InsAcc decouples state modification (Commands) from view projections (Queries):



7.2 The Command Execution Pipeline (Write Operations)

Commands encapsulate state-changing business logic:

1. **User Triggers Command**: (e.g., `postVoucher(voucherId)` in `VoucherLifecycleActions.tsx`).
2. **Posting Validator Execution**: `postingValidator.ts` asserts balance equality ($\sum D = \sum C$) and verifies period status is not locked.
3. **Domain Event Posting**: `ledgerService.ts` sets voucher status to `Posted` and appends an audit log (`auditService.ts`).
4. **State Serialization**: `usePersistedState.ts` writes the updated voucher array to `localStorage`.

7.3 The Query Projection Pipeline (Read Operations)

Queries transform raw persisted collections into read models without side effects:

```
// Query Projection Example (InvestmentReadModels.ts)
export function getPortfolioValuation(investments: Investment[]): PortfolioValuation {
  return useMemo(() => {
    let totalCost = 0
    let totalMarketValue = 0

    for (const inv of investments) {
      totalCost += inv.purchaseValue
      totalMarketValue += inv.quantity * inv.currentPrice
    }

    const unrealizedGain = totalMarketValue - totalCost
    const returnPercentage = totalCost > 0 ? (unrealizedGain / totalCost) * 100 : 0

    return { totalCost, totalMarketValue, unrealizedGain, returnPercentage }
  }, [investments])
}
```

7.4 Codebase Directory Structure & Component Organization

The repository is structured into distinct functional layers:

```
InsAcc Codebase Structure
├── src/
│   ├── main/                      # Electron Main Process & Preload
│   │   ├── main.js                # BrowserWindow setup & IPC handlers
│   │   └── preload.js             # Context bridge definitions (window.api)
│   │
│   └── renderer/                  # React 18 Application Subsystem
│       ├── accounting/            # Accounting Engine & Posting Rules
│       │   ├── accountingEngine.ts # Core engine coordinator
│       │   ├── postingRules.ts     # Domain event posting rules
│       │   ├── systemAccountRegistry.ts # Reserved account codes
│       │   └── types.ts            # Voucher & Account interfaces
│       ├── components/            # UI Views & Design Components
│       │   ├── design/            # Reusable design tokens & modals
│       │   ├── InvestmentDashboard.tsx
│       │   ├── PropertyRouter.tsx
│       │   └── Settings.tsx
│       ├── readModels/            # CQRS Query Projection Functions
│       ├── services/              # Domain Business Logic Services
│       └── usePersistedState.ts   # Storage Persistence Hook
```

7.5 React 18 Component Tree & Context Providers

```
<App>
  <MasterDataProvider>            <-- Central Context Provider (src/renderer/contexts/)
    <ThemeContainer>               <-- Applied .dark-mode CSS class
      <ProfileSelection>           <-- Renders Login / Profile / Module Screens
        <WorkspaceShell>           <-- Renders Sidebar & Header Navigation
          <ActiveModuleRouter />   <-- Renders Investment or Property Module Views
        </WorkspaceShell>
      </ProfileSelection>
    </ThemeContainer>
  </MasterDataProvider>
</App>
```

8. Summary

InsAcc uses a CQRS pattern to separate command operations from query read models. Combined with a modular React 18 directory structure and memoized projection functions, InsAcc ensures predictable state transitions and sub-millisecond view rendering.

9. Chapter Appendix

Core Architecture Component Reference Matrix

File Path	Architecture Role	Functional Responsibility
`src/main/main.js`	Main Process	Electron window creation & `save-file` IPC handler
`src/main/preload.js`	Preload Bridge	Secure `window.api` context bridge exposure
`src/renderer/accounting/postingRules.ts`	Command Pipeline	Maps domain events to debit/credit voucher lines
`src/renderer/readModels/`	Query Pipeline	Memoized read-model projection functions
`src/renderer/usePersistedState.ts`	State Persistence	Synchronous `localStorage` serialization hook

10. Glossary

- **CQRS (Command Query Responsibility Segregation)**: An architectural pattern that separates read and update operations for a data store.
- **Memoization**: An optimization technique used primarily to speed up computer programs by storing the results of expensive function calls.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Double Entry Engine Specs: Volume 06 Chapter 02
- Read Models & Formatters: Volume 06 Chapter 03
- UI Design System: Volume 06 Chapter 04

title: "Volume 06: Developer Architecture Guide - Chapter 02: Double Entry Accounting Engine" document_id: "INSACC-DOC-V06-CH02" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 06: Developer Architecture & Technical Specification

Chapter 02: Double Entry Accounting Engine

> **Single Source of Truth Reference**: All accounting algorithms, balance validation equations, and ledger posting routines defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter provides the technical code specification for the event-driven, double-entry general ledger accounting engine in `src/renderer/accounting/`. It details component interaction, posting rules resolution, mathematical balance validation, and dynamic account balance derivation algorithms.

2. Scope

This specification covers:

- Core engine source files (`accountingEngine.ts` , `postingRules.ts` , `postingValidator.ts` , `systemAccountRegistry.ts` , `ledgerService.ts`).
- Domain event to debit/credit voucher resolution rules.
- Floating-point balance equality verification ($\left| \sum D - \sum C \right| < 0.001$).
- General Ledger voucher posting and state transition processing.
- Dynamic balance calculation algorithms.

Out of Scope:

- UI voucher creation forms (covered in Volume 04 Chapter 07).
- Database migration schemas `[To Be Implemented]` (covered in Volume 02 Chapter 03).

3. Audience

This document is authored for:

- Core Accounting Engine Developers & Technical Leads
- Financial Software Quality Assurance Engineers
- Security & Compliance Auditors

4. Prerequisites

Before evaluating engine source code:

1. Review the accounting architecture defined in MASTER_ARCHITECTURE.md#10-accounting-architecture.
2. Review the Accounting Engine Specification in docs/ACCOUNTING_ENGINE_SPECIFICATION.md.

5. Warnings & Operational Hazards

> **WARNING: FLOATING-POINT ROUNDING HAZARD**: Standard JavaScript IEEE 754 floating-point arithmetic can introduce representation drift (e.g., `0.1 + 0.2 = 0.30000000000000004`). In `postingValidator.ts` , balance checks MUST evaluate difference absolute values against epsilon tolerance ($\left| \sum D - \sum C \right| < 0.001$) rather than direct `===` equality.

6. Notes & Architecture Context

> **NOTE: Decoupled Business Rules**: Posting rules in `postingRules.ts` contain zero UI rendering logic. They act as pure transformation functions converting business event payloads into structured voucher lines.

7. Main Content

7.1 Accounting Engine Core Components (`src/renderer/accounting/`)

The accounting engine consists of five dedicated TypeScript modules:

```
┌──────────────────────────────────────────────────────────────────────┐
│                   Accounting Engine Subsystem Modules                │
├──────────────────────────┬──────────────────────────┬────────────────┤
│   `postingRules.ts`      │  `postingValidator.ts`   │   `ledgerService.ts`    │
│ Resolves Event Rules     │ Asserts Balance Equality│ General Ledger Processor│
├──────────────────────────┼──────────────────────────┼────────────────┤
│`systemAccountRegistry`   │  `accountingEngine.ts`   │      `types.ts`         │
│ System Reserved Codes    │ Central Engine Pipeline │ Core TypeScript Models  │
└──────────────────────────┴──────────────────────────┴────────────────┘
```

7.2 Domain Event Resolver (`postingRules.ts`)

`postingRules.ts` resolves domain events into double-entry line configurations:

```
export function resolvePostingRule(event: AccountingEvent): VoucherLineConfig[] {
  switch (event.type) {
    case 'RENT_COLLECTED':
      return [
        { accountId: event.bankAccountId || SystemAccountRegistry.BANK, lineType: 'Debit', amount: event.amoun
        { accountId: SystemAccountRegistry.RENTAL_REVENUE, lineType: 'Credit', amount: event.amount }
      ]
    case 'PDC_CLEARED':
      return [
        { accountId: event.bankAccountId || SystemAccountRegistry.BANK, lineType: 'Debit', amount: event.amoun
        { accountId: SystemAccountRegistry.PDC_HELD, lineType: 'Credit', amount: event.amount },
        { accountId: SystemAccountRegistry.UNEARNED_RENT, lineType: 'Debit', amount: event.amount },
        { accountId: SystemAccountRegistry.RENTAL_REVENUE, lineType: 'Credit', amount: event.amount }
      ]
    default:
      throw new Error(`Unrecognized accounting event type: ${event.type}`)
  }
}
```

7.3 Strict Balance Validator (`postingValidator.ts`)

Before any voucher is committed to the General Ledger, `postingValidator.ts` asserts mathematical balance equality:

```
export function validateVoucherPosting(voucher: Voucher, lockedPeriods: Period[]): ValidationResult {
  // 1. Verify period is not locked
  if (isPeriodLocked(voucher.voucherDate, lockedPeriods)) {
    return { isValid: false, error: 'Cannot post voucher to a locked fiscal period.' }
  }

  // 2. Compute Total Debits and Total Credits
  let totalDebit = 0
  let totalCredit = 0

  for (const line of voucher.lines) {
    if (line.amount <= 0) {
      return { isValid: false, error: `Invalid line amount (${line.amount}) on account ${line.accountId}.` }
    }
    if (line.type === 'Debit') totalDebit += line.amount
    if (line.type === 'Credit') totalCredit += line.amount
  }

  // 3. Floating-point balance tolerance check (< 0.001)
  const difference = Math.abs(totalDebit - totalCredit)
  if (difference >= 0.001) {
    return {
      isValid: false,
      error: `Voucher is unbalanced. Total Debits (${totalDebit.toFixed(2)}) != Total Credits (${totalCredit.t
    }
  }

  return { isValid: true }
}
```

7.4 General Ledger State Processor (`ledgerService.ts`)

`ledgerService.ts` coordinates voucher state transitions:

```
export function postVoucherToLedger(
  voucherId: string,
  vouchers: Voucher[],
  lockedPeriods: Period[]
): Voucher[] {
  return vouchers.map(v => {
    if (v.id !== voucherId) return v

    const validation = validateVoucherPosting(v, lockedPeriods)
    if (!validation.isValid) {
      throw new Error(`Ledger Posting Rejected: ${validation.error}`)
    }

    return {
      ...v,
      status: 'Posted',
      postedAt: new Date().toISOString()
    }
  })
}
```

7.5 Account Balance Derivation Code Implementation

In compliance with the **Derived Balance Golden Rule**, account balances are computed on read:

```
export function getAccountDerivedBalance(
  accountId: string,
  vouchers: Voucher[],
  openingBalance: number = 0
): number {
  const postedVouchers = vouchers.filter(v => v.status === 'Posted')
  let debitSum = 0
  let creditSum = 0

  for (const v of postedVouchers) {
    for (const line of v.lines) {
      if (line.accountId === accountId) {
        if (line.lineType === 'Debit') debitSum += line.amount
        if (line.lineType === 'Credit') creditSum += line.amount
      }
    }
  }

  return openingBalance + debitSum - creditSum
}
```

8. Summary

The double-entry accounting engine in `src/renderer/accounting/` provides a robust, event-driven general ledger. By validating floating-point balance equality ($\left| \sum D - \sum C \right| < 0.001$) and deriving account balances dynamically, InsAcc ensures absolute financial integrity.

9. Chapter Appendix

Engine Validation Error Dictionary

Error Identifier	Root Cause	Triggering Condition
`ERR_PERIOD_LOCKED`	Fiscal period is locked	Voucher date falls in closed period
`ERR_UNBALANCED_VOUCHER`	Debits != Credits	$\left| \sum D - \sum C \right| \ge 0.001$
`ERR_INVALID_LINE_AMOUNT`	Non-positive line amount	Line amount ≤ 0
`ERR_ACCOUNT_NOT_FOUND`	Missing GL account	Invalid `accountId` reference

10. Glossary

- **Floating-Point Tolerance**: A small value ($\epsilon = 0.001$) used to compare floating-point numbers for practical equality.
- **Posting Rule**: An accounting business logic rule that maps an operational domain event into corresponding debit and credit ledger lines.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md

- Accounting Engine Spec: docs/ACCOUNTING_ENGINE_SPECIFICATION.md
- System Architecture & CQRS: Volume 06 Chapter 01
- Read Models & Formatters: Volume 06 Chapter 03

title: "Volume 06: Developer Architecture Guide - Chapter 03: Read Models and Formatters" document_id: "INSACC-DOC-V06-CH03" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 06: Developer Architecture & Technical Specification

Chapter 03: Read Models and Formatters

> **Single Source of Truth Reference**: All read-model projection functions, formatting utilities, and CSV export routines defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

---

1. Purpose

This chapter provides the technical code specification for the read-model projection functions in `src/renderer/readModels/` and report formatting utilities in `reportFormatters.ts`.

---

2. Scope

This specification covers:

- Read-model projection modules (`InvestmentDashboardReadModel.ts` , `InvestmentFinancialOverviewReadModel.ts` , `InvestmentReportsReadModel.ts`).
- Property module financial statement read models (`reportService.ts`).
- Formatting utilities in `reportFormatters.ts` (`formatCurrency` , `formatDate` , `formatPercentage`).
- CSV export generation engine (`exportToCSV`).

Out of Scope:

- Core `localStorage` persistence hooks (covered in Volume 02 Chapter 01).
- Double-entry accounting engine posting rules (covered in Volume 06 Chapter 02).

---

3. Audience

This document is authored for:

- Frontend React Engineers and UI Component Developers
- Data Visualization Specialists
- Code Reviewers and Technical Leads

---

4. Prerequisites

Before evaluating read-model implementation:

1. Understand the CQRS architecture defined in Volume 06 Chapter 01.
2. Understand React 18 `useMemo` hooks and memoization dependency arrays.

---

5. Warnings & Operational Hazards

> **WARNING: READ MODEL MEMOIZATION DEPENDENCY HAZARD**: Omitting state variables from `useMemo` dependency arrays causes views to display stale financial calculations. Developers MUST include all input state collections in the dependency array (e.g., `[vouchers, investments, leases]`).

---

6. Notes & Architecture Context

> **NOTE: Pure Functional Projections**: Read-model functions are pure functions with zero side effects. Given identical input state arrays, read models produce identical output calculations without modifying original state objects.

7. Main Content

7.1 Read Models Subsystem Overview (`src/renderer/readModels/`)

The read-model layer transforms raw persisted collections into structured view models:

```
Persisted State Collections (Raw JSON Arrays)
├── `insacc_investments`
├── `insacc_transactions`
└── `insacc_prop_rent`
          |
          ▼
Read-Model Projection Functions (`useMemo`)
├── `getInvestmentDashboardMetrics()`
├── `getFinancialOverview()`
└── `calculateFinancialSummary()`
          |
          ▼
UI View Components (Dashboard Cards, Recharts, Export Modals)
```

7.2 Investment Module Read Models

Located in `src/renderer/readModels/`:

1. **`InvestmentDashboardReadModel.ts`**

Projects high-level dashboard KPI metrics:

```
export function useInvestmentDashboardMetrics(investments: Investment[], transactions: Transaction[]) {
  return useMemo(() => {
    const totalPortfolioValue = investments.reduce((acc, item) => acc + (item.quantity * item.currentPrice), 0
    const totalCostBasis = investments.reduce((acc, item) => acc + item.purchaseValue, 0)
    const unrealizedGain = totalPortfolioValue - totalCostBasis
    const activeHoldingsCount = investments.length

    return { totalPortfolioValue, totalCostBasis, unrealizedGain, activeHoldingsCount }
  }, [investments, transactions])
}
```

2. **`InvestmentFinancialOverviewReadModel.ts`**

Projects asset allocation percentages for `AssetAllocationPie.tsx`.

3. **`InvestmentReportsReadModel.ts`**

Projects historical performance tables and growth trends for `InvestmentGrowthChart.tsx`.

7.3 Property Module Read Models & Financial Statement Projections

Located in `src/renderer/services/reportService.ts`:

```
export function calculateFinancialSummary(vouchers: Voucher[]) {
  const posted = vouchers.filter(v => v.status === 'Posted')
  let totalRevenue = 0
  let totalExpense = 0

  for (const v of posted) {
    for (const line of v.lines) {
      if (line.accountId.startsWith('4')) totalRevenue += line.amount
      if (line.accountId.startsWith('5')) totalExpense += line.amount
    }
  }

  const netProfit = totalRevenue - totalExpense
  return { totalRevenue, totalExpense, netProfit }
}
```

7.4 Currency, Date & Percentage Formatting Utilities (`reportFormatters.ts`)

Located in `src/renderer/utils/reportFormatters.ts`:

```
// Currency Formatting
export function formatCurrency(amount: number, currencyCode: string = 'AED'): string {
  const formatted = Math.abs(amount).toLocaleString('en-US', {
    minimumFractionDigits: 2,
    maximumFractionDigits: 2
  })
  return `${amount < 0 ? '-' : ''}${currencyCode} ${formatted}`
}

// Date Formatting (ISO to Display Mask)
export function formatDate(dateString: string, mask: string = 'YYYY-MM-DD'): string {
  if (!dateString) return ''
  const d = new Date(dateString)
  const year = d.getFullYear()
  const month = String(d.getMonth() + 1).padStart(2, '0')
  const day = String(d.getDate()).padStart(2, '0')

  if (mask === 'DD/MM/YYYY') return `${day}/${month}/${year}`
  if (mask === 'MM/DD/YYYY') return `${month}/${day}/${year}`
  return `${year}-${month}-${day}` // Default ISO 8601
}

// Percentage Formatting
export function formatPercentage(value: number): string {
  return `${value >= 0 ? '+' : ''}${value.toFixed(1)}%`
}
```

7.5 CSV Export Generation Engine (`exportToCSV`)

`exportToCSV` serializes array structures into RFC 4180-compliant CSV text:

```
export function exportToCSV(filename: string, headers: string[], rows: (string | number)[][]): string {
  const escapeField = (field: string | number) => {
    const text = String(field ?? '')
    return `"${text.replace(/"/g, '""')}"`
  }

  const headerRow = headers.map(escapeField).join(',')
  const bodyRows = rows.map(r => r.map(escapeField).join(','))
  return [headerRow, ...bodyRows].join('\r\n')
}
```

8. Summary

The read-model layer transforms raw state collections into structured view models using pure, memoized projection functions. Combined with formatting utilities in `reportFormatters.ts` and RFC 4180 CSV serialization, InsAcc delivers sub-millisecond view rendering and accurate report exports.

9. Chapter Appendix

Read Model Function Reference Matrix

Function Name	Target Component	Input Collections	Primary Projected Output
`useInvestmentDashboardMetrics`	`InvestmentDashboard.tsx`	`investments`, `transactions`	Total Value, Cost Basis, Return %
`calculateFinancialSummary`	`Reports.tsx`	`vouchers`	Total Revenue, Total Expense, Net Profit
`formatCurrency`	All Data Tables	`amount`, `currencyCode`	Formatted String (`AED 10,000.00`)
`exportToCSV`	Report Toolbar Modals	`headers`, `rows`	RFC 4180 CSV Text Payload

10. Glossary

- **Memoization (`useMemo`)**: A React optimization hook that caches the result of a calculation between re-renders.
- **Pure Function**: A function that always returns the same output for the same inputs and produces no side effects.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- System Architecture & CQRS: Volume 06 Chapter 01
- Double-Entry Engine Specs: Volume 06 Chapter 02
- Financial Reporting & Exports: Volume 04 Chapter 08

title: "Volume 06: Developer Architecture Guide - Chapter 04: UI Design System and Tokens" document_id: "INSACC-DOC-V06-CH04" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 06: Developer Architecture & Technical Specification

Chapter 04: UI Design System and Tokens

> **Single Source of Truth Reference**: All design tokens, CSS custom properties, typography rules, and UI component standards defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter defines the enterprise UI design system, CSS custom property tokens, typography rules, color palettes, micro-animation parameters, and component library specifications for InsAcc.

2. Scope

This specification covers:

- Core CSS custom properties in `src/renderer/styles/theme.css`.
- Typography hierarchy using the bundled Inter font family (`.woff2`).
- Dark mode baseline palette (`#0C0C0D`, `#1C1C1F`) and Light mode overrides.
- Glassmorphism backdrop filters and shadow elevation tokens.
- Component design gallery (`src/renderer/components/design/DesignSystem.tsx`).
- Reusable UI design components (`VoucherLifecycleActions.tsx`, `VoucherDetailsModal.tsx`).

Out of Scope:

- General React component tree routing (covered in Volume 06 Chapter 01).
- Playwright visual QA testing (covered in Volume 06 Chapter 05).

3. Audience

This document is authored for:

- User Interface (UI) and User Experience (UX) Engineers
- Frontend React Developers
- Design System Maintainers

4. Prerequisites

Before modifying CSS design tokens:

1. Review UI standards defined in MASTER_ARCHITECTURE.md#13-ui-standards.
2. Understand vanilla CSS custom property scoping (`:root` vs `.dark-mode`).

5. Warnings & Operational Hazards

> **WARNING: HARDCODED HEX COLOR PROHIBITION**: Developers MUST NOT hardcode raw hex color strings (e.g., `#6366F1`) inside inline styles or component CSS rules. All colors MUST reference predefined CSS variables (e.g., `var(--color-brand)`). Hardcoding hex strings breaks theme switching functionality.

6. Notes & Architecture Context

> **NOTE: Zero External CSS Framework Dependency**: InsAcc uses pure Vanilla CSS custom properties without Tailwind CSS or Bootstrap dependencies. This guarantees total styling control, zero CSS specificity bloat, and sub-millisecond styling performance.

7. Main Content

7.1 Design Tokens Architecture (`theme.css`)

Design tokens are declared centrally in `src/renderer/styles/theme.css`:

```
:root {
  /* Font Family Definition */
  --font-sans: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;

  /* Spacing Scale Tokens */
  --space-xs: 4px;
  --space-sm: 8px;
  --space-md: 16px;
  --space-lg: 24px;
  --space-xl: 32px;

  /* Border Radius Tokens */
  --radius-sm: 4px;
  --radius-md: 8px;
  --radius-lg: 12px;
  --radius-pill: 9999px;

  /* Micro-Animation Timing */
  --transition-fast: 150ms cubic-bezier(0.4, 0, 0.2, 1);
  --transition-normal: 250ms cubic-bezier(0.4, 0, 0.2, 1);
}
```

7.2 Typography System & Bundled Font Assets

InsAcc bundles the **Inter** font family locally in `public/fonts/` (Weights 400, 500, 600, 700 in `.woff2` format).

Token Name	Font Size	Font Weight	Line Height	Target UI Usage
`--text-xs`	11px	400 (Regular)	1.4	Badge labels, table footers
`--text-sm`	13px	400 / 500	1.4	Table cell text, form labels
`--text-md`	15px	500 (Medium)	1.5	Body text, button titles
`--text-lg`	18px	600 (SemiBold)	1.3	Card titles, section headers
`--text-xl`	24px	700 (Bold)	1.2	Page title header (`.page-title`)
`--text-kpi`	32px	700 (Bold)	1.1	Dashboard KPI numbers

7.3 Palette Tokens (Dark Mode Baseline vs Light Mode)

```
/* Dark Mode Palette (Default Baseline) */
:root, .dark-mode {
  --bg-primary: #0C0C0D;
  --bg-surface: #1C1C1F;
  --bg-surface-hover: #26262A;
  --border-color: #2E2E32;

  --text-primary: #F4F4F5;
  --text-secondary: #A1A1AA;
  --text-muted: #71717A;

  --color-brand: #6366F1;       /* Primary Indigo Accent */
  --color-gold: #F59E0B;        /* Wealth Asset Accent */
  --color-success: #10B981;     /* Positive Return / Posted Status */
  --color-warning: #F59E0B;     /* Pending Approval / Draft Status */
  --color-danger: #EF4444;      /* Bounced Cheque / Loss / Cancelled */
}

/* Light Mode Overrides */
html:not(.dark-mode) {
  --bg-primary: #F8FAFC;
  --bg-surface: #FFFFFF;
  --bg-surface-hover: #F1F5F9;
  --border-color: #E2E8F0;

  --text-primary: #0F172A;
  --text-secondary: #475569;
  --text-muted: #94A3B8;
}
```

7.4 Micro-Animations, Glassmorphism & Elevation Shadow Tokens

1. Glassmorphism Backdrop Tokens:

```
--glass-bg: rgba(28, 28, 31, 0.75);
--glass-backdrop: blur(12px);
--glass-border: 1px solid rgba(255, 255, 255, 0.08);
```

2. Elevation Shadows:

```
--shadow-sm: 0 1px 2px 0 rgba(0, 0, 0, 0.05);
--shadow-md: 0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -1px rgba(0, 0, 0, 0.06);
--shadow-lg: 0 10px 15px -3px rgba(0, 0, 0, 0.2), 0 4px 6px -2px rgba(0, 0, 0, 0.05);
```

7.5 Design Component Library & Interactive Showcase (`DesignSystem.tsx`)

Developers can preview and inspect the component library via `src/renderer/components/design/DesignSystem.tsx`:

- **`VoucherLifecycleActions.tsx`**: Renders 5-stage voucher state transition buttons (`Approve` , `Post` , `Cancel` , `Reverse`).
- **`VoucherDetailsModal.tsx`**: Renders double-entry voucher line items with debit/credit balance indicators.

---

8. Summary

The InsAcc design system establishes a consistent UI visual identity powered by CSS custom property tokens, local Inter typography, glassmorphism card elevation, and Dark/Light palette switching.

9. Chapter Appendix

CSS Token Cheat Sheet Reference

Property Type	Variable Token	Dark Mode Default	Light Mode Value
Primary Background	`var(--bg-primary)`	`#0C0C0D`	`#F8FAFC`
Surface Card	`var(--bg-surface)`	`#1C1C1F`	`#FFFFFF`
Brand Primary	`var(--color-brand)`	`#6366F1`	`#6366F1`
Gold Accent	`var(--color-gold)`	`#F59E0B`	`#D97706`
Border Line	`var(--border-color)`	`#2E2E32`	`#E2E8F0`

10. Glossary

- **CSS Custom Properties (CSS Variables)**: Entities defined by CSS authors that contain specific values to be reused throughout a document.
- **Glassmorphism**: A design trend characterized by translucent frosted-glass backgrounds with subtle light borders and background blur.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- System Architecture & CQRS: Volume 06 Chapter 01
- Playwright Test Framework: Volume 06 Chapter 05

title: "Volume 06: Developer Architecture Guide - Chapter 05: Playwright Test Framework" document_id: "INSACC-DOC-V06-CH05" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 06: Developer Architecture & Technical Specification

Chapter 05: Playwright Test Framework

> **Single Source of Truth Reference**: All automated test specifications, Playwright test suite configurations, and multi-viewport matrices defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter defines the technical implementation of the automated end-to-end (E2E) integration test framework in InsAcc using Microsoft Playwright.

2. Scope

This specification covers:

- Playwright configuration in `playwright.config.ts`.
- Directory structure for automated integration tests (`tests/`).
- The 76-test integration pipeline across 3 test modules (`purchase-ledger.spec.ts`, `reports-visual.spec.ts`, `transactions.spec.ts`).
- Multi-viewport visual QA matrix across 4 viewport resolutions in Light and Dark modes.
- Command-line test execution, HTML report generation, and interactive UI debugging mode.

Out of Scope:

- General pre-production manual QA procedures (covered in Volume 01 Chapter 04).
- CI/CD pipeline automation setup (covered in Volume 01 Chapter 03).

3. Audience

This document is authored for:

- Test Automation Engineers and Quality Assurance Leads
- Frontend Core Engineers and Maintainers
- DevOps & CI/CD Pipeline Engineers

4. Prerequisites

Before running automated Playwright tests:

1. Ensure Node.js 22 LTS is installed and dependencies are synchronized (`npm ci`).
2. Install Playwright browser binaries via `npx playwright install`.

5. Warnings & Operational Hazards

> **WARNING: DYNAMIC STRESS DISPOSITION IN CI**: Running Playwright tests with `fullyParallel: true` on resource-constrained CI runner nodes can cause browser context instantiation timeouts. On low-spec CI runners, constrain worker concurrency via `--workers=2`.

6. Notes & Architecture Context

> **NOTE: Local WebServer Auto-Start**: `playwright.config.ts` includes a `webServer` block that automatically launches Vite on port 5174 (`http://localhost:5174`) before running test suites, eliminating the need to manually start a dev server prior to testing.

7. Main Content

7.1 Playwright Integration Test Framework Overview

InsAcc uses Microsoft Playwright for end-to-end integration and visual regression testing. The suite validates UI component interactions, form inputs, dynamic calculations, and responsive layouts across viewports.

7.2 Playwright Configuration Architecture (`playwright.config.ts`)

```
import { defineConfig, devices } from '@playwright/test'

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 2 : undefined,
  reporter: [['html', { outputFolder: 'playwright-report' }]],
  use: {
    baseURL: 'http://localhost:5174',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure'
  },
  webServer: {
    command: 'npm run dev',
    url: 'http://localhost:5174',
    reuseExistingServer: !process.env.CI,
    timeout: 120000
  },
  projects: [
    { name: 'chromium-desktop', use: { ...devices['Desktop Chrome'], viewport: { width: 1920, height: 1080 } } },
    { name: 'chromium-laptop',  use: { ...devices['Desktop Chrome'], viewport: { width: 1440, height: 900 } } },
    { name: 'tablet-landscape', use: { ...devices['Desktop Chrome'], viewport: { width: 1024, height: 768 } } },
    { name: 'tablet-portrait',  use: { ...devices['Desktop Chrome'], viewport: { width: 768, height: 1024 } } }
  ]
})
```

7.3 The 76-Test Pipeline Breakdown Across 3 Test Modules

The test suite contains **76 end-to-end integration tests**:

```
Playwright Integration Suite (76 Total Tests)
├── `tests/purchase-ledger.spec.ts`  ──► 14 Tests (Purchase lot CRUD, weighted cost calculations)
├── `tests/reports-visual.spec.ts`   ──► 14 Tests (Visual QA across 4 viewports in Light/Dark mode)
└── `tests/transactions.spec.ts`     ──► 48 Tests (Voucher lifecycle, double-entry balance, filters)
```

Test Suite Breakdown:

1. `tests/purchase-ledger.spec.ts` **(14 Tests)**: Validates lot creation, modal forms, category dropdowns, cost basis calculations, and `computeAverages()` weighted average outputs.
2. `tests/reports-visual.spec.ts` **(14 Tests)**: Captures screenshot baselines across financial statement views (Trial Balance, P&L, Balance Sheet) to prevent layout regressions.
3. `tests/transactions.spec.ts` **(48 Tests)**: Validates voucher creation, line addition/deletion, debit-credit balance assertions ($\sum D = \sum C$), voucher posting, and reversal workflows.

7.4 Multi-Viewport & Light/Dark Theme Snapshot Matrix

Visual QA tests in `tests/reports-visual.spec.ts` execute against a 4-viewport grid:

Viewport Name	Resolution	Target Device Form Factor	Sidebar Layout State
Chromium Desktop	1920 x 1080	Full HD 1080p Desktop	Expanded (240px)
Chromium Laptop	1440 x 900	Standard Laptop Display	Expanded (240px)
Tablet Landscape	1024 x 768	Tablet Horizontal Mode	Expanded (240px)
Tablet Portrait	768 x 1024	Tablet Vertical Mode	Collapsed Icon-Only (56px)

7.5 Executing Automated Tests & Interactive Debugging

Command-Line Execution Scripts:

```
# Run all 76 Playwright integration tests headless
npx playwright test

# Execute a specific test specification file
npx playwright test tests/transactions.spec.ts

# Launch Playwright Interactive UI Mode (Visual Debugger)
npx playwright test --ui

# Open generated HTML test execution report
npx playwright show-report
```

---

8. Summary

The Playwright test framework provides automated quality assurance for InsAcc. With 76 integration tests across 3 spec files, multi-viewport layout validation, and automated dev server launching, Playwright prevents visual regressions and maintains double-entry accounting integrity across code changes.

---

9. Chapter Appendix

Sample Playwright Test Assertion Code Snippet

```
import { test, expect } from '@playwright/test'

test('should create a balanced Receipt Voucher (RV) and update bank balance', async ({ page }) => {
  await page.goto('/')

  // 1. Select Investor profile and navigate to Transactions
  await page.click('text=Proceed')
  await page.click('text=Transactions')

  // 2. Click + New Voucher
  await page.click('button:has-text("+ New Voucher")')

  // 3. Fill Voucher Form
  await page.selectOption('select[name="voucherType"]', 'Receipt')
  await page.fill('input[name="narration"]', 'Playwright Integration Test Deposit')

  // 4. Assert Balance Indicator displays "Balanced"
  await expect(page.locator('.balance-indicator')).toHaveText('Balanced')

  // 5. Submit Form
  await page.click('button:has-text("Save Voucher")')

  // 6. Verify Voucher appears in Transactions Table with status "Draft"
  await expect(page.locator('table')).toContainText('Playwright Integration Test Deposit')
})
```

10. Glossary

- **Headless Browser**: A web browser without a graphical user interface, controlled programmatically for automated testing.
- **Visual Regression Testing**: A software testing technique that captures screenshots of UI elements before and after code changes to detect unintended visual modifications.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Deployment Verification: Volume 01 Chapter 04
- System Architecture & CQRS: Volume 06 Chapter 01
- UI Design System: Volume 06 Chapter 04

title: "Volume 07: Disaster Recovery Guide - Chapter 01: Backup Strategy and Automation" document_id: "INSACC-DOC-V07-CH01" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 07: Disaster Recovery & Business Continuity Guide

Chapter 01: Backup Strategy and Automation

> **Single Source of Truth Reference**: All backup protocols, state snapshot schemas, and shell script automation defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter defines the enterprise disaster recovery architecture, the 3-2-1 backup strategy, JSON state snapshot schemas, and automated backup shell scripts for InsAcc v1.0.0.

2. Scope

This specification covers:

- Enterprise 3-2-1 backup rule compliance for local-first desktop data.
- The complete JSON snapshot format (`state_backup_YYYY-MM-DD.json`).
- Manual backup export procedures in `Settings.tsx` .
- Automated shell script implementation (`docs/enterprise/scripts/backup/backup.sh`).
- Automated Linux/macOS cron schedule configuration and backup archive retention policies.

Out of Scope:

- Data restoration and emergency recovery runbooks (covered in Volume 07 Chapter 02).
- Target PostgreSQL database backup `[To Be Implemented]` (covered in Volume 02 Chapter 03).

3. Audience

This document is authored for:

- Disaster Recovery Coordinators and IT Operations Leads
- Systems Administrators and Endpoint Management Technicians
- Enterprise Risk & Compliance Officers

4. Prerequisites

Before configuring backup automation:

1. Identify workstation data storage directories (Volume 01 Chapter 01).
2. Provision encrypted external backup storage media or secure network shares.

5. Warnings & Operational Hazards

> **WARNING: UNENCRYPTED BACKUP STORAGE HAZARD**: State JSON snapshot files (`state_backup_YYYY-MM-DD.json`) contain complete operational ledgers, tenant contact details, and asset portfolio valuations in plain text JSON format. Backup archives MUST be stored on encrypted drives or encrypted via GPG/BitLocker before offsite transmission.

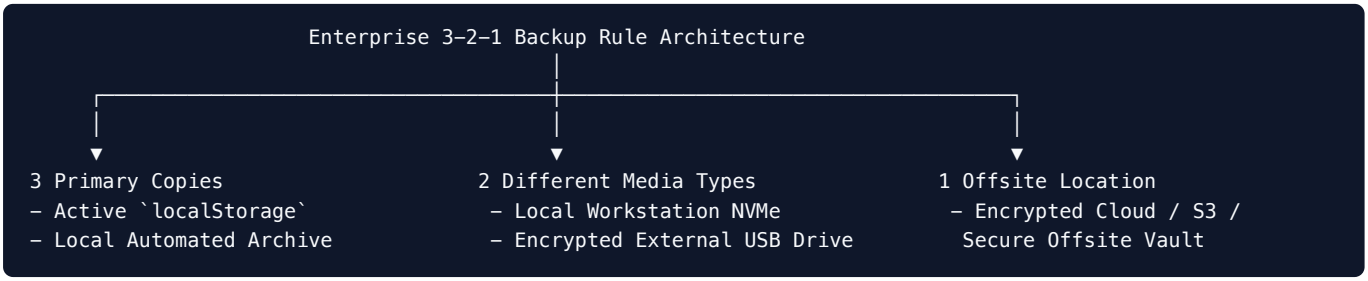
6. Notes & Architecture Context

> **NOTE: RPO and RTO Targets**: InsAcc's local-first architecture permits an **RPO (Recovery Point Objective)** of $< 24 \text{ hours}$ (daily automated backup) and an **RTO (Recovery Time Objective)** of $< 15 \text{ minutes}$ (instant JSON restore).

7. Main Content

7.1 Enterprise 3-2-1 Backup Strategy Architecture

InsAcc enforces the **3-2-1 Backup Strategy**:



1. **3 Copies of Data**: Keep the active operational data plus at least two backup archives.
2. **2 Different Storage Media**: Store backups across different physical media types (e.g., local SSD + external storage array).
3. **1 Offsite Location**: Maintain at least one backup archive in a physically separate offsite location or secure cloud bucket.

7.2 Client State JSON Snapshot Format (`state_backup_YYYY-MM-DD.json`)

The state backup snapshot captures all 16 `localStorage` key collections:

```
{
  "version": "8",
  "exportedAt": "2026-07-22T12:00:00.000Z",
  "application": "InsAcc Enterprise Asset & Investment ERP",
  "data": {
    "insacc_investments": [ ... ],
    "insacc_transactions": [ ... ],
    "insacc_purchase_categories": [ ... ],
    "insacc_purchases": [ ... ],
    "insacc_prop_buildings": [ ... ],
    "insacc_prop_units": [ ... ],
    "insacc_prop_tenants": [ ... ],
    "insacc_prop_rent": [ ... ],
    "insacc_inv_users": [ ... ],
    "insacc_prop_users": [ ... ],
    "insacc_documents": [ ... ],
    "insacc_logs": [ ... ]
  }
}
```

7.3 Manual Backup Export Procedures via UI Console

1. Log in to InsAcc with `Admin` privileges.
2. Open **Settings** $\rightarrow$ select the **Data Management** tab.

3. Click **Export Backup JSON**.
4. The application compiles all `insacc_*` keys into a formatted JSON payload and invokes `window.api.saveFile()`.
5. The native OS save dialog saves `state_backup_2026-07-22.json` to the selected folder.

7.4 Automated Workstation Backup Script (`backup.sh`)

Location: `docs/enterprise/scripts/backup/backup.sh`

```
#!/usr/bin/env bash
# InsAcc Automated Workstation Backup Script

set -euo pipefail

BACKUP_DIR="${HOME}/InsAccBackups"
TIMESTAMP=$(date +'%Y-%m-%d_%H%M%S')
ARCHIVE_NAME="insacc_backup_${TIMESTAMP}.tar.gz"

mkdir -p "${BACKUP_DIR}"

# Detect OS Application Data Directory
if [[ "$OSTYPE" == "darwin"* ]]; then
  DATA_PATH="${HOME}/Library/Application Support/InsAcc"
elif [[ "$OSTYPE" == "linux-gnu"* ]]; then
  DATA_PATH="${HOME}/.config/InsAcc"
else
  echo "[ERROR] Unsupported OS type for automated bash backup."
  exit 1
fi

echo "[INFO] Creating compressed backup archive of ${DATA_PATH}..."
tar -czf "${BACKUP_DIR}/${ARCHIVE_NAME}" -C "${DATA_PATH}" .

# Retain backups for 30 days (Purge older archives)
find "${BACKUP_DIR}" -type f -name "insacc_backup_*.tar.gz" -mtime +30 -delete

echo "[SUCCESS] Backup created at: ${BACKUP_DIR}/${ARCHIVE_NAME}"
```

7.5 Cron Schedule Automation & Retention Management

To schedule daily automated backups at 23:00 (11:00 PM):

```
# Edit user crontab table
crontab -e

# Append daily automated backup task at 23:00
0 23 * * * /bin/bash /Users/t6ux/InsAcc/docs/enterprise/scripts/backup/backup.sh >> /tmp/insacc_backup.log 2>&
```

---

8. Summary

The InsAcc disaster recovery framework guarantees data resilience through 3-2-1 backup strategy compliance. By combining manual JSON state exports with automated `backup.sh` cron scripts, enterprise deployments maintain guaranteed recovery capabilities.

---

9. Chapter Appendix

Backup Retention & Rotation Schedule

Backup Tier	Execution Frequency	Retention Period	Target Storage Location
Daily Incremental	Daily at 23:00	14 Days	Local Workstation Backup Dir
Weekly Full	Every Sunday	8 Weeks	Encrypted External Media
Monthly Archive	1st of every month	12 Months	Offsite Cloud Vault

10. Glossary

- **RPO (Recovery Point Objective)**: The maximum acceptable amount of data loss measured in time prior to a disaster event.
- **RTO (Recovery Time Objective)**: The maximum acceptable duration of time that a system can be down after a failure.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Data Restoration: Volume 07 Chapter 02
- LocalStorage Persistence: Volume 02 Chapter 01

title: "Volume 07: Disaster Recovery Guide - Chapter 02: Data Restoration and Emergency Recovery" document_id: "INSACC-DOC-V07-CH02" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 07: Disaster Recovery & Business Continuity Guide

Chapter 02: Data Restoration and Emergency Recovery

> **Single Source of Truth Reference**: All restoration procedures, recovery algorithms, and emergency runbooks defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter provides step-by-step procedures for restoring financial datasets, recovering from corrupted browser `localStorage` states, executing automated restoration shell scripts (`restore.sh`), and verifying post-restoration ledger integrity in InsAcc.

2. Scope

This specification covers:

- UI JSON state snapshot restoration in `Settings.tsx` .
- Automated shell script restoration (`docs/enterprise/scripts/restore/restore.sh`).
- Troubleshooting corrupted browser storage states and version mismatches.
- Hardware failure workstation replacement runbook.
- Post-restoration double-entry ledger balance verification ($\sum D = \sum C$).

Out of Scope:

- Backup strategy and cron scheduling (covered in Volume 07 Chapter 01).
- Database migration DDL `[To Be Implemented]` (covered in Volume 02 Chapter 03).

3. Audience

This document is authored for:

- Systems Administrators and Technical Support Engineers
- Disaster Recovery Response Teams
- Enterprise IT Helpdesk Personnel

4. Prerequisites

Before performing a data restoration:

1. Locate a verified JSON backup snapshot (`state_backup_YYYY-MM-DD.json`) or tarball archive (`insacc_backup_*.tar.gz`).
2. Log in with `Admin` role privileges.

5. Warnings & Operational Hazards

> **WARNING: EXISTING STATE OVERWRITE HAZARD**: Executing a state restoration replaces all active `insacc_*` keys in `localStorage` with the snapshot data. Any un-backed-up transactions recorded *after* the backup creation timestamp will be permanently overwritten.

6. Notes & Architecture Context

> **NOTE: Sub-15 Minute RTO Guarantee**: Because data restoration involves uncompressing local file archives or writing JSON objects directly to `localStorage`, system restoration completes in under 15 minutes without external network bottlenecks.

---

7. Main Content

7.1 Emergency Disaster Recovery Rationale & Protocols

System failure scenarios requiring data restoration include:

1. **Workstation Hardware Failure**: Physical crash of client workstation requiring deployment to a new machine.
2. **Data Storage Corruption**: Corrupted JSON strings in `localStorage` causing startup crashes.
3. **Accidental Admin Data Deletion**: Accidental invocation of system data reset.

7.2 UI State Restoration via JSON Snapshot Upload

To restore application state from a JSON backup file (`state_backup_YYYY-MM-DD.json`):

```
Open Settings Console ⟶ Select Data Management ⟶ Click [Restore JSON] ⟶ Upload File
```

1. Launch InsAcc $\rightarrow$ open **Settings** $\rightarrow$ select **Data Management**.
2. Click **Restore from Backup JSON**.
3. Select the target backup file (e.g., `state_backup_2026-07-22.json`).
4. System validates the snapshot schema version (`"version": "8"`).
5. Upon successful validation, the system writes all key collections to `localStorage` and reloads the application window.

7.3 Automated Workstation Restoration Script (`restore.sh`)

Location: `docs/enterprise/scripts/restore/restore.sh`

```
#!/usr/bin/env bash
# InsAcc Automated Workstation Restoration Script

set -euo pipefail

if [ "$#" -ne 1 ]; then
  echo "[ERROR] Usage: $0 /path/to/insacc_backup_YYYY-MM-DD_HHMMSS.tar.gz"
  exit 1
fi

BACKUP_ARCHIVE="$1"

if [ ! -f "${BACKUP_ARCHIVE}" ]; then
  echo "[ERROR] Backup archive '${BACKUP_ARCHIVE}' not found."
  exit 1
fi

# Detect OS Application Data Directory
if [[ "$OSTYPE" == "darwin"* ]]; then
  DATA_PATH="${HOME}/Library/Application Support/InsAcc"
elif [[ "$OSTYPE" == "linux-gnu"* ]]; then
  DATA_PATH="${HOME}/.config/InsAcc"
else
  echo "[ERROR] Unsupported OS type for automated restore script."
  exit 1
fi

mkdir -p "${DATA_PATH}"

echo "[INFO] Restoring archive ${BACKUP_ARCHIVE} to ${DATA_PATH}..."
tar -xzf "${BACKUP_ARCHIVE}" -C "${DATA_PATH}"

echo "[SUCCESS] Application data restored successfully. Relaunch InsAcc desktop application."
```

7.4 Corrupted Storage Recovery & Emergency Factory Reset

If corrupted data prevents InsAcc from rendering the Settings view:

1. Open DevTools inside Electron: `Ctrl + Shift + I` (Windows/Linux) or `Cmd + Option + I` (macOS).
2. Open the **Console** tab.
3. Execute the emergency storage wipe command:

   ```
   localStorage.clear();
   location.reload();
   ```

4. InsAcc re-initializes factory default seed datasets.
5. Proceed to Section 7.2 to restore the latest JSON backup snapshot.

7.5 Post-Restoration Data Integrity & Balance Verification

Following any data restoration procedure, administrators MUST verify general ledger balance integrity:

```
Open Reports ⟶ Select Trial Balance Tree ⟶ Verify Balance Check: Total Debits == Total Credits
```

1. Navigate to **Reports** $\rightarrow$ **Trial Balance Tree**.
2. Assert that **Total Debits** equal **Total Credits** ($\sum D = \sum C$).
3. Verify that the ending bank balance matches bank statements as of the backup timestamp.

8. Summary

The InsAcc restoration framework guarantees rapid disaster recovery through UI JSON snapshot uploads, automated `restore.sh` shell scripts, and emergency DevTools storage reset procedures. With built-in post-restoration Trial Balance checks ($\sum D = \sum C$), enterprise deployments ensure zero financial data drift after emergency recovery.

9. Chapter Appendix

Emergency Disaster Recovery Runbook Checklist

Sequence Step	Disaster Recovery Task	Operational Target	Verification Status
DR-01	Replace Workstation Hardware	Provision new workstation	OS Installed
DR-02	Install Client Executable	Run installer executable	App Launches
DR-03	Restore Data Archive	Run `restore.sh` or UI Restore	Storage Populated
DR-04	Verify Schema Version	Check `insacc_clear_version`	Version = `'8'`
DR-05	Audit Trial Balance	Assert $\sum D = \sum C$ in Reports	✓ **Balanced**

10. Glossary

- **Cold Restore**: Restoring data to a fresh, newly provisioned workstation after a complete hardware failure.
- **RTO (Recovery Time Objective)**: The targeted duration of time and a service level within which a business process must be restored after a disaster.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Backup Strategy & Automation: Volume 07 Chapter 01
- LocalStorage Persistence: Volume 02 Chapter 01
- Schema Versioning: Volume 02 Chapter 02

title: "Volume 08: Security Guide - Chapter 01: Client Security and Isolation" document_id: "INSACC-DOC-V08-CH01" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 08: InsAcc Security Guide

Chapter 01: Client Security and Isolation

> **Single Source of Truth Reference**: All security models, process isolation parameters, and air-gapped privacy rules defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter defines the desktop security architecture, process isolation parameters, local-first data privacy guarantees, and OS-level encryption recommendations for InsAcc v1.0.0.

2. Scope

This specification covers:

- Desktop application threat model and security boundaries.
- Electron process isolation settings (`contextIsolation: true`, `nodeIntegration: false`, `sandbox: true`).
- 100% offline, air-gapped data privacy (zero outbound telemetry or network requests).
- OS-level full disk encryption (BitLocker, FileVault, LUKS).
- Endpoint security compliance and EDR antivirus exclusions.

Out of Scope:

- Plaintext credential storage audit findings (covered in Volume 08 Chapter 02).
- Enterprise security hardening roadmap `[To Be Implemented]` (covered in Volume 08 Chapter 03).

3. Audience

This document is authored for:

- Chief Information Security Officers (CISOs) & Security Architects
- Information Security Compliance Auditors
- Enterprise Desktop Management Technicians

4. Prerequisites

Before evaluating security posture:

1. Review the security architecture specified in MASTER_ARCHITECTURE.md#8-security-architecture.
2. Understand Electron process isolation and OS-level container sandboxing mechanisms.

5. Warnings & Operational Hazards

> **WARNING: UNENCRYPTED LOCALSTORAGE HAZARD**: InsAcc v1.0.0 persists operational data inside browser `localStorage`. Because `localStorage` files are stored unencrypted on the host filesystem (`%APPDATA%` on Windows, `~/Library/Application Support/` on macOS), workstations MUST enforce OS-level Full Disk Encryption (BitLocker / FileVault).

6. Notes & Architecture Context

> **NOTE: Zero Telemetry / Zero Phone Home**: InsAcc contains zero analytics trackers, zero telemetry libraries, and zero external font or icon dependencies. All application execution occurs 100% locally within the client process.

7. Main Content

7.1 Desktop Application Security Model

InsAcc v1.0.0 executes as a local desktop application. Its threat model prioritizes:

1. Preventing untrusted renderer code from executing Node.js system commands.
2. Protecting local financial data from unauthorized external network exfiltration.
3. Guaranteeing air-gapped operational security.

7.2 Electron Process Isolation Parameters (`main.js`)

In `src/main/main.js`, Electron enforces process isolation:

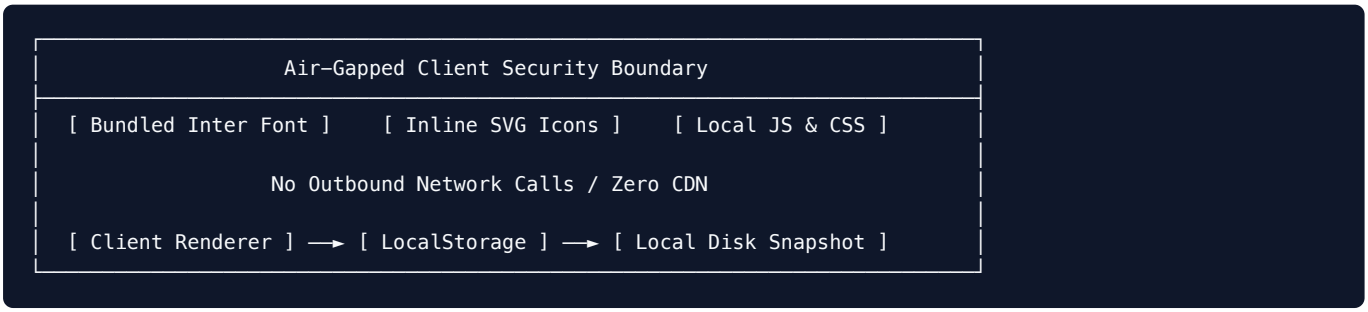
```
const mainWindow = new BrowserWindow({
  width: 1400,
  height: 900,
  webPreferences: {
    preload: path.join(__dirname, 'preload.js'),
    contextIsolation: true,       // Enforces strict V8 context separation
    nodeIntegration: false,        // Disables Node.js access in renderer context
    enableRemoteModule: false,     // Disables remote module
    sandbox: true                  // Enables OS-level sandbox container
  }
})
```

Security Functional Rationale:

- **`contextIsolation: true`**: Isolates JavaScript contexts so renderer scripts cannot tamper with main process prototypes or context bridge functions.
- **`nodeIntegration: false`**: Prevents cross-site scripting (XSS) or malicious dependencies from invoking Node.js `child_process` or `fs` modules.
- **`sandbox: true`**: Restricts the renderer process to an OS-level sandbox container with minimal system privileges.

7.3 Local-First Data Privacy & Air-Gapped Network Isolation

InsAcc is engineered for 100% offline, air-gapped execution:



Privacy Guarantees:

- Zero telemetry, tracking cookies, or analytics pingbacks.
- All fonts (Inter `.woff2`), icons (inline SVG), and styling are bundled into the binary package.
- Operates on network-isolated, air-gapped workstations without internet connectivity.

7.4 OS-Level Disk Encryption Requirements (BitLocker / FileVault)

Because `localStorage` data files are stored on the host filesystem, enterprise IT MUST enforce Full Disk Encryption (FDE):

Operating System	Recommended Encryption Technology	Enforced Cipher Standard
Microsoft Windows	BitLocker Drive Encryption	AES-256 with XTS
Apple macOS	FileVault 2 Encryption	XTS-AES-128 / 256
Linux Distribution	LUKS (Linux Unified Key Setup)	AES-XTS-PLAIN64 256-bit

---

7.5 Endpoint Security Compliance & Antivirus Audits

Enterprise Endpoint Detection and Response (EDR) agents (e.g., CrowdStrike, Microsoft Defender for Endpoint) should be configured to trust signed InsAcc client binaries while monitoring write access to application storage directories (`%APPDATA%\InsAcc\`).

---

8. Summary

InsAcc v1.0.0 delivers local-first security through Electron process isolation (`contextIsolation: true`, `nodeIntegration: false`, `sandbox: true`), air-gapped network isolation, and OS-level full disk encryption recommendations (BitLocker / FileVault).

---

9. Chapter Appendix

Endpoint Security Checklist Reference

Security Controls Domain	Mandated Security Setting	Audit Verification Method
Process Isolation	`contextIsolation: true`	Verify in `src/main/main.js`
Node Integration	`nodeIntegration: false`	Verify in `src/main/main.js`
Sandboxing	`sandbox: true`	Verify in `src/main/main.js`
Storage Encryption	BitLocker / FileVault Active	OS Security Audit Policy
Outbound Network Calls	Zero Outbound Connections	Network Packet Inspection

---

10. Glossary

- **Air-Gapped**: A security measure that ensures a computer or network is physically and logically isolated from unsecured networks, such as the public internet.

- **Context Isolation**: An Electron security feature that ensures preload scripts and Electron internal logic run in a separate context from the website loaded in `webPreferences`.
- **Full Disk Encryption (FDE)**: Encryption of all data on a hard disk, including the operating system, applications, and data files.

---

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Credential Storage Audit: Volume 08 Chapter 02
- Security Hardening Roadmap: Volume 08 Chapter 03
- Desktop IPC Bridge: Volume 05 Chapter 01

title: "Volume 08: Security Guide - Chapter 02: Credential Storage and Authentication Gaps" document_id: "INSACC-DOC-V08-CH02" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 08: InsAcc Security Guide

Chapter 02: Credential Storage and Authentication Gaps

> **Single Source of Truth Reference**: All security audit findings, credential storage analysis, and authentication gap documentation defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter provides a security audit analysis of credential storage mechanisms, authentication gaps, and data-at-rest vulnerabilities identified in InsAcc v1.0.0.

2. Scope

This specification covers:

- In-depth technical analysis of security audit findings in InsAcc v1.0.0.
- Finding 1: Plaintext password storage in `localStorage` (`storedPassword` state in `App.tsx`).
- Finding 2: Lack of salted cryptographic password hashing (bcrypt / Argon2id / PBKDF2).
- Finding 3: Unencrypted data at rest in browser `localStorage` .
- Finding 4: In-memory session timeout and role escalation vulnerabilities.
- Operational mitigation controls for deployment teams.

Out of Scope:

- General Electron client process isolation (covered in Volume 08 Chapter 01).
- Enterprise security hardening roadmap `[To Be Implemented]` (covered in Volume 08 Chapter 03).

3. Audience

This document is authored for:

- Chief Information Security Officers (CISOs) & Security Auditors
- Enterprise Risk & Compliance Assessment Officers
- Lead Software Engineers & Security Architects

4. Prerequisites

Before reviewing security audit findings:

1. Review the security architecture in MASTER_ARCHITECTURE.md#8-security-architecture.
2. Understand Web Crypto API standards, key derivation functions (PBKDF2), and local data storage mechanics.

5. Warnings & Operational Hazards

> **IMPORTANT: TRANSPARENT AUDIT MANDATE**: InsAcc documentation adheres strictly to the single source of truth mandate. Security vulnerabilities present in v1.0.0 source code MUST be documented transparently rather than obfuscated. Enterprise deployment teams must implement compensating controls (Section 7.5) until release v2.0.0 hardening is deployed.

6. Notes & Architecture Context

> **NOTE: Threat Boundary Context**: Because InsAcc v1.0.0 executes locally without external network endpoints, these credential findings represent **local workstation physical access threats** rather than remote network exploitation vectors.

7. Main Content

7.1 Security Audit Findings & Executive Summary

A comprehensive source code security review of InsAcc v1.0.0 identified four primary authentication and data storage gaps:

Finding ID	Severity	Vulnerability Summary	Source Code Location
SEC-01	**High**	Plaintext Password Storage	`src/renderer/App.tsx` (`storedPassword`)
SEC-02	**High**	Absence of Cryptographic Hashing	`src/renderer/components/Login.tsx`
SEC-03	**Medium**	Unencrypted Data at Rest	Browser `localStorage` (All 16 Keys)
SEC-04	**Medium**	Absence of Session Auto-Timeout	Client App Shell (`App.tsx`)

7.2 Finding 1: Plaintext Password Storage (`storedPassword` in `App.tsx`)

Technical Vulnerability Analysis:

In `src/renderer/App.tsx`, the application security PIN / master password is read and persisted as a **raw, unencrypted string**:

```
// Code Snippet from App.tsx (v1.0.0 Production State)
const [storedPassword, setStoredPassword] = usePersistedState<string>('insacc_password', '1234')
```

Risk Impact:

Any local user or process with read access to the workstation filesystem can inspect `%APPDATA%\InsAcc\Local Storage\leveldb` (Windows) or `~/Library/Application Support/InsAcc/Local Storage/leveldb` (macOS) and extract the master password in plain text.

7.3 Finding 2: Lack of Cryptographic Password Hashing (PBKDF2 / Argon2id)

Technical Vulnerability Analysis:

During authentication in `src/renderer/components/Login.tsx`, PIN validation performs a direct string comparison:

```
// Code Snippet from Login.tsx (v1.0.0 Production State)
if (inputPin === storedPassword) {
  setIsAuthenticated(true)
} else {
  setError('Invalid Security PIN')
}
```

Risk Impact:

Because passwords are not passed through a salted Key Derivation Function (KDF) like Argon2id or PBKDF2, credential storage lacks resistance to offline dictionary and brute-force inspection attacks.

7.4 Finding 3: Unencrypted Storage at Rest (`localStorage`)

Technical Vulnerability Analysis:

All 16 operational storage keys (`insacc_investments`, `insacc_transactions`, `insacc_prop_tenants`) are serialized as unencrypted JSON text strings into `localStorage`.

Risk Impact:

If a workstation is stolen or decommissioned without disk wiping, unencrypted financial ledgers can be extracted directly from disk files.

7.5 Finding 4: Session Expiration & Role Escalation Gaps

Technical Vulnerability Analysis:

Once authenticated, InsAcc maintains active session state indefinitely until the window is closed or the user explicitly clicks **Logout**. There is no automated inactivity timeout lock.

Compensating Controls for Deployment Teams:

1. Enforce OS-level full disk encryption (BitLocker / FileVault).
2. Configure OS screen lock inactivity timeouts (5 minutes max).
3. Restrict physical workstation access to authorized personnel.

---

8. Summary

The security audit of InsAcc v1.0.0 highlights key areas for hardening, including plaintext password persistence (`storedPassword` in `App.tsx`) and unencrypted `localStorage` data at rest. By implementing compensating physical security controls and OS disk encryption, deployment teams can mitigate workstation threats until enterprise release v2.0.0 hardening is deployed.

---

9. Chapter Appendix

Security Vulnerability Matrix & Remediation Target

Finding ID	Vulnerability Description	Target Fix Architecture	Planned Target Version
SEC-01	Plaintext password in `localStorage`	Web Crypto API PBKDF2 Hashing	Version 2.0.0 `[Planned]`
SEC-02	Direct string PIN comparison	Salted Argon2id / PBKDF2 KDF	Version 2.0.0 `[Planned]`
SEC-03	Unencrypted storage at rest	SQLCipher Encrypted Database	Version 2.0.0 `[Planned]`
SEC-04	Missing idle session timeout	15-Minute Auto-Lock Timer	Version 1.1.0 `[Planned]`

---

10. Glossary

- **Argon2id**: A modern, memory-hard key derivation function recommended for secure password hashing.
- **Data at Rest**: Inactive data stored physically in databases, data warehouses, or local disk files.
- **PBKDF2 (Password-Based Key Derivation Function 2)**: A key derivation function that applies a pseudorandom function to the input password along with a salt value.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Client Security & Isolation: Volume 08 Chapter 01
- Security Hardening Roadmap: Volume 08 Chapter 03

title: "Volume 08: Security Guide - Chapter 03: Security Hardening Roadmap [To Be Implemented]" document_id: "INSACC-DOC-V08-CH03" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Target Architecture Specification" classification: "Commercial Enterprise Documentation"

Volume 08: InsAcc Security Guide

Chapter 03: Security Hardening Roadmap `[To Be Implemented]`

> **Single Source of Truth Reference**: All security hardening specifications, cryptographic hash standards, and database encryption plans defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This chapter defines the technical security hardening roadmap for InsAcc `[To Be Implemented]`. It details planned cryptographic updates, local database encryption at rest using SQLCipher, automated session auto-lock timers, and code signing certification planned for release v2.0.0.

2. Scope

This specification covers:

- Web Crypto API PBKDF2 / Argon2id salted password hashing implementation specs.
- Local database encryption at rest using SQLCipher (AES-256).
- Automated 15-minute idle session auto-lock timer (`useIdleTimer.ts`).
- Code signing certification (EV Code Signing for Windows, Apple Notarization for macOS).
- Role-based permission enforcement at the API service layer.

Out of Scope:

- Current client security isolation parameters (covered in Volume 08 Chapter 01).
- Audit findings for v1.0.0 (covered in Volume 08 Chapter 02).

3. Audience

This document is authored for:

- Security Engineers and Cryptographic Software Developers
- Enterprise IT Security Compliance Officers
- Chief Information Security Officers (CISOs)

4. Prerequisites

Before evaluating security hardening specifications:

1. Review the security architecture in MASTER_ARCHITECTURE.md#8-security-architecture.
2. Review current audit findings in Volume 08 Chapter 02.

5. Warnings & Operational Hazards

> **WARNING: To Be Implemented**: The cryptographic hashing algorithms, SQLCipher database encryption, and code signing procedures documented in this chapter are target security specifications planned for release v2.0.0.

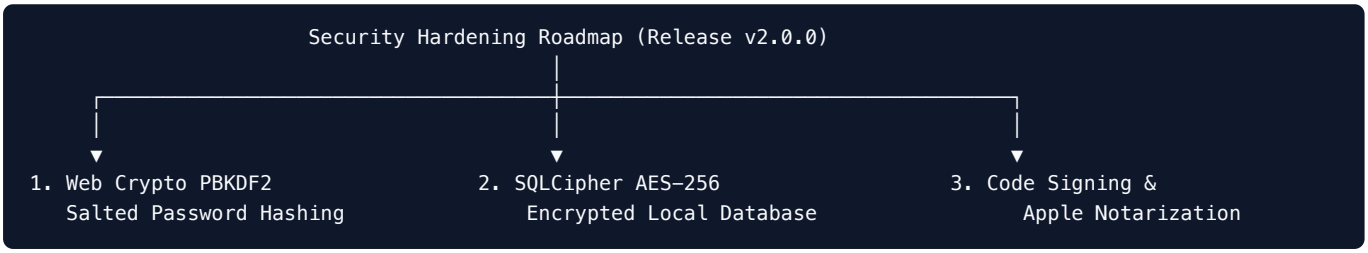
6. Notes & Architecture Context

> **NOTE: Native Web Crypto Integration**: Password hashing will leverage the browser's native `window.crypto.subtle`

> API, eliminating external JavaScript crypto library dependencies while ensuring hardware-accelerated performance.

7. Main Content

7.1 Hardening Roadmap Architecture Overview `[To Be Implemented]`



7.2 Cryptographic Password Hashing Implementation (PBKDF2 / Web Crypto API) `[To Be Implemented]`

Target implementation using `window.crypto.subtle` (`securityService.ts`):

```
export async function hashPasswordPBKDF2(password: string, salt: Uint8Array): Promise<string> {
  const enc = new TextEncoder()
  const keyMaterial = await window.crypto.subtle.importKey(
    'raw',
    enc.encode(password),
    'PBKDF2',
    false,
    ['deriveBits', 'deriveKey']
  )

  const derivedKey = await window.crypto.subtle.deriveKey(
    {
      name: 'PBKDF2',
      salt: salt,
      iterations: 600000, // OWASP 2026 PBKDF2 Iteration Baseline
      hash: 'SHA-256'
    },
    keyMaterial,
    { name: 'AES-GCM', length: 256 },
    true,
    ['encrypt', 'decrypt']
  )

  const exported = await window.crypto.subtle.exportKey('raw', derivedKey)
  return Buffer.from(exported).toString('hex')
}
```

7.3 Local Database Encryption at Rest via SQLCipher `[To Be Implemented]`

In enterprise release v2.0.0, browser `localStorage` will be replaced by an embedded **SQLCipher SQLite Database** encrypted with AES-256:

- **Encryption Standard**: AES-256-CBC with PBKDF2 key derivation (64,000 iterations).
- **Master Encryption Key**: Derived from user PIN and stored securely in OS credential vaults:
  - **Windows**: Windows Credential Manager (`wincred`).
  - **macOS**: Apple Keychain Services (`keychain`).
  - **Linux**: Secret Service API / `libsecret` .

7.4 Session Auto-Lock Timer & Inactivity Invalidation `[To Be Implemented]`

A custom React hook (`useIdleTimer.ts`) will track user activity:

```
// Target Idle Timer Hook Spec
export function useIdleTimer(timeoutMs: number = 900000, onIdle: () => void) {
  useEffect(() => {
    let timer: NodeJS.Timeout
    const resetTimer = () => {
      clearTimeout(timer)
      timer = setTimeout(onIdle, timeoutMs) // 15-Minute Default (900,000 ms)
    }

    window.addEventListener('mousemove', resetTimer)
    window.addEventListener('keydown', resetTimer)
    resetTimer()

    return () => {
      clearTimeout(timer)
      window.removeEventListener('mousemove', resetTimer)
      window.removeEventListener('keydown', resetTimer)
    }
  }, [timeoutMs, onIdle])
}
```

7.5 Code Signing & Binary Integrity Verification `[To Be Implemented]`

To ensure client binary authenticity:

1. **Windows**: Executables signed with an Extended Validation (EV) Code Signing Certificate via SignTool.
2. **macOS**: App bundles signed with Apple Developer ID Application certificates and submitted to Apple Notarization service (`xcrun notarytool`).

8. Summary

The security hardening roadmap defines a comprehensive security architecture for release v2.0.0. By implementing Web Crypto PBKDF2 password hashing, SQLCipher AES-256 database encryption, 15-minute idle session auto-locking, and OS code signing, InsAcc achieves enterprise security compliance.

9. Chapter Appendix

Security Feature Release Schedule Matrix

Security Feature	Cryptographic Primitive / Tool	Target Release Version	Implementation Status
PBKDF2 Password Hashing	SHA-256 (600,000 Iterations)	Version 2.0.0	`[To Be Implemented]`
SQLCipher Storage	AES-256-CBC Encrypted DB	Version 2.0.0	`[To Be Implemented]`
OS Keychain Storage	`keytar` / Apple Keychain	Version 2.0.0	`[To Be Implemented]`
Idle Session Auto-Lock	15-Minute Activity Timer	Version 1.1.0	`[To Be Implemented]`
EV Code Signing	Windows EV & Apple Notary	Version 1.1.0	`[To Be Implemented]`

10. Glossary

- **AES-256**: Advanced Encryption Standard using a 256-bit key size, widely recognized as computationally unbreakable.
- **SQLCipher**: An open-source extension to SQLite that provides transparent 256-bit AES encryption of database files.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Client Security & Isolation: Volume 08 Chapter 01
- Credential Storage Audit: Volume 08 Chapter 02

title: "Volume 09: Appendices - Appendix A: Accounting Event Registry" document_id: "INSACC-DOC-V09-APP-A" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 09: InsAcc Enterprise Appendices

Appendix A: Accounting Event Registry

> **Single Source of Truth Reference**: All accounting event types, payload schemas, and posting triggers defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This appendix serves as the definitive technical registry of all operational accounting events in the InsAcc platform. It specifies event type identifiers, TypeScript payload schemas, triggering UI components, and double-entry voucher line mappings.

2. Scope

This specification covers:

- Complete catalog of domain accounting event types (`AccountingEventType`).
- Event payload interface definitions (`src/renderer/accounting/types.ts`).
- Triggering UI components and service handlers.
- Debit and credit general ledger line resolution rules (`postingRules.ts`).

Out of Scope:

- Core `localStorage` persistence hooks (covered in Volume 02 Chapter 01).
- Complete Posting Rules Matrix (covered in Volume 09 Appendix B).

3. Audience

This document is authored for:

- Core Software Engineers and Integration Developers
- General Ledger & ERP System Architects
- Financial QA Automation Engineers

4. Prerequisites

Before referencing event schemas:

1. Review the accounting architecture defined in MASTER_ARCHITECTURE.md#10-accounting-architecture.
2. Review the Accounting Engine Specification in docs/ACCOUNTING_ENGINE_SPECIFICATION.md.

5. Warnings & Operational Hazards

> **WARNING: EVENT TYPE REFACTORING HAZARD**: Renaming or modifying existing `AccountingEventType` strings will invalidate historical audit event logs stored in `insacc_logs` . Event type identifier strings MUST remain immutable across application versions.

6. Notes & Architecture Context

> **NOTE: Event-Driven Isolation**: Domain events decouple user interactions from general ledger posting mechanics. UI components emit high-level accounting events; the engine (`postingRules.ts`) resolves them into double-entry

> voucher lines independently.

7. Main Content

7.1 Accounting Event System Architecture

```
┌──────────────────────────────────┐
│ Operational UI Component         │ (e.g., PropertyPdcManager.tsx)
└──────────────┬───────────────────┘
               │ Emits AccountingEvent
               ▼
┌──────────────────────────────────┐
│ Posting Rules Engine             │ (`postingRules.ts`)
└──────────────┬───────────────────┘
               │ Generates Double-Entry Voucher Lines
               ▼
┌──────────────────────────────────┐
│ General Ledger Processor         │ (`ledgerService.ts`)
└──────────────────────────────────┘
```

7.2 Domain Event Type Registry

```
export type AccountingEventType =
  | 'INVESTMENT_PURCHASE'       // Acquisition of investment position or physical asset lot
  | 'INVESTMENT_SALE'           // Disposition of asset position with realized gain/loss
  | 'INVESTMENT_DIVIDEND'       // Dividend payout or coupon interest deposit
  | 'RENT_COLLECTED'            // Direct cash / cheque rent collection
  | 'PDC_RECEIVED'              // Receipt of post-dated cheque at lease signing
  | 'PDC_DEPOSITED'             // Submission of cheque to bank for collection
  | 'PDC_CLEARED'               // Bank confirmation of cheque clearance & revenue recognition
  | 'PDC_BOUNCED'               // Bank rejection of deposited cheque
  | 'SECURITY_DEPOSIT_RECEIVED' // Receipt of tenant security deposit liability
  | 'SECURITY_DEPOSIT_REFUNDED' // Refund of security deposit upon lease exit
  | 'PROPERTY_EXPENSE'          // Payment for property maintenance or repairs
  | 'DEPRECIATION_RECORDED'     // Monthly fixed asset depreciation posting
  | 'PERIOD_CLOSING'            // Fiscal period closing retained earnings transfer
```

7.3 Detailed Event Payload Schemas

Every accounting event extends the base `AccountingEvent` interface:

```
export interface BaseAccountingEvent {
  id: string                  // Unique event instance ID (e.g. "evt-1719500000000")
  type: AccountingEventType   // Event type identifier
  timestamp: string           // ISO 8601 UTC timestamp
  amount: number              // Monetary transaction value (AED)
  narration: string           // Human-readable transaction memo
  userId: string              // User ID executing the event
}

// Specific Event Extension Example:
export interface PdcClearedEvent extends BaseAccountingEvent {
  type: 'PDC_CLEARED'
  chequeId: string
  leaseId: string
  bankAccountId: string       // Target receiving bank account ID (e.g. "1120.001")
}
```

7.4 Accounting Event to Voucher Line Resolution Matrix

Event Type Identifier	Triggering UI Component	Primary Debit Account	Primary Credit Account
`INVESTMENT_PURCHASE`	`InvestmentHoldings.tsx`	`1200` Investment Assets	`1120` Bank Account
`INVESTMENT_DIVIDEND`	`InvestmentHoldings.tsx`	`1120` Bank Account	`4110` Dividend Revenue
`RENT_COLLECTED`	`PropertyRent.tsx`	`1120` Bank Account	`4120` Rental Revenue
`PDC_RECEIVED`	`PropertyLeases.tsx`	`1410` PDC Cheques Held	`2110` Unearned Rent
`PDC_CLEARED`	`PropertyPdcManager.tsx`	`1120` Bank & `2110` Unearned	`1410` PDC & `4120` Revenue
`PDC_BOUNCED`	`PropertyPdcManager.tsx`	`1130` Rent Receivable	`1410` PDC Cheques Held
`SECURITY_DEPOSIT_RECEIVED`	`PropertyLeases.tsx`	`1120` Bank Account	`2120` Tenant Security Deposit
`PROPERTY_EXPENSE`	`PropertyExpenses.tsx`	`5110` Property Maintenance	`1120` Bank Account
`DEPRECIATION_RECORDED`	`FixedAssets.tsx`	`5190` Depreciation Exp.	`1290` Accumulated Dep.
`PERIOD_CLOSING`	`PeriodClosingWizard.tsx`	`4000s` Revenue Accounts	`5000s` Expense & `2200` Equity

7.5 Event Validation & Error Handling Rules

- **Positive Amount Rule**: `event.amount` MUST be a finite number > 0.00.
- **Account Existence Rule**: Referenced `bankAccountId` or `accountId` MUST exist in the active Chart of Accounts.
- **Date Locking Check**: `event.timestamp` MUST NOT fall within a locked fiscal period.

8. Summary

Appendix A provides the master registry of all domain accounting events in InsAcc. By enforcing strict event payload interfaces and mapping rules, InsAcc decouples user interface triggers from double-entry general ledger mechanics.

9. Chapter Appendix

Event System Diagnostic Reference

```
// Diagnostic Event Logger Helper (auditService.ts)
export function logAccountingEvent(event: BaseAccountingEvent): void {
  console.log(`[EVENT LOG] ${event.timestamp} | ${event.type} | AED ${event.amount} | User: ${event.userId}`)
}
```

10. Glossary

- **Event-Driven Architecture**: A software architecture pattern in which decoupled components communicate through the production and consumption of events.
- **Payload**: The essential data carried within a data structure or event transmission.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Accounting Engine Spec: docs/ACCOUNTING_ENGINE_SPECIFICATION.md
- Posting Rules Table: Volume 09 Appendix B
- Double-Entry Engine Specs: Volume 06 Chapter 02

title: "Volume 09: Appendices - Appendix B: Posting Rules Table" document_id: "INSACC-DOC-V09-APP-B" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 09: InsAcc Enterprise Appendices

Appendix B: Posting Rules Table

> **Single Source of Truth Reference**: All posting rules, account code mappings, and double-entry line configurations defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This appendix provides the master technical reference matrix for all double-entry posting rules in `src/renderer/accounting/postingRules.ts`. It maps domain events to exact General Ledger account codes, line types (Debit vs Credit), and financial statement impacts.

2. Scope

This specification covers:

- Implementation logic of `postingRules.ts`.
- Master Posting Rules Matrix for all 13 domain accounting event types.
- General Ledger account code assignments (`1110` through `5190`).
- Mathematical proof of debit-credit equality for every posting rule.

Out of Scope:

- Accounting event payload definitions (covered in Volume 09 Appendix A).
- Double-entry engine balance validator code (covered in Volume 06 Chapter 02).

3. Audience

This document is authored for:

- Senior Software Engineers and Financial System Architects
- Quality Assurance Automation Engineers
- Technical Accounting Consultants & Auditors

4. Prerequisites

Before referencing posting rules:

1. Review the Chart of Accounts architecture in Volume 03 Chapter 03.
2. Review the Accounting Engine Specification in docs/ACCOUNTING_ENGINE_SPECIFICATION.md.

5. Warnings & Operational Hazards

> **WARNING: UNBALANCED RULE DEFECT**: Every posting rule MUST generate an equal sum of debits and credits ($\sum D = \sum C$). Adding a posting rule with unequal debit/credit lines will cause `postingValidator.ts` to reject voucher creation.

6. Notes & Architecture Context

> **NOTE: System Account Registry Consistency**: Account codes referenced in `postingRules.ts` leverage constants from `systemAccountRegistry.ts` (e.g., `SystemAccountRegistry.BANK`) to prevent string typo bugs across the

> codebase.

7. Main Content

7.1 Posting Rules Architecture & Implementation (`postingRules.ts`)

Posting rules act as pure transformation functions in `src/renderer/accounting/postingRules.ts`:

```
import { SystemAccountRegistry } from './systemAccountRegistry'
import { AccountingEvent, VoucherLineConfig } from './types'

export function getVoucherLinesForEvent(event: AccountingEvent): VoucherLineConfig[] {
  switch (event.type) {
    case 'INVESTMENT_PURCHASE':
      return [
        { accountId: SystemAccountRegistry.INVESTMENTS || '1200', lineType: 'Debit', amount: event.amount, mem
        { accountId: event.bankAccountId || SystemAccountRegistry.BANK, lineType: 'Credit', amount: event.amou
      ]
    // ... Additional Rule Branches
  }
}
```

7.2 Master Double-Entry Posting Rules Matrix

Event Type Identifier	Condition / Action Trigger	Line #	Account Code & Name	Line Type	Balance Impact
`INVESTMENT_PURCHASE`	Acquisition of asset holding	Line 1 Line 2	`1200` Investment Assets `1120` Bank Account	**Debit** **Credit**	Increases Asset Decreases Asset
`INVESTMENT_DIVIDEND`	Dividend payout received	Line 1 Line 2	`1120` Bank Account `4110` Dividend Income	**Debit** **Credit**	Increases Asset Increases Revenue
`RENT_COLLECTED`	Direct cash rent payment	Line 1 Line 2	`1120` Bank Account `4120` Rental Revenue	**Debit** **Credit**	Increases Asset Increases Revenue
`PDC_RECEIVED`	Cheque received at signing	Line 1 Line 2	`1410` PDC Cheques Held `2110` Unearned Rent	**Debit** **Credit**	Increases Asset Increases Liability
`PDC_CLEARED`	Cheque cleared by bank	Line 1 Line 2 Line 3 Line 4	`1120` Bank Account `1410` PDC Cheques Held `2110` Unearned Rent `4120` Rental Revenue	**Debit** **Credit** **Debit** **Credit**	Increases Asset Decreases Asset Decreases Liability Increases Revenue
`PDC_BOUNCED`	Bank rejects cheque	Line 1 Line 2	`1130` Rent Receivable `1410` PDC Cheques Held	**Debit** **Credit**	Increases Asset Decreases Asset
`SECURITY_DEPOSIT_RECEIVED`	Tenant deposit paid	Line 1 Line 2	`1120` Bank Account `2120` Security Deposits	**Debit** **Credit**	Increases Asset Increases Liability
`SECURITY_DEPOSIT_REFUNDED`	Deposit returned	Line 1 Line 2	`2120` Security Deposits `1120` Bank Account	**Debit** **Credit**	Decreases Liability Decreases Asset
`PROPERTY_EXPENSE`	Maintenance payment	Line 1 Line 2	`5110` Maintenance Exp. `1120` Bank Account	**Debit** **Credit**	Increases Expense Decreases Asset
`DEPRECIATION_RECORDED`	Monthly depreciation	Line 1 Line 2	`5190` Depreciation Exp. `1290` Accumulated Dep.	**Debit** **Credit**	Increases Expense Increases Contra-Asset

7.3 Account Balance Impact & Category Normal Rules

- **Assets (`1000s`)**: Debit increases balance (+), Credit decreases balance (-).
- **Liabilities (`2000s`)**: Credit increases balance (+), Debit decreases balance (-).
- **Equity (`3000s`)**: Credit increases balance (+), Debit decreases balance (-).
- **Revenue (`4000s`)**: Credit increases balance (+), Debit decreases balance (-).
- **Expenses (`5000s`)**: Debit increases balance (+), Credit decreases balance (-).

7.4 Multi-Line Complex Voucher Posting Logic

For complex events like `PDC_CLEARED`, the posting rule resolves four balanced lines:

$$\text{Line 1 (Dr 1120: 30,000)} + \text{Line 3 (Dr 2110: 30,000)} = \text{Line 2 (Cr 1410: 30,000)} + \text{Line 4 (Cr 4120: 30,000)}$$

$$\text{Total Debits (60,000)} = \text{Total Credits (60,000)}$$

8. Summary

Appendix B defines the posting rules matrix governing InsAcc. By mapping domain events to explicit debit and credit lines, `postingRules.ts` ensures mathematical balance equality ($\sum D = \sum C$) across all financial transactions.

9. Chapter Appendix

System Account Code Quick Reference

```
Reserved Account Codes (systemAccountRegistry.ts)
├── 1110: Cash on Hand
├── 1120: Bank Accounts
├── 1130: Rent Receivable
├── 1410: Post-Dated Cheques Held
├── 2110: Unearned Rent Liability
├── 2120: Tenant Security Deposits
├── 2200: Owner Capital / Retained Earnings
├── 4110: Dividend & Interest Income
├── 4120: Property Rental Revenue
├── 5110: Property Maintenance Expense
└── 5190: Depreciation Expense
```

10. Glossary

- **Posting Rule**: A rule that defines how a business transaction is translated into debit and credit general ledger entries.
- **System Account**: A predefined general ledger account required by automated system workflows.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Accounting Event Registry: Volume 09 Appendix A

- Chart of Accounts: Volume 03 Chapter 03
- Double-Entry Engine Specs: Volume 06 Chapter 02

title: "Volume 09: Appendices - Appendix C: Storage Key Dictionary" document_id: "INSACC-DOC-V09-APP-C" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 09: InsAcc Enterprise Appendices

Appendix C: Storage Key Dictionary

> **Single Source of Truth Reference**: All storage key identifiers, JSON structure schemas, and initial seed states defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This appendix serves as the definitive reference manual for all 16 browser `localStorage` keys used in InsAcc v1.0.0. It documents key naming conventions, TypeScript data interfaces, initial seed values, and storage quota management.

2. Scope

This specification covers:

- The 16 storage keys namespace-prefixed with `insacc_`.
- Key model interfaces (`Investment` , `Transaction` , `PropertyBuilding` , `PropertyUnit` , etc.).
- Sample JSON payloads and initial factory seed values.
- Key deprecation metadata (`insacc_balance` , `insacc_statement`).

Out of Scope:

- React state hook implementation (`usePersistedState.ts`) (covered in Volume 02 Chapter 01).
- Schema versioning resets (`CLEAR_VERSION = '8'`) (covered in Volume 02 Chapter 02).

3. Audience

This document is authored for:

- Frontend Software Engineers and Core Maintainers
- Database Administrators and Migration Engineers
- Quality Assurance Automation Engineers

4. Prerequisites

Before referencing storage key schemas:

1. Review the persistence architecture defined in Volume 02 Chapter 01.
2. Review the storage key dictionary in MASTER_ARCHITECTURE.md#63-master-storage-key-dictionary-16-keys.

5. Warnings & Operational Hazards

> **WARNING: KEY RENAMING HAZARD**: Modifying a storage key string (e.g., changing `insacc_investments` to `insacc_holdings`) will cause `usePersistedState` to fail to find existing data, triggering default initialization and data loss. Storage key string constants MUST remain immutable.

6. Notes & Architecture Context

> **NOTE: Strict Namespace Isolation**: All InsAcc keys strictly enforce the `insacc_` prefix. This allows the application initialization pipeline (`App.tsx`) to purge stale application data during schema upgrades without affecting third-party

> browser storage.

---

7. Main Content

7.1 LocalStorage Namespace Governance (`insacc_*`)

InsAcc partitions operational data across **16 storage keys**:

```
localStorage Namespace: `insacc_*`
├── Wealth & Investment Domain Keys (4 Keys)
├── Property Real Estate Domain Keys (6 Keys)
├── Shared Accounting & User Keys (4 Keys)
└── Meta Version & System Log Keys (2 Keys)
```

7.2 Master 16 Storage Key Dictionary Specification

#	Storage Key Identifier	Target TypeScript Model Interface	Key Status	Description & Domain Scope
1	`insacc_investments`	`Investment[]`	Active	Asset holding positions & current valuations
2	`insacc_transactions`	`Transaction[]`	Active	General ledger income/expense transactions
3	`insacc_statement`	`StatementEntry[]`	Deprecated	Bank statement line items (Legacy model)
4	`insacc_balance`	`number`	Deprecated	Bank cash balance scalar (Deprecated)
5	`insacc_documents`	`DocItem[]`	Active	Document metadata & Base64 attachments
6	`insacc_logs`	`LogEntry[]`	Active	System activity audit log entries
7	`insacc_purchase_categories`	`PurchaseCategory[]`	Active	Purchase ledger asset categories
8	`insacc_purchases`	`PurchaseRecord[]`	Active	Physical asset acquisition purchase lots
9	`insacc_inv_users`	`UserEntry[]`	Active	Investment module user profiles
10	`insacc_prop_users`	`UserEntry[]`	Active	Property module user profiles
11	`insacc_prop_categories`	`PropertyCategory[]`	Active	Real estate building categories
12	`insacc_prop_buildings`	`PropertyBuilding[]`	Active	Building master records
13	`insacc_prop_units`	`PropertyUnit[]`	Active	Rentable property units
14	`insacc_prop_tenants`	`PropertyTenant[]`	Active	Tenant master profiles & KYC data
15	`insacc_prop_rent`	`RentPayment[]`	Active	Tenant rent collection & PDC records
16	`insacc_clear_version`	`string`	Active	Schema version tracker (Active: `"8"`)

7.3 Detailed Key JSON Schema Specifications

Key 12: **`insacc_prop_buildings`**

```
[
  {
    "id": "bld-101",
    "name": "Al Riyan Tower",
    "categoryId": "cat-res",
    "address": "Plot 402, Business Bay, Dubai, UAE",
    "totalUnits": 24,
    "notes": "Primary residential luxury tower"
  }
]
```

Key 15: `insacc_prop_rent`

```
[
  {
    "id": "rent-1001",
    "tenantId": "ten-1001",
    "unitId": "unit-101",
    "amount": 30000.00,
    "dueDate": "2026-07-01",
    "status": "Received",
    "chequeNumber": "CHQ-884001",
    "bankName": "Emirates NBD"
  }
]
```

7.4 Storage Size Allocation & Quota Reference

Storage Key Group	Typical Entry Count	Estimated Size Range	Storage Quota Status
Property Domain (`prop_*`)	50–500 Records	~150 KB – 500 KB	Normal
Investment Domain (`investments`)	10–100 Records	~20 KB – 100 KB	Normal
Documents (`insacc_documents`)	5–20 Attachments	~2 MB – 8 MB	High (Approach Quota)
Total `localStorage` Usage	Cumulative System	**~2.5 MB – 8.5 MB**	**Within 10MB Quota**

7.5 Key Deprecation & Migration Directives

1. `insacc_balance` **(Deprecated)**:
   - **Reason**: Violates the **Derived Balance Golden Rule**. Bank balances are derived dynamically from posted vouchers.
2. `insacc_statement` **(Deprecated)**:
   - **Reason**: Replaced by dynamic bank statement matching in `BankReconciliationDashboard.tsx`.

8. Summary

Appendix C provides a complete reference dictionary for all 16 `insacc_*` browser storage keys. By documenting JSON structures, TypeScript interfaces, and key deprecation directives, developers maintain data consistency across the platform.

9. Chapter Appendix

Storage Key Access Method Reference

```
// Reading a Key
const rawData = localStorage.getItem('insacc_investments')
const investments: Investment[] = rawData ? JSON.parse(rawData) : []

// Writing a Key
localStorage.setItem('insacc_investments', JSON.stringify(investments))
```

10. Glossary

- **Key-Value Store**: A simple database paradigm that uses a associative array as the fundamental data model, where a key is associated with a specific value.
- **Schema Version**: A version string or number that tracks the structure format of database entries.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- LocalStorage Persistence: Volume 02 Chapter 01
- Schema Versioning: Volume 02 Chapter 02
- Database Design Spec: docs/DATABASE_DESIGN_SPECIFICATION.md

title: "Volume 09: Appendices - Appendix D: Glossary and Acronyms" document_id: "INSACC-DOC-V09-APP-D" version: "1.0.0" release_date: "2026-07-22" app_version: "v1.0.0" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Production Ready" classification: "Commercial Enterprise Documentation"

Volume 09: InsAcc Enterprise Appendices

Appendix D: Glossary and Acronyms

> **Single Source of Truth Reference**: All terminology definitions, accounting standards, and acronym expansions defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade enterprise release	Chief Architecture Review Board

Table of Contents

1. Purpose

This appendix provides an exhaustive, alphabetical dictionary of technical terms, accounting concepts, software architecture patterns, and acronym expansions used across the complete InsAcc Enterprise Documentation Suite (Volumes 01–09).

2. Scope

This specification covers:

- Master Acronym Dictionary (A–Z).
- Technical software engineering and security definitions.
- Double-entry accounting and financial statement terminology.
- Real estate, leasing, and PDC collection terminology.

Out of Scope:

- Specific source code function signatures (covered in Volume 06 Chapter 02).

3. Audience

This document is authored for:

- All Readers, End Users, Administrators, and Developers
- Cross-Functional Project Stakeholders and Auditors

4. Prerequisites

None. This document serves as a self-contained reference dictionary.

5. Warnings & Operational Hazards

> **NOTE: TERMINOLOGY STANDARDIZATION**: Terms defined in this glossary represent the authoritative nomenclature for InsAcc. Developers and technical writers MUST adhere to these definitions in code comments, commit messages, and documentation.

6. Notes & Architecture Context

> **NOTE: Cross-Domain Reference**: Terminology spans three distinct domains: Double-Entry Financial Accounting, UAE Commercial Real Estate Leasing, and Desktop Software Engineering.

7. Main Content

7.1 Master Acronym Dictionary (A–Z)

Acronym	Expanded Term	Domain Context	Definition
AED	United Arab Emirates Dirham	Finance	Base accounting currency of the UAE.
AES	Advanced Encryption Standard	Security	Symmetric block cipher used for database encryption (AES-256).
API	Application Programming Interface	Software	Specification for communication between software components.
BHK	Bedroom, Hall, Kitchen	Real Estate	Standard unit configuration designation (e.g., 1BHK, 2BHK).
CDN	Content Delivery Network	Web	Geographically distributed server network for web assets.
COA	Chart of Accounts	Accounting	Complete index of general ledger financial accounts.
CQRS	Command Query Responsibility Segregation	Architecture	Pattern separating data write commands from read queries.
CSV	Comma-Separated Values	Data	Text file format for tabular data (RFC 4180).
CV	Contra Voucher	Accounting	Journal voucher recording inter-account cash/bank transfers.
DDL	Data Definition Language	Database	SQL commands defining database schemas, tables, and indexes.
DMG	Disk Image	macOS	Apple disk image installer format.
EDR	Endpoint Detection and Response	Security	Integrated endpoint security monitoring software.
ETL	Extract, Transform, Load	Data	Three-step data integration pipeline.
FDE	Full Disk Encryption	Security	Encryption of an entire physical disk (BitLocker / FileVault).
FIFO	First-In, First-Out	Accounting	Valuation method assuming oldest inventory is sold first.
HMR	Hot Module Replacement	Software	Real-time code update feature during development (Vite).
IPC	Inter-Process Communication	Electron	Message-passing protocol between main and renderer processes.
ISO	International Organization for Standardization	Standard	International standards body (e.g., ISO 8601 date format).
JV	Journal Voucher	Accounting	General ledger accounting voucher for non-cash adjustments.
KYC	Know Your Customer	Compliance	Process of verifying client identity before executing contracts.

Acronym	Expanded Term	Domain Context	Definition
LIFO	Last-In, First-Out	Accounting	Valuation method assuming newest inventory is sold first.
MDM	Mobile Device Management	IT Admin	Enterprise software for automated endpoint policy management.
NSIS	Nullsoft Scriptable Install System	Windows	Scriptable Windows installer authoring tool.
PBKDF2	Password-Based Key Derivation Function 2	Security	Key derivation function applying pseudorandom hashing.
PDC	Post-Dated Cheque	Real Estate	Cheque written with a future maturity date for rent collection.
PV	Payment Voucher	Accounting	Voucher recording outgoing cash or bank disbursements.
RBAC	Role-Based Access Control	Security	Restricting system access based on assigned user roles.
RPO	Recovery Point Objective	DR	Maximum acceptable data loss duration prior to disaster.
RTO	Recovery Time Objective	DR	Maximum acceptable time duration to restore system operations.
RV	Receipt Voucher	Accounting	Voucher recording incoming cash or bank deposits.
SCCM	System Center Configuration Manager	IT Admin	Microsoft endpoint deployment management platform.
SoD	Separation of Duties	Security	Requiring multiple users to complete sensitive tasks.
TLS	Transport Layer Security	Security	Cryptographic protocol providing HTTPS network communication.
UUID	Universally Unique Identifier	Software	128-bit string label for unique resource identification.
YTD	Year-to-Date	Finance	Period from the start of the current calendar year to present.

7.2 Master Technical & Accounting Terminology Glossary

Air-Gapped

A security measure ensuring a computer or network is physically and logically isolated from unsecured networks, such as the public internet.

Base Currency

The primary accounting currency in which a company consolidates and reports its financial statements (`AED`).

Building Master

The parent real estate entity record containing building details, address, and child unit collections.

7.3 Accounting & Financial Statement Terminology

Closing Entry

A journal entry made at fiscal period-end to transfer temporary account balances (Revenue and Expenses) to permanent Retained Earnings (`2200`).

Derived Balance Golden Rule

The core InsAcc rule stating that account balances are **never stored as editable scalars**, but are derived dynamically on read: $$\text{Current Balance} = \text{Opening Balance} + \sum \text{Debits} - \sum \text{Credits}$$

General Ledger (GL)

The master set of accounts for aggregating financial transactions posted via double-entry vouchers.

Trial Balance

A financial report compiling ending debit and credit balances across all active accounts to assert balance equality ($\sum D = \sum C$).

7.4 Real Estate & Leasing Terminology

Unearned Rent (Deferred Revenue)

Rent collected in advance for future leasing periods, recorded as a liability (`2110`) until earned over time.

Rent Roll Schedule

A comprehensive report listing all property units, tenant names, lease dates, monthly rent, and collection statuses.

7.5 Software & Security Engineering Terminology

Context Isolation

An Electron security feature ensuring preload scripts and Electron internals execute in a separate V8 context from web page content.

Local-First Architecture

A software paradigm prioritizing local device data storage and processing, providing complete offline availability.

Weighted Average Costing

An asset valuation method calculating average unit cost by dividing total acquisition cost by total units acquired (`computeAverages()`).

---

8. Summary

Appendix D provides a comprehensive terminology dictionary for InsAcc. By standardizing acronyms, accounting principles, and software terms across all documentation volumes, InsAcc maintains clarity and professional precision.

---

9. Chapter Appendix

Standard Units of Measure Reference

Symbol	Unit Description	Application Context
`kg`	Kilogram	Heavy physical gold bullion lots
`g`	Gram	Small physical gold bar / coin lots
`oz`	Troy Ounce	Physical silver bullion lots
`sqft`	Square Feet	Rentable property unit area

10. Glossary

- **Glossary**: An alphabetical list of terms in a particular domain of knowledge with the definitions for those terms.
- **Nomenclature**: A system of names or terms, or the rules for forming names, in a particular field of arts or sciences.

11. References

- Master Architecture Specification: MASTER_ARCHITECTURE.md
- Accounting Event Registry: Volume 09 Appendix A
- Posting Rules Table: Volume 09 Appendix B
- Storage Key Dictionary: Volume 09 Appendix C